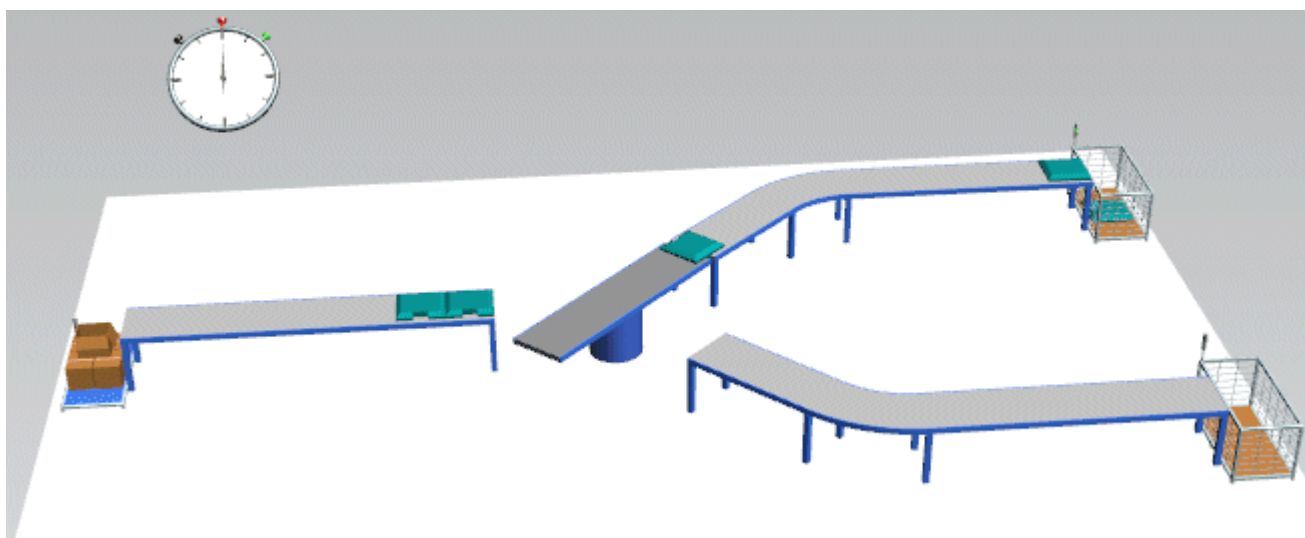




Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

Back to [Modeling the Material Flow](#)

Modeling Transport Systems

Model Transport Systems

You will use transport systems in a variety of ways in your simulation models. You might have to model a simple conveyor that transports parts between two stations, as well as complex electric overhead monorail systems or forklifts that transport parts from one location to another, as well as cranes and automated guided vehicle systems.

For this task you can use the [Conveyor](#), the [Track](#) and the [TwoLaneTrack](#), which you find in the folder **MaterialFlow** in the *Class Library*.

- The *Conveyor*  is an active material flow object that has a propulsion system of its own. It transports passive parts that do not have a propulsion system of their own.
- The *Track*  and the *TwoLaneTrack*  are passive material flow objects without a propulsion system of their own. They mainly serve as the route on which the active material flow object [Transporter](#)  drives, either forward or in reverse.

When modeling your transport system keep these characteristics of *Conveyor*, *Track* and *TwoLaneTrack* in mind:

- All three are length-oriented objects, i.e., *Plant Simulation* uses their own [Length](#), and the [MU Length](#) of the *MUs*, which you type into their dialog boxes to determine how many *MUs* they can hold at

the same time. The point-oriented material flow objects on the other hand, such as the *Station*, the *ParallelStation*, etc. do not use a length.

- You can create **Controls** and **Sensors** for them.
- Parts transfer from a point-oriented object to a length-oriented object in their entirety, i.e., they are located on *Conveyor* and *Track/TwoLaneTrack* in their full length as soon as their front has moved onto these objects.
- Parts transfer from one length-oriented object to another length-oriented object with the **Speed** which you type into the dialog box of the length-oriented object. This is of importance, when you use **Entrance Controls**.
- To model simple transport systems, you will use the active object *Conveyor*.
- To model cross-transfer systems, AGV systems, cranes, etc. you will use the passive objects *Track* or *TwoLaneTrack*.

By activating **Automatic Routing** of the moving objects *Transporter*, *Part*, and *Container* you can make sure that they always find their **Destination Object** on the shortest possible **route**.

Back to [Modeling the Material Flow](#)

Model a Transport System with Active Objects of Type Conveyor

Model a Transport System with Active Objects

When you model transport systems with active objects, such as stationary belt conveyors, roller conveyors, etc., you will use the *Conveyor*. Below we outline how you can use the *Conveyor*.

You can use the **Conveyor**  to:

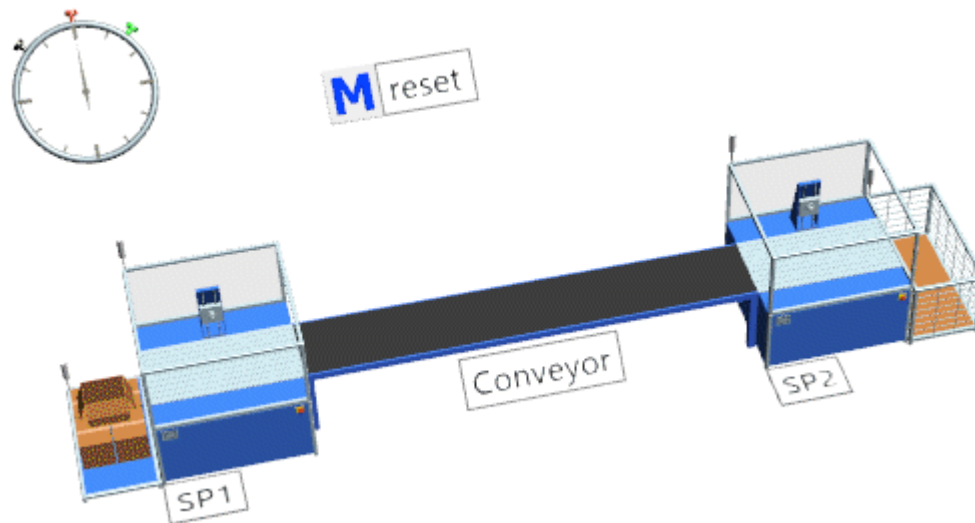
- [Model a Simple Conveyor Between Two Stations](#)
- [Model an Accumulating/a Non-Accumulating Transport System](#)
- [Model a Fixed Gap or a No Gap Conveyor](#)
- [Model a Multiple Gap or a Multiple Pitch Conveyor](#)
- [Model a Transport System with Passive Objects](#)
- [Model Transport Systems](#)

Model a Simple Conveyor Between Two Stations

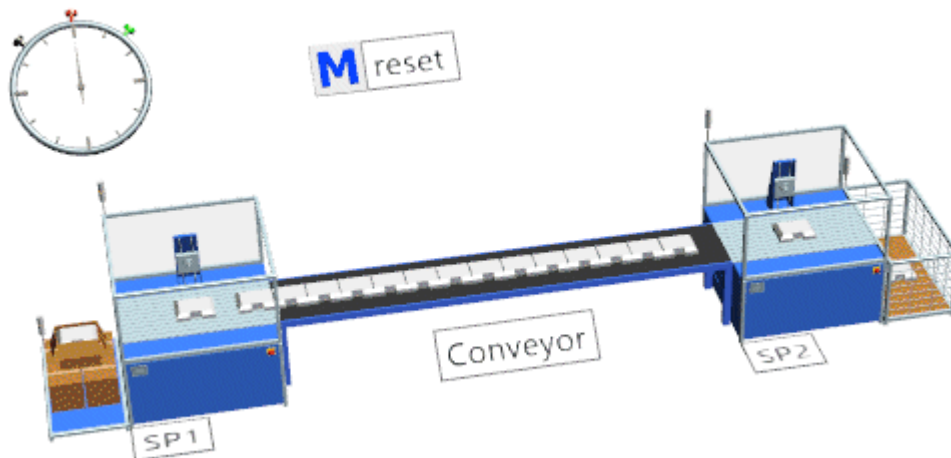
In this example we model a simple *Conveyor* between two *Station*.

In its most basic way, a transport system consists of a conveyor that moves parts between two processing stations. In our example we inserted a **Conveyor**  between two *Stations*, *SP1* and *SP2*. The *MUs* are

transferred from the point-oriented station *SP1* onto the length-oriented *Conveyor*, which then transports them to the point-oriented station *SP2*.



The *Conveyor* moves the *MUs* from the station *SP1* with the **Final Speed** we enter into the station *SP2*. *Plant Simulation* computes the **Transport Time** using the **Final Speed** and the **Length** of the *Conveyor*. **Speed**, **Length** and **Transport Time** depend on each other. When you change **Speed** or **Length**, *Plant Simulation* recomputes the **Transport Time** anew. Changing the **Transport Time** also changes the **speed**.



Back to [Model a Transport System with Active Objects](#)

Model an Accumulating/a Non-Accumulating Transport System

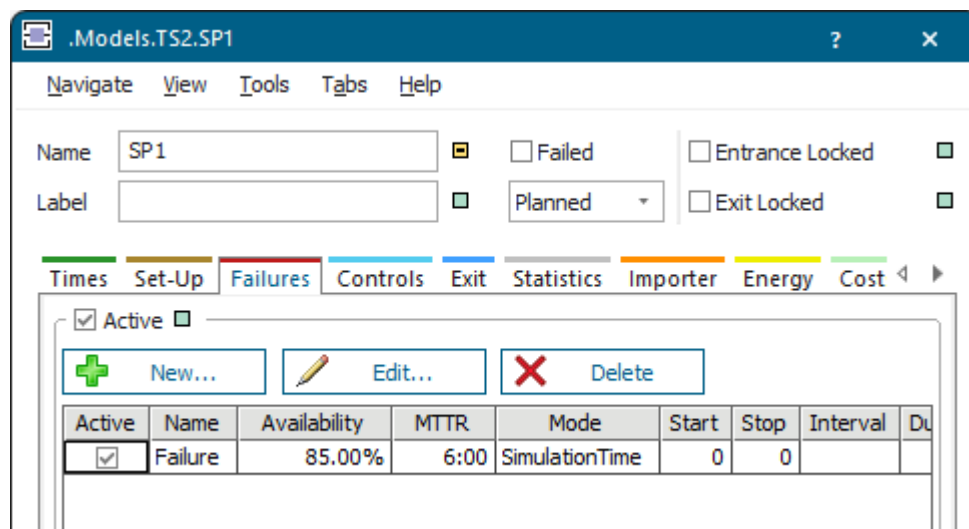
In this example we model an accumulating and a non-accumulating transport system.

You can use the [Conveyor](#)  to model an accumulating or a non-accumulating transport system between stations.

- To make the *MUs* accumulate on the *Conveyor*, select the check box **Accumulating**. This enables the *MUs* to move front to end to each other, when the exit of the *Conveyor* is blocked.
- To make the *MUs* retain their distance to each other, i.e., make all succeeding *MUs* stop moving when the preceding *MU* cannot exit, clear the check box to deactivate **Accumulating**.

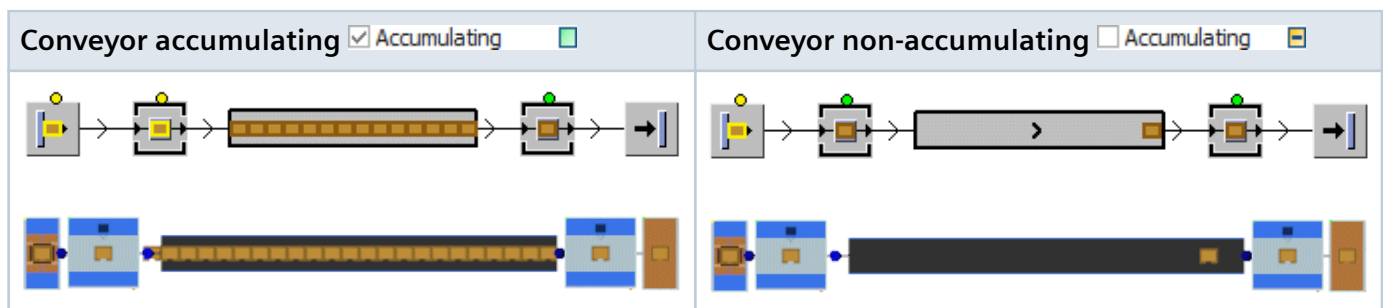
As we want to apply the default behavior of the *Conveyor*, we do not change its **Capacity** of -1, which stands for an infinite number of *MUs*. This way its capacity is defined by its own length and by the length of the parts (*MUs*), which it transports.

In our sample we entered an **Availability** of 85 percent and a **MTTR** of 6 : 00 minutes for the two *Stations*, *SP1* and *SP2*.



Plant Simulation marks a failed failure profile red.

You can use an accumulating *Conveyor* to decouple stations. Then, the *Conveyor* behaves like a roller conveyor that enables the *MUs* to move up to those in front of them, when the exit of the conveyor is blocked.



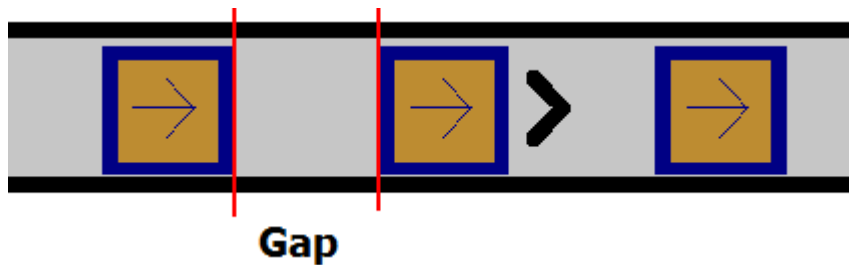
When you do not decouple the stations, the *Conveyor* behaves like a belt conveyor, meaning that the belt stops when a *MU* blocks the exit of the *Conveyor* and restarts as soon as the *MU* at the exit has moved on to the successor.

Back to [Model a Transport System with Active Objects](#)

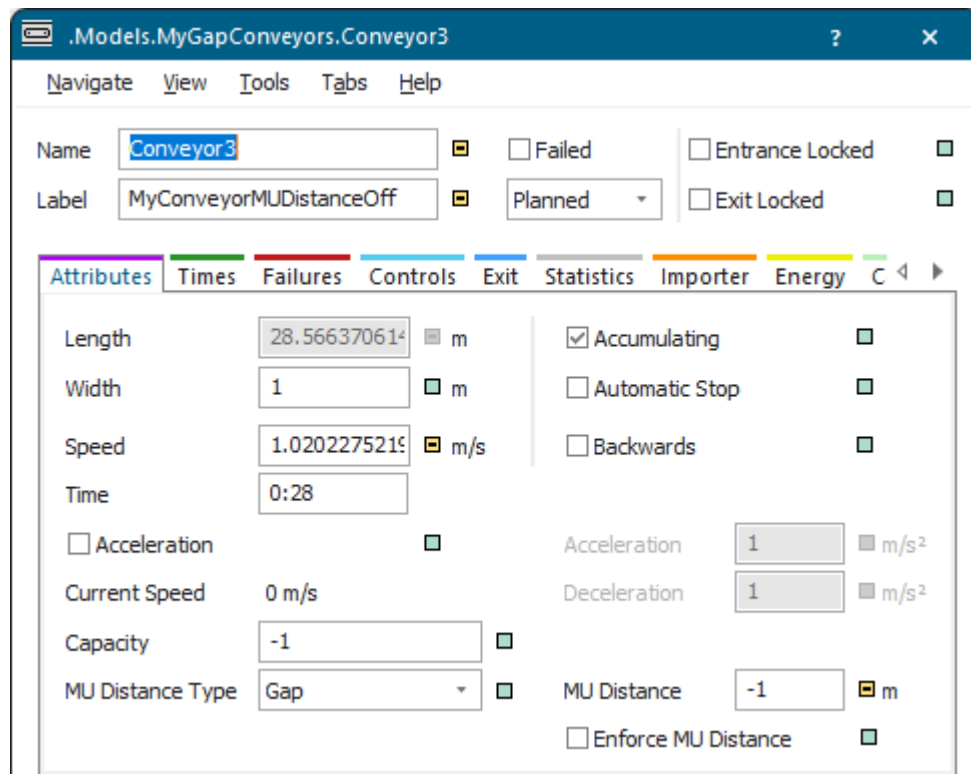
Model a Fixed Gap or a No Gap Conveyor

In this example we model a fixed gap or a no gap Conveyor.

You can use the [Conveyor](#)  to model conveyors ensuring gaps between the parts moving on it. The gap is the distance between the rear of the preceding part and the front of the succeeding part.

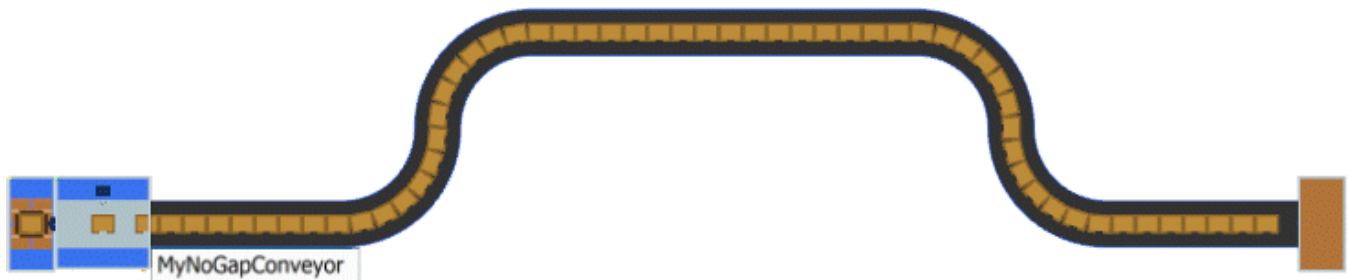
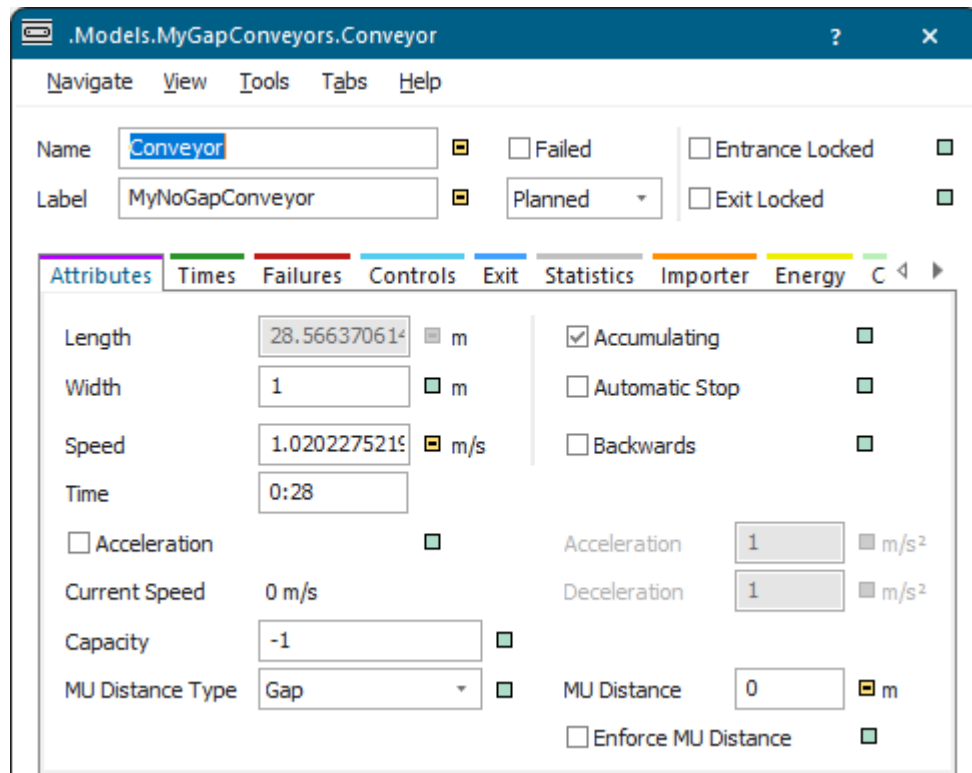


- The default setting of the *Conveyor* for the **MU Distance** is **-1**, meaning that the feature **MU Distance** is deactivated and that the parts can move onto the conveyor as soon as they arrive from the predecessor.



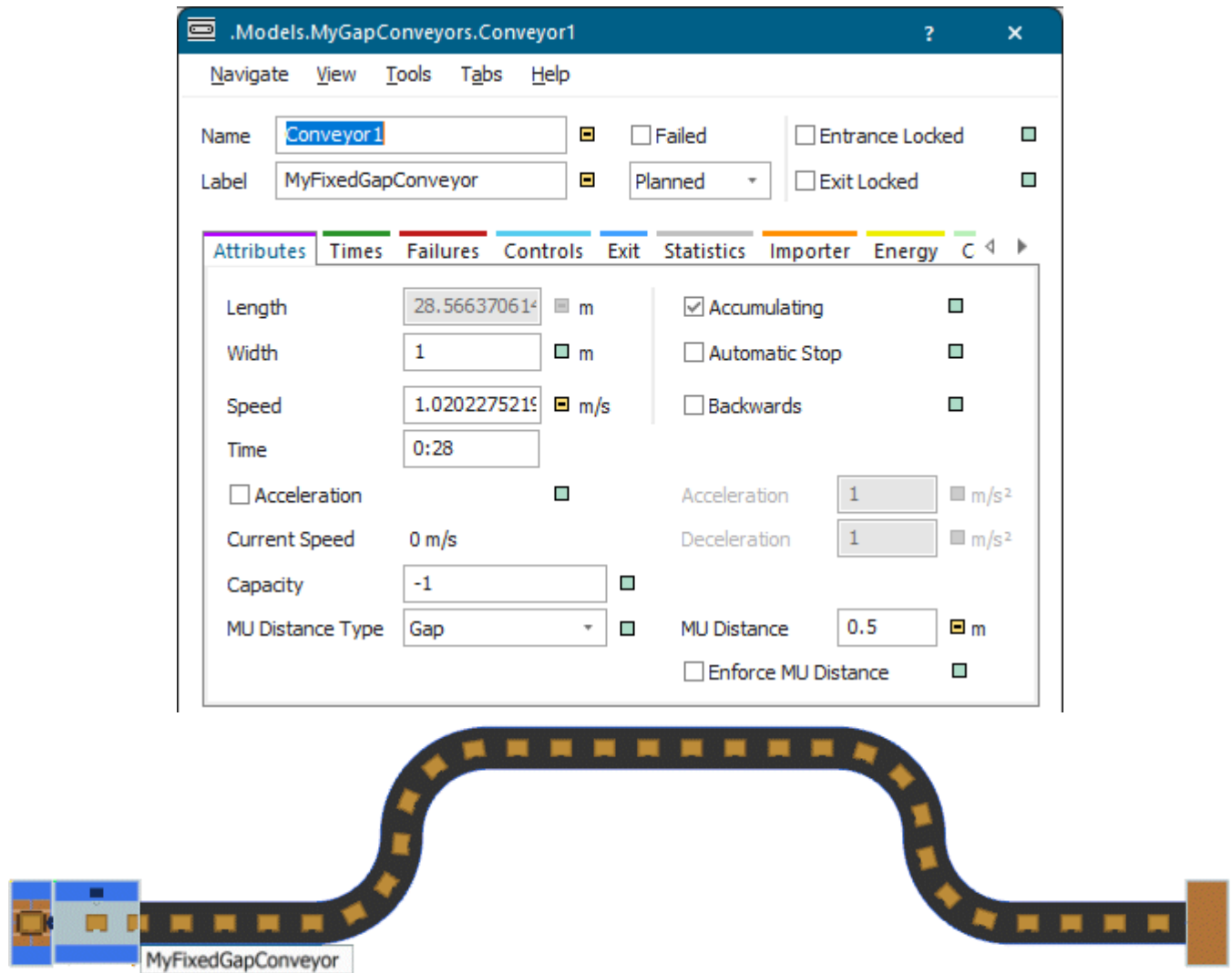


- A conveyor with **no gap** between the parts does not allow a space between them, when you type in a **MU Distance** of 0.



- A conveyor with a **fixed gap** ensures that the space between the parts moving on it always is the same. The fixed gap causes the conveyor to frequently stop though.

In the example below we typed in 0.5 as the **MU Distance**, which ensures a space of **0.5** meters between the transported parts.

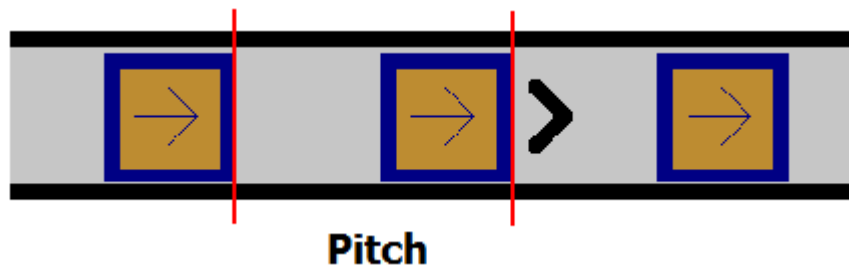


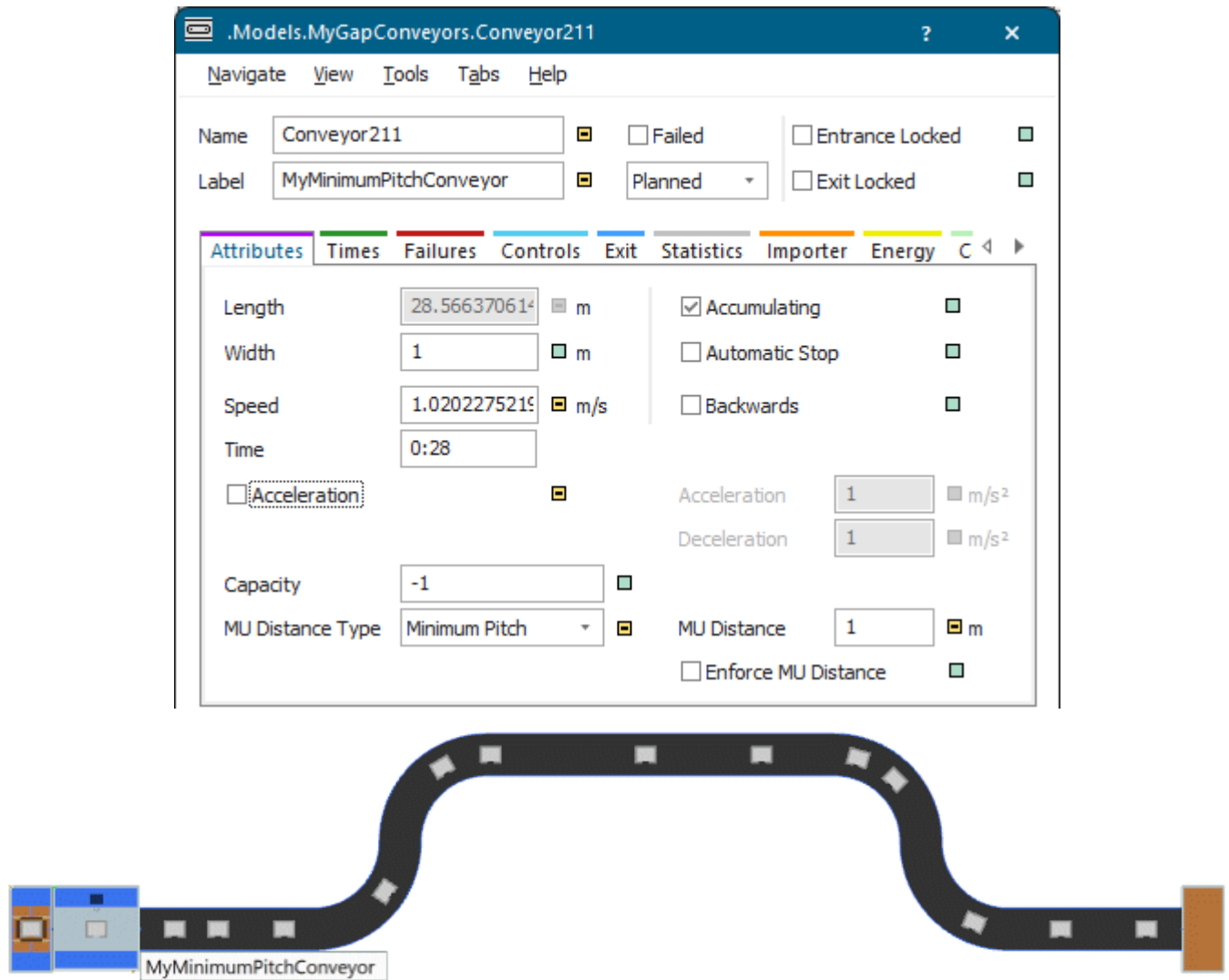
- A conveyor with a **Minimum Gap** ensures that the space between the parts moving on it will not become less than the minimum gap. This setting prevents the conveyor from stopping and ensures that all parts exit the conveyor if it transports a certain number of parts.

In our example we selected the **MU Distance Type > Minimum Gap** and typed in 1 as the **MU Distance**, which ensures a minimum space of 1 meter between the transported parts.

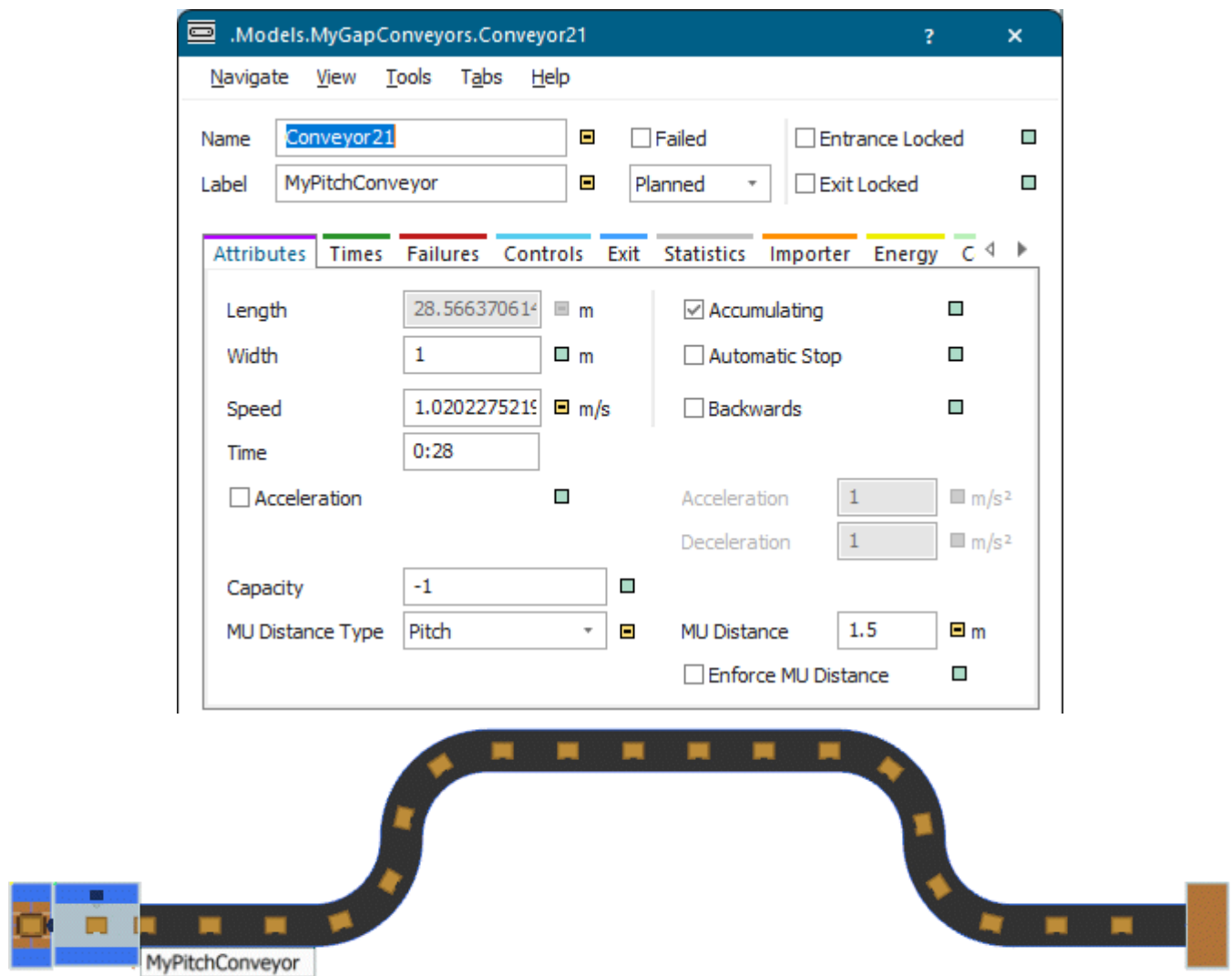


- A conveyor with a **Minimum Pitch** determines the minimum distance between the front of the preceding part and the front of the succeeding part. This setting prevents the *Conveyor* from stopping and from waiting for new parts and enables it to run dry.

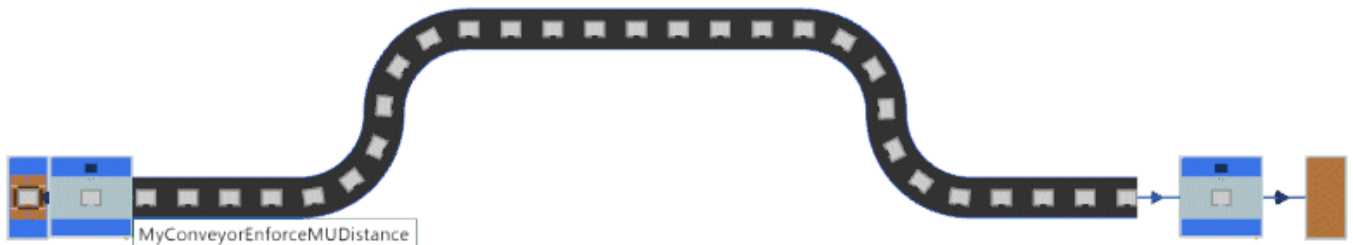
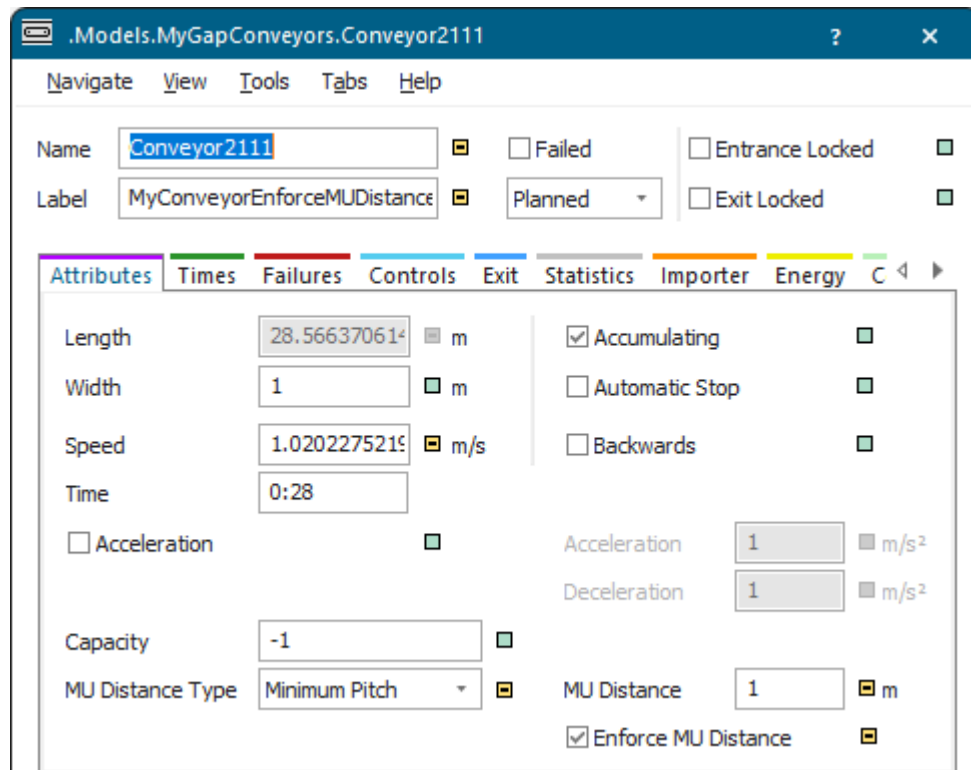




- A conveyor with a **Pitch** sets the distance between the front of the preceding part and the front of the succeeding part. This setting ensures that the conveyor stops and waits for new parts. It also enables the conveyor to run dry.



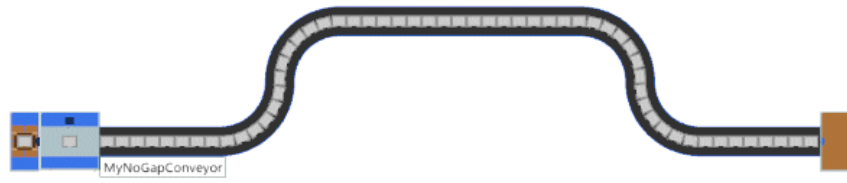
- To force the Conveyor to keep the specified **Distance** between the **MUs**, even if the **MUs** accumulate on the Conveyor, select **Enforce MU distance**.



When you clear **Enforce MU Distance**, the *MUs* accumulate on the Conveyor.



When running the simulation in our sample model, you'll notice the continuous flow of parts on the *minimum gap conveyor* and on the conveyor which does not use a **MU Distance**. The *no gap conveyor* and the *fixed gap conveyor* on the other hand stop quite often, denoted by the waiting parts.



Back to [Model a Transport System with Active Objects](#)

Model a Multiple Gap or a Multiple Pitch Conveyor

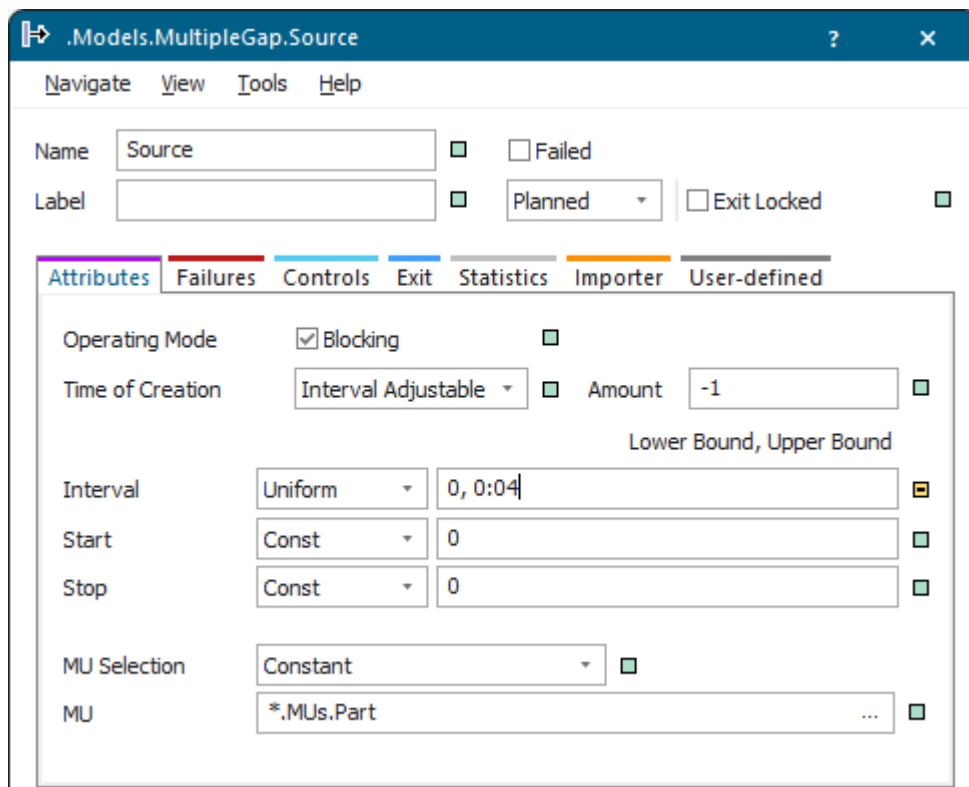
In this example we model a multiple gap and a multiple pitch *Conveyor*.

Model a Multiple Gap Conveyor

You can use the [Conveyor](#)  to model conveyors ensuring multiple gaps between the parts moving on it.

Multiple Gap sets the distance between the rear of the preceding *MU* and the front of the succeeding *MU*. The distance can be an integer multiple of the defined **MU Distance**.

For the *Source* we selected the **Uniform** distribution with a **Lower Bound** of 0 and an **Upper Bound** of 0:04.



.Models.MultipleGap.Source

Navigate View Tools Help

Name: Source ☐ ☐ Failed

Label: ☐ Planned ☐ Exit Locked ☐

Attributes Failures Controls Exit Statistics Importer User-defined

Operating Mode ☒ Blocking ☐

Time of Creation: Interval Adjustable ☐ Amount: -1 ☐

Interval: Uniform ☐ Lower Bound, Upper Bound: 0, 0:04 ☐

Start: Const ☐ 0 ☐

Stop: Const ☐ 0 ☐

MU Selection: Constant ☐

MU: *,MUs.Part ☐

For the *Conveyor* we selected the settings below with the **MU Distance Type > Multiple Gap** and an **MU Distance** of 1 meter.

.Models.MultipleGap.Conveyor

Name: ☐ Failed ☐ Entrance Locked ☐
 Label: ☐ Planned ☐ Exit Locked ☐

Attributes Times Failures Controls Exit Statistics Importer Energy C

Length: m ☐ Accumulating ☐
 Width: m ☐ Automatic Stop ☐
 Speed: m/s ☐ Backwards ☐
 Time:
☐ Acceleration ☐ Acceleration: m/s²
 Deceleration: m/s²
 Capacity: ☐
 MU Distance Type: ☐ MU Distance: m ☐
☒ Enforce MU Distance ☐

When we run the simulation, we see that the *Conveyor* does not stop to wait for new *MUs* after the gap has passed, but that the next *MU* can only enter the *Conveyor* if an integer multiple of the gap can be preserved.

Open the *EventDebugger*, step through the simulation event by event, and watch the event types **CheckMUDistance**, **CreateMU**, and **Out**.



Event Debugger - 5.5000

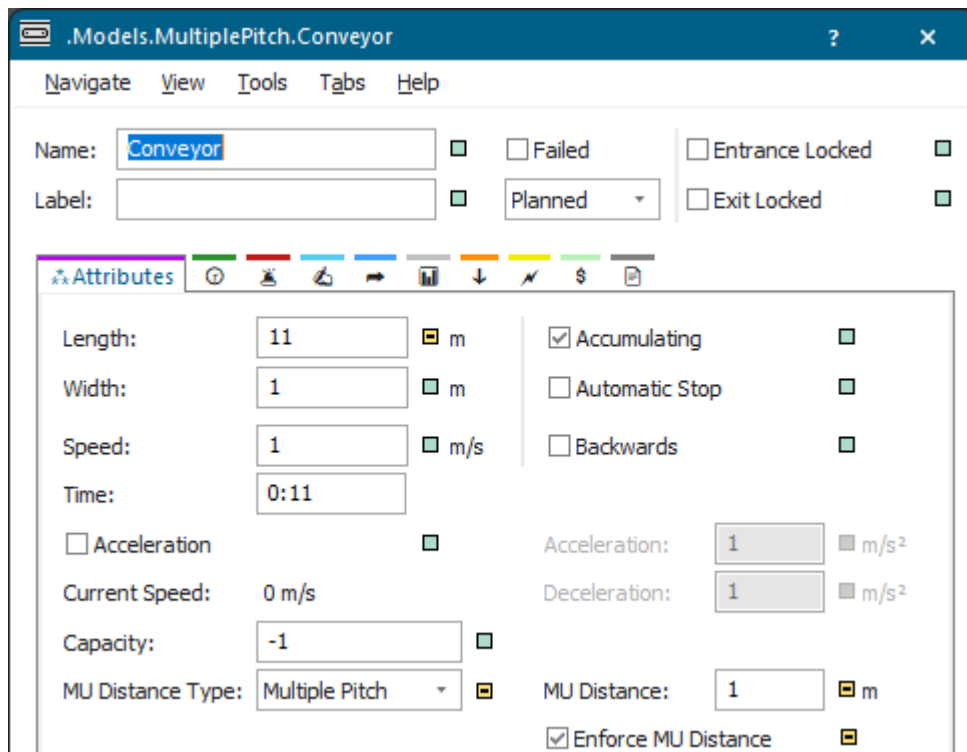
☐ Breakpoints Active Run to Time: ☐ Trace File

B..	Type	Time	Receiver	Sender	Insertion Time	Parameters
	CheckMUDistance	6.0000	Conveyor		5.5000	
	CreateMU	8.1946	Source		5.0000	
	Out	11.0000	.MUs.Part: 16	Conveyor	0.5000	

Model a Multiple Pitch Conveyor

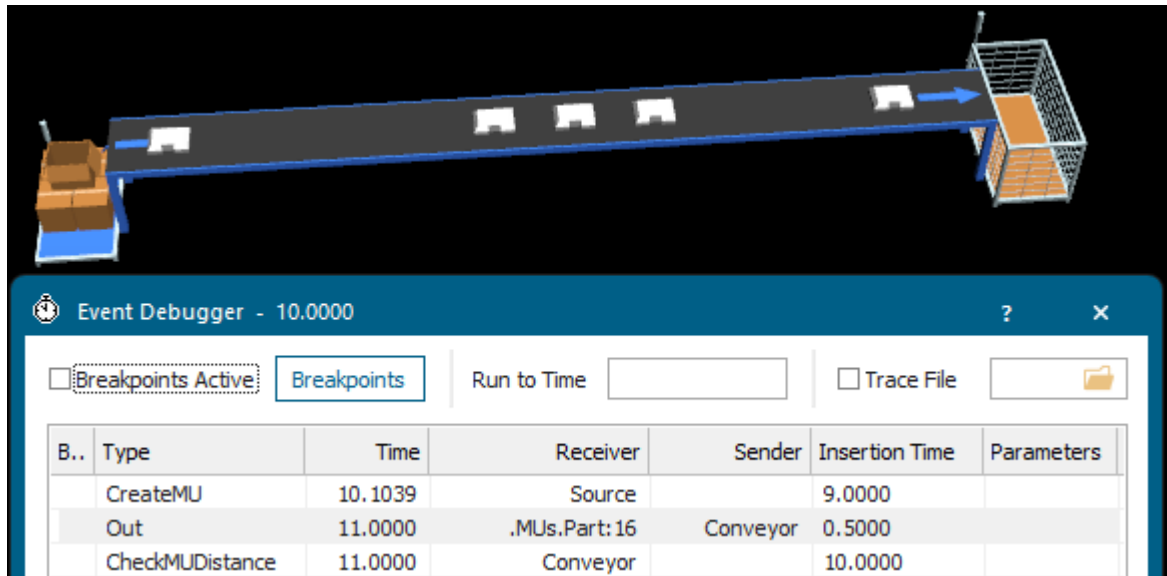
To model a **Multiple Pitch Conveyor**, we duplicated our **Multiple Gap Conveyor** model and changed the setting **MU Distance Type** to **Multiple Pitch**.

Multiple Pitch sets the distance between the front of the preceding *MU* and the front of the succeeding *MU*. The distance can be an integer multiple of the defined **MU Distance**. We did not change any other settings.



When we run the simulation, we see that the *Conveyor* does not stop to wait for new *MUs* after the pitch has passed, but that the next *MU* can only enter the *Conveyor* if an integer multiple of the pitch can be preserved. For this setting the *Conveyor* behaves like a chain conveyor.

Open the *EventDebugger*, step through the simulation event by event, and watch the event types **CheckMUDistance**, **CreateMU**, and **Out**.



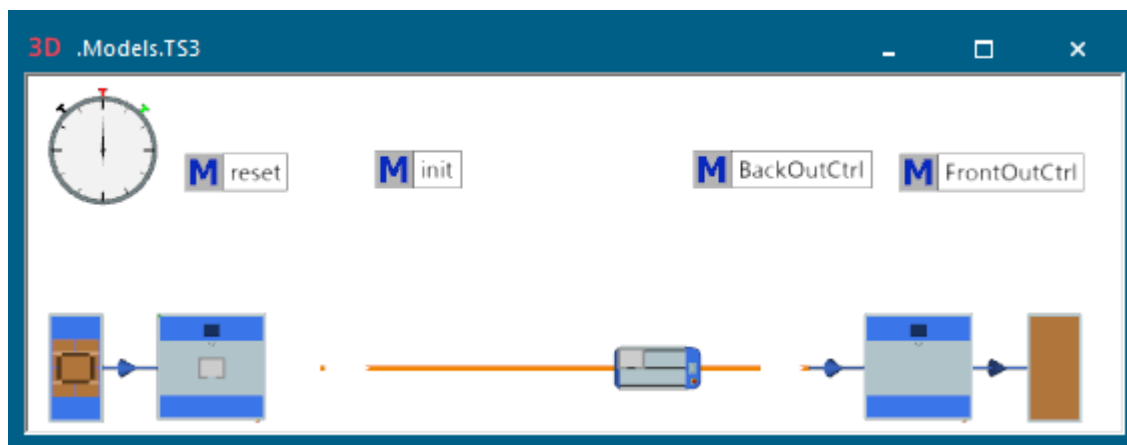
Back to [Model a Transport System with Active Objects](#)

Model a Transport System with Passive Objects of Type Track

Model a Transport System with Passive Objects

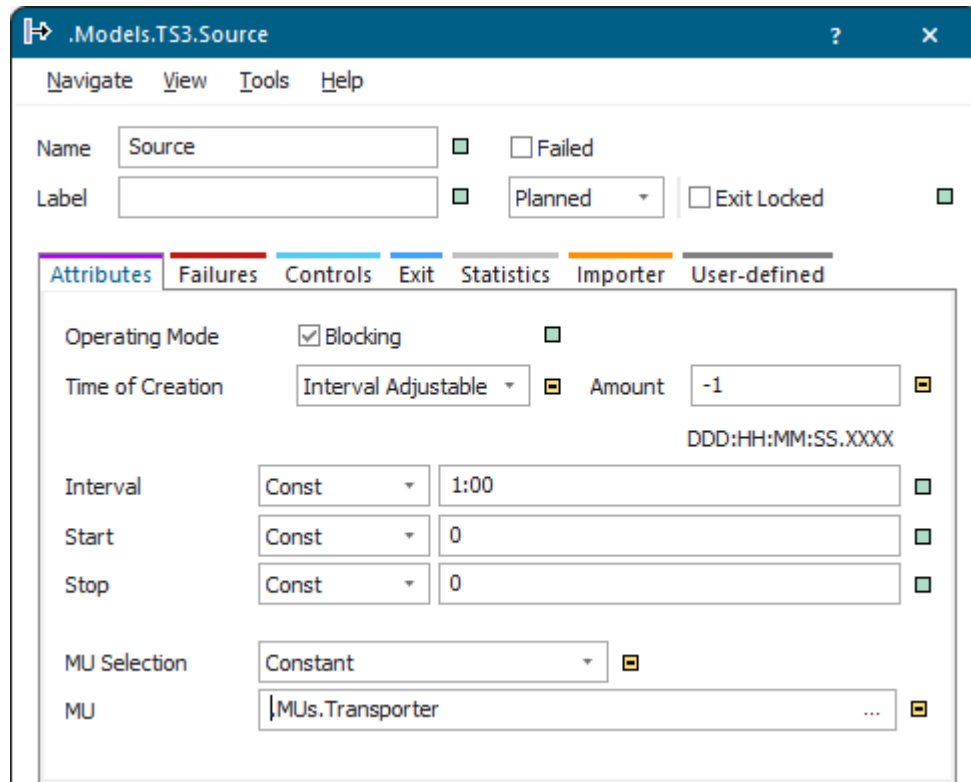
When you model transport systems with passive objects, you will use the *Track* or the *TwoLaneTrack* on which the active *Transporter* drives.

As the *Track* is a passive object, you cannot just connect it with two stations, but have to make sure that a *Transporter* is available, plus you have to load and unload this *Transporter*. To prevent the *MUs* from automatically transferring from the preceding station, in our example *SP1*, onto the [Track](#) , do not connect them with a *Connector*.



To create and insert a [Transporter](#)  you can do one of the following:

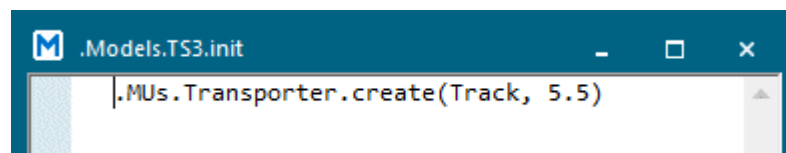
- Use the *Source*: Select **Attributes > Time of Creation > Number Adjustable** and enter an **Amount** of 1. To tell it to create transporters, select **MU > .MUs.Transporter**.



- Use the *SimTalk* method *create* and enter this command into the *init method* of your simulation model:

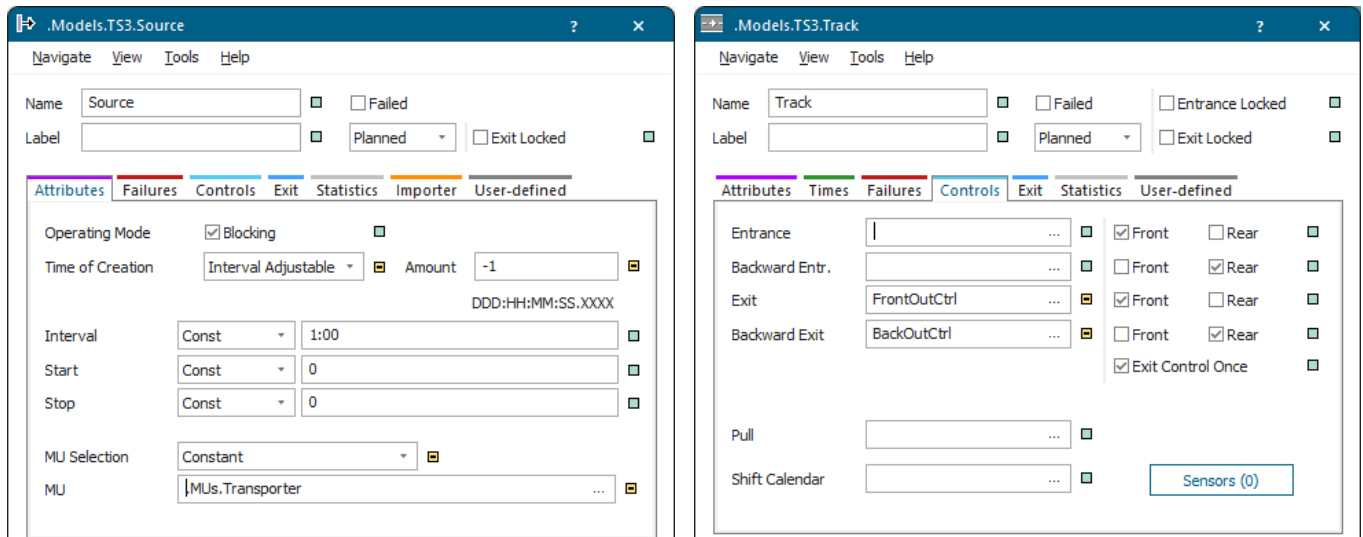
```
.MUs.Transporter.create(Track)
```

This inserts a *Transporter* at the end of the *Track*. If you want to insert a *Transporter* anywhere else on the *Track*, enter a position into the *Method*. This example creates the *Transporter* that is positioned 5.5 meters along the length of the *Track*:



```
.MUs.Transporter.create(Track, 5.5)
```

As the *Transporter* is to move parts from station to station, *SP1* and *SP2* in our example, you first have to produce those parts and then load them onto the *Transporter* at the station *SP1* and unload them at the station *SP2*. For this you will program a control in a *Method* and enter its name into the respective text box on the tab **Controls**.



Without entering a control, the *Transporter* must move to the end of the *Track* and stop there. To make the *Transporter* move back to the beginning of the *Track*, once it unloaded the parts, select the check box **Front** and enter this source code into the **Exit Control**, *FrontOutCtrl* in our example:

```
if @.empty
  @.Backwards := true
end
```

Once the *Transporter* backed up to the beginning of the *Track*, *Plant Simulation* activates the rear backward exit control, when you select **Rear** and enter the name of the *Method M* into the text box **Backward Exit**. This control has to accomplish these tasks:

- Wait until a *MU* is ready to exit the station, *SP1* in our example.
- Load the *MUs* into the *Transporter*.
- Move the *Transporter* to the end of the **Track**.

In our example we entered these instructions:

```
waituntil Sp1.occupied and SP1.cont.finished
SP1.cont.move( @ ) // load the part
@.Backwards := false
```

Line 1 of the source code tells *Plant Simulation* to wait until a *MU* is located on the station *SP1* and is processed all the way.

Line 2 loads the *MU* onto the *Transporter*. We use the anonymous identifier *@* to address the *Transporter*.

Line 3 tells the *Transporter* to move forward again to the end of the *Track*.

When the *Transporter* reaches the end of the *Track* it has to accomplish these tasks:

- Wait until the station, *SP2* in our example, is **Empty**.
- Unload the *Transporter*.
- Move the *Transporter* back to the beginning of the *Track*.

As the *Transporter* initially moves to the end of the *Track* and then activates the following exit control while being unloaded, we query if it is loaded or empty.

In our example we entered these instructions:

```
if @.empty
  @.Backwards := true
else
  waituntil SP2.empty prio 1
  @.cont.move(SP2)
  @.Backwards := true
end
```

Back to [Model Transport Systems](#)

Model a Tugger Train

Model a Tugger Train

To model a *tugger train*, consisting of a *tractor* and a number of *trailers* it pulls, you will use a number of extended features of the *Transporter*.

In our sample model a **tugger train** is loaded with parts, which will be unloaded at a station on the main line and on the side line each.

The model demonstrates how to:

- **Define the Tractor of the Tugger Train.**
- **Model the Source Creating the Tugger Trains.**
- **Model the Tracks on Which the Tugger Trains Drive.**
- **Configure the Loading and Unloading Stations.**

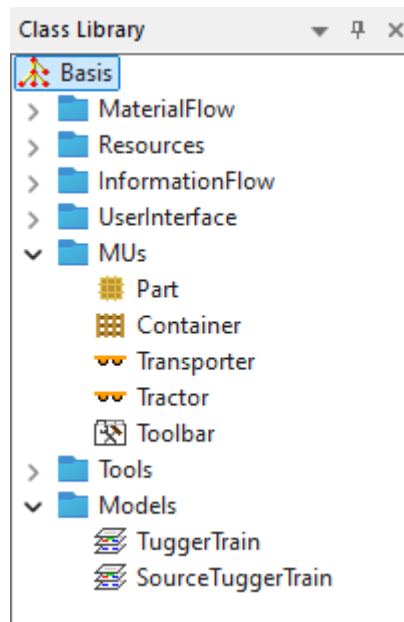
Back to [Model a Transport System with Passive Objects](#)

Define the Tractor of the Tugger Train

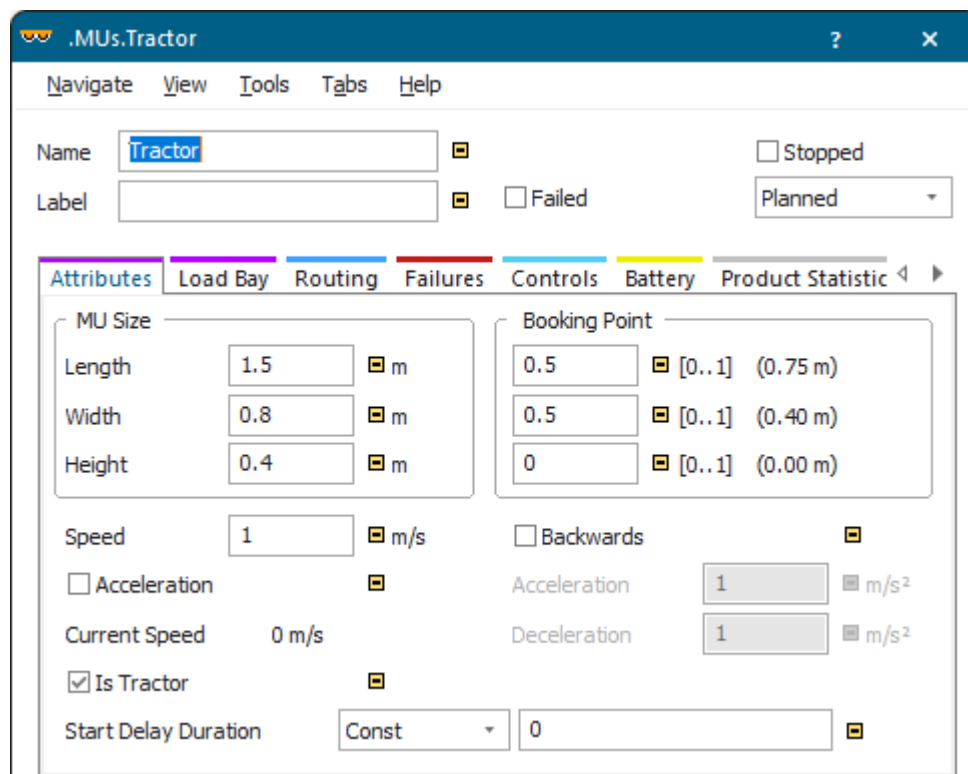
First, we set the *tractor unit* of the *tugger train*.

Proceed as follows:

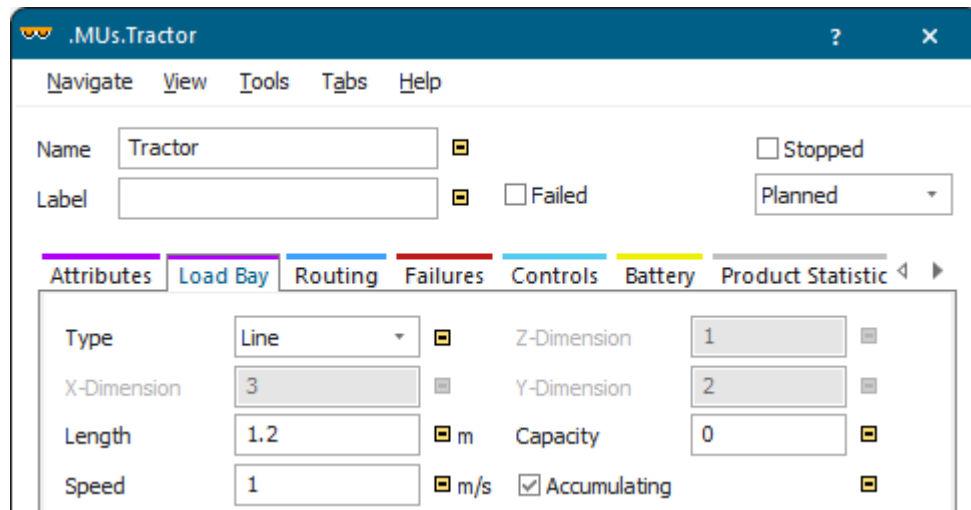
- Duplicate the object **Transporter** in the *Class Library*. Rename it to **Tractor**.



- Double-click the object *Tractor*. To designate it as the *tractor* of the *tugger train*, which pulls the *trailers*, select the check box **Is Tractor**.



- To prevent the *tractor* from loading parts onto its bed, enter 0 as the **X-Dimension** and the **Y-Dimension**. For the loading space types **Track** and **Line**, enter 0 as the **Capacity**.



Plant Simulation 2606 provides a pre-configured *tractor* unit and different types of *trailers* under **Manage Class Library > Standard Objects > Containers and Transporters**.

[Back to Model a Tugger Train](#)

Model the Source Creating the Tugger Trains

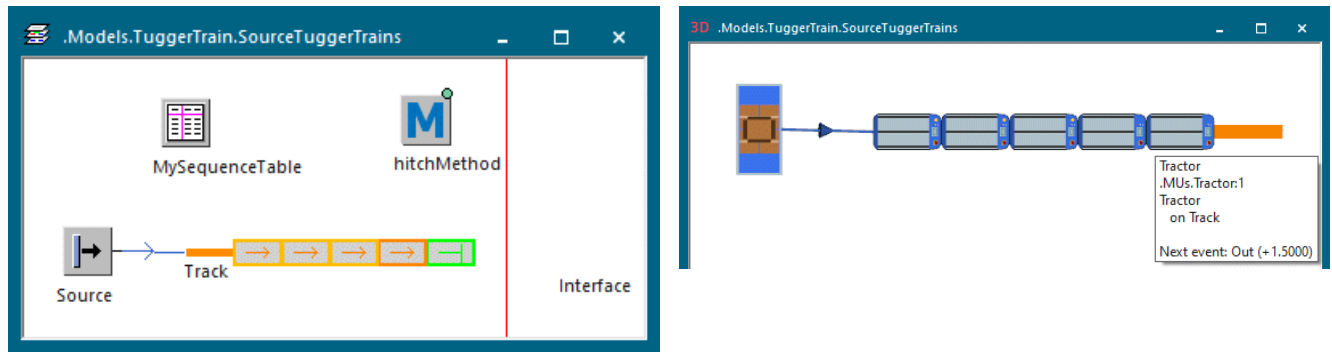
Model the Source Creating the Tugger Trains

We modeled the source, which creates the *tugger trains*, in the *Frame* named *SourceTuggerTrains*. We did this to prevent our overall model from becoming too cluttered with objects

.

The *Frame* contains of these objects:

- A **Source** object, which creates *tractors* and *trailers* according to the settings, which we enter into a *sequence table*.
- A *Track*, which should be long enough to accommodate the *tugger train* with its entire length after the *Source* has created it. This ensures that the *tugger train* is able to move onto the succeeding object as a whole. The *tugger train* in our example consists of a *tractor* and four *trailers*, each of which is 1.5 meters long. So we entered a length of 10 meters for the *Track*.



- A **Method**, in which we program how the *trailers* are hitched to the *tractor* when they collide with the *trailer* in front of them.

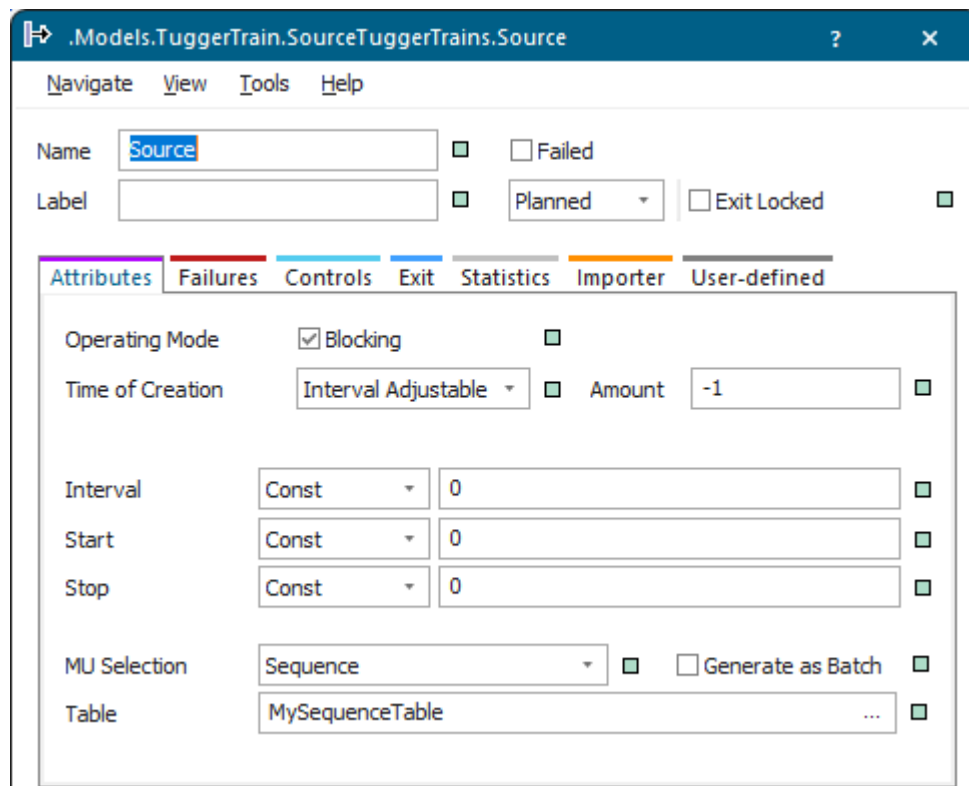
Back to **Model a Tugger Train**

Configure the Source and Create the Sequence Table

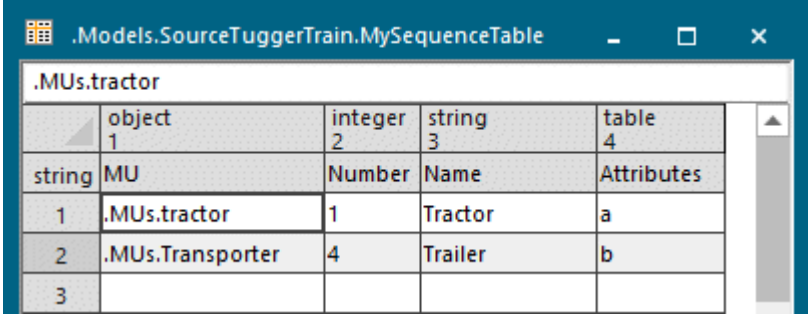
To configure the *Source* object, which creates the *tractors* and the *trailers*, which make up the *tugger train*, double-click its icon.

To produce the objects as a sequence according to the settings, which we enter into the sequence table:

- Select **MU Selection** > **Sequence**.



- Click  and select the *DataTable* named *MySequenceTable*, which we inserted above. To create a *tractor* object using the **MU class** *Tractor* and four *trailer* objects using the **MU class** *Transporter*, we entered the values shown in the screenshot below.



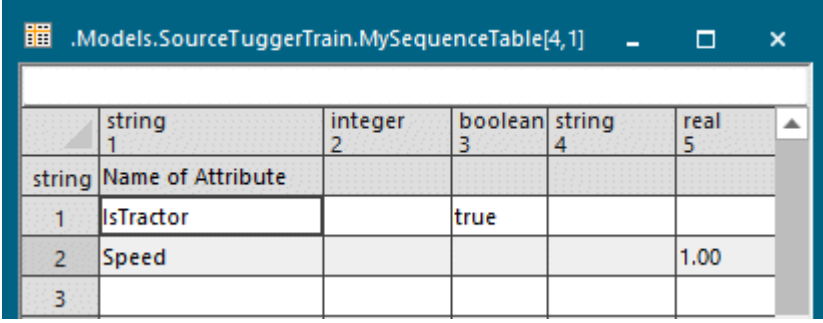
	object 1	integer 2	string 3	table 4
string	MU	Number	Name	Attributes
1	.MUs.tractor	1	Tractor	a
2	.MUs.Transporter	4	Trailer	b
3				

- The characters, which we enter into the column **Attributes**, only serve as a placeholder for the sub-tables contained in these cells. Within the sub-tables we set the attributes of the *tractor* and of the *trailers*.

To designate the *tractor* we entered the attribute *IsTractor* and set it to *true*. To set its speed, we entered the attribute *Speed* and entered *1.0* m/s.

Note

We already defined the attributes of the *tractor* in the class **Tractor**. We just entered them again to make sure that the settings in the *tractor* are the correct ones, in case the settings of the class get changed.



	string 1	integer 2	boolean 3	string 4	real 5
string	Name of Attribute				
1	IsTractor		true		
2	Speed				1.00
3					

To designate the *trailers*, we set the attribute *IsTractor* to *false*. As we want the *trailers* to drive faster than the *tractor* and to collide with the object in front of them, we entered *1.1* m/s as their *Speed*. To designate the **Collision Control** (*CollisionCtrl*), we entered the name of the *Method*, which we inserted above, *hitchMethod* in our case. The collision control hitches the *trailers* to each other and the front *trailer* to the *tractor*.

.Models.SourceTuggerTrain.MySequenceTable[4,2]

	string 1	integer 2	boolean 3	string 4	real 5
string	Name of Attribute				
1	IsTractor		false		
2	Speed				1.10
3	CollisionCtrl			.Models.TuggerTrain.SourceTuggerTrains.hitchMethod	
4					

Back to Model the Source Creating the Tugger Trains

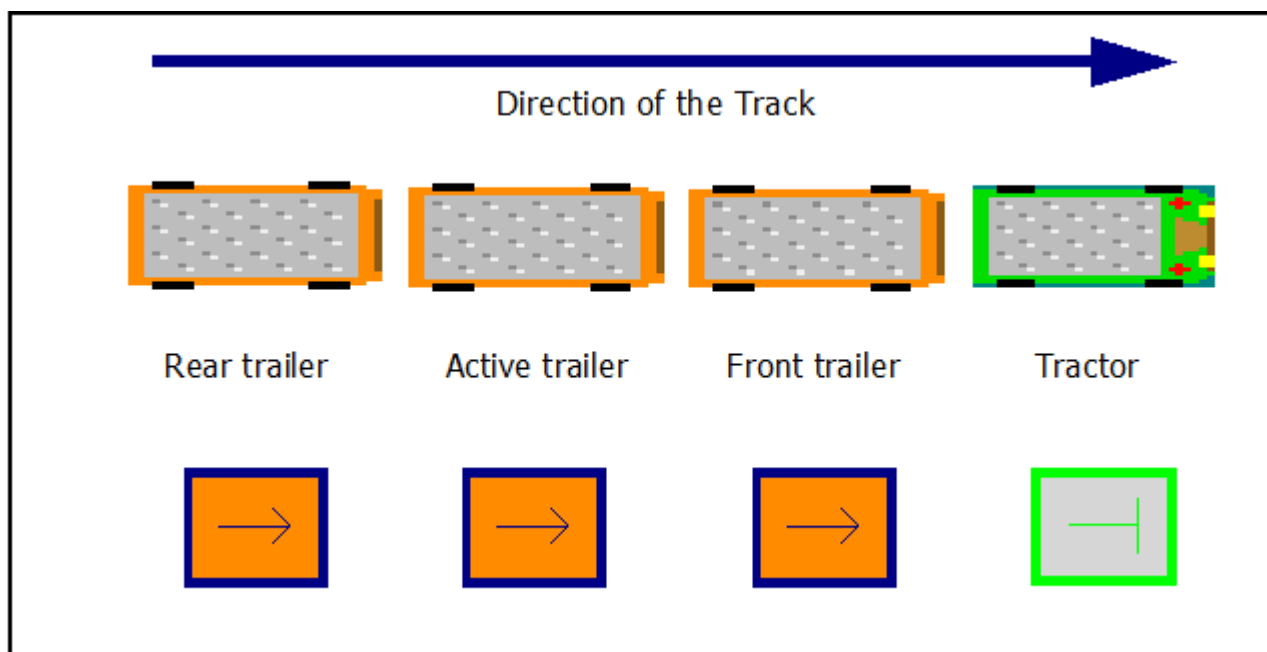
Video on YouTube

<https://youtu.be/BvHLQCljGIA?si=SbE5dRcUJQea823K&t=425>

Program the Collision Control

To program the **Collision Control** named *hitchMethod*, which we inserted above, double-click the *Method* object. We want this control to hitch the colliding *trailer* to the one in front of it. To do so, we have to establish the *trailer* moving in front of the active *trailer*.

Finally, we are going to hitch the rear *Transporter* to the one moving in front of it to create the *tugger train*.



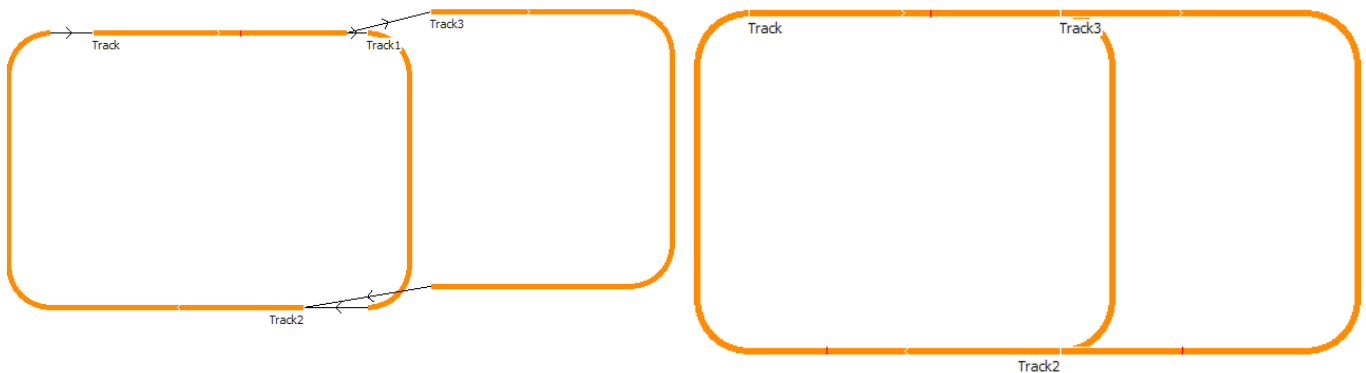
These statements above translate into this source code in our **Collision Control** like this:

```
if @.isTractor = false then
@.hitchFront(@.FrontMU)
end
```

Back to **Model the Source Creating the Tugger Trains**

Model the Tracks on Which the Tugger Trains Drive

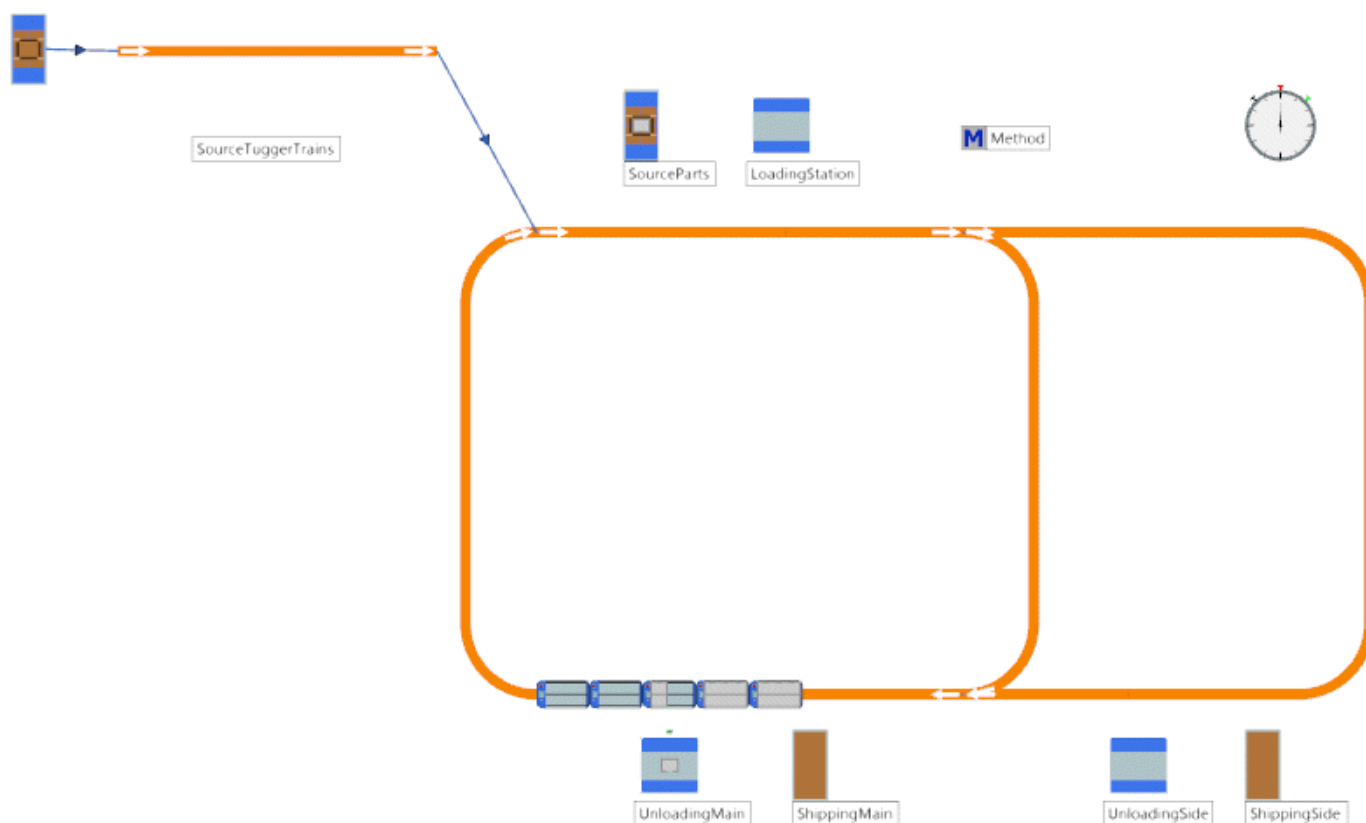
To demonstrate how the *tugger trains* move as a single unit on a track system, let's model a main line, from which a side line splits, which then feeds back into the main line. For this we use four *Tracks* and connect them with *Connectors*. To move the side line over the main line, hold down the **Shift** and/or **Ctrl** keys and press the arrow keys.



After we have finished creating the track system, we have to insert:

- The station producing the *tugger trains*, which we modeled in the *Frame* named *SourceTuggerTrains*. We connect this *Frame* with the object *Track*.
- A *Source* which produces the parts with the default settings and a station, which loads them onto the *trailers* of the *tugger train*.
- An unloading station on the main line and the side line each, plus a *Drain* each, which removes the parts from the lines. For the two *Drains*, which we named *ShippingMain* and *ShippingSide*, we use the default settings.
- An *EventController* for running the simulation.

The finished simulation model looks like this.



Back to [Model a Tugger Train](#)

Configure the Loading and Unloading Stations

We will model the loading station and the unloading stations with the object *TransferStation*.

You find more information under [Load, Unload, and Reload Parts with the TransferStation](#).

Configure the loading station on the tab **Attributes**. Select the object which provides the parts (**parts from**) and the object on which the *tugger train* moves (**target is on**). In our case this is the *SourceParts* and the *Track*.

Enter the position of the sensor, which triggers loading the *tugger trains* with parts. We entered 7 (meters). The **TransferStation**  then enters these settings into the dialog **Sensor** of the *Track*.

The screenshot shows the configuration window for a Loading Station. The title bar is ".Models.TuggerTrain.LoadingStation". The "Name" field is set to "LoadingStation" and the "Label" field is empty. The "Planned" checkbox is checked. The "Station type" section has four radio buttons: "Load" (selected), "Unload", "Reload", and "Move". The "Parts from:" field is set to "SourceParts" and the "Target is on:" field is set to "Track". The "Sensor position:" field is set to "7" and the "Lane A" dropdown is selected. The "OK", "Cancel", and "Apply" buttons are at the bottom.

Next, configure the stations *UnloadingMain* and *UnloadingSide* as shown in the screenshots below.

The screenshot shows the configuration window for an Unloading Main station. The title bar is ".Models.TuggerTrain.UnloadingMain". The "Name" field is set to "UnloadingMain" and the "Label" field is empty. The "Planned" checkbox is checked. The "Station type" section has four radio buttons: "Load", "Unload" (selected), "Reload", and "Move". The "Parts from:" field is set to "Track2" and the "Target is on:" field is set to "ShippingMain". The "Sensor position:" field is set to "9" and the "Lane A" dropdown is selected. The "OK", "Cancel", and "Apply" buttons are at the bottom.

The screenshot shows the configuration window for an Unloading Side station. The title bar is ".Models.TuggerTrain.UnloadingSide". The "Name" field is set to "UnloadingSide" and the "Label" field is empty. The "Planned" checkbox is checked. The "Station type" section has four radio buttons: "Load", "Unload" (selected), "Reload", and "Move". The "Parts from:" field is set to "Track3" and the "Target is on:" field is set to "ShippingSide". The "Sensor position:" field is set to "29.5" and the "Lane A" dropdown is selected. The "OK", "Cancel", and "Apply" buttons are at the bottom.

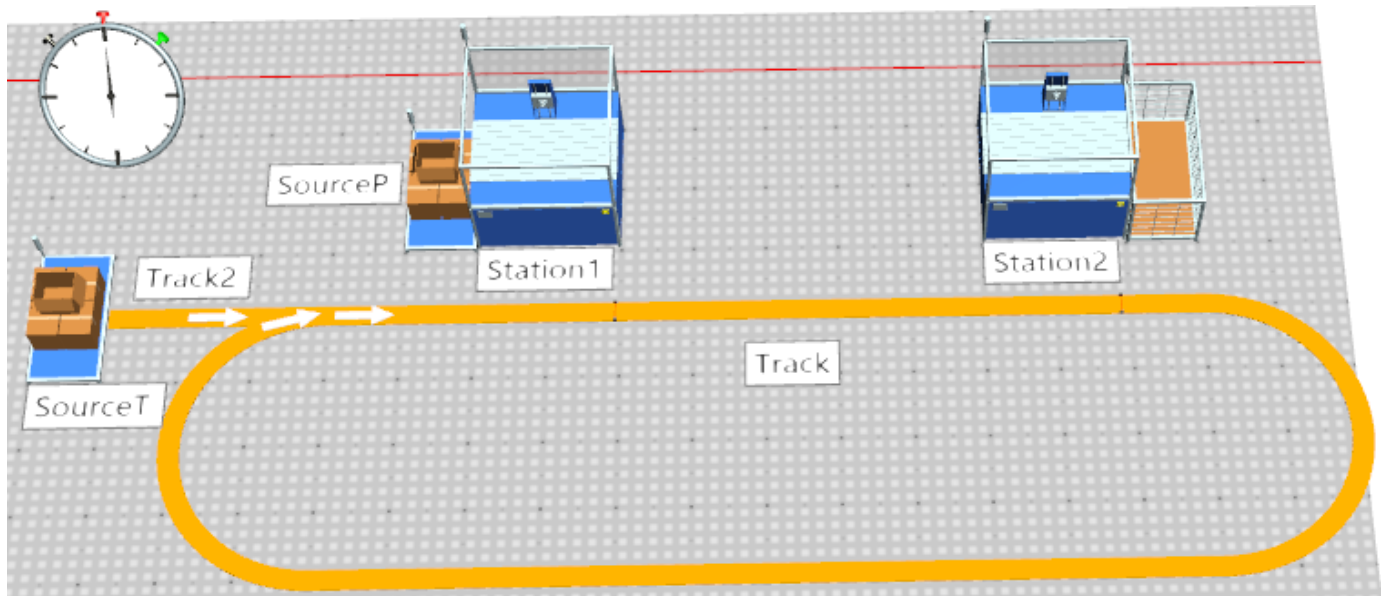
When you are finished, start the simulation and watch how the *tugger trains* are loaded with parts, move on the main line, are unloaded, are loaded again, and are unloaded on the side line on their second trip.

Back to [Model a Tugger Train](#)

Stop the Transporter at Its Destination

Here we demonstrate how to stop the *Transporter* at its **Destination**, make it wait for a *Part* to be transported, and then make it start driving again.

Our finished model looks like this:



Insert the required objects:

- The *SourceT* produces a single *Transporter*.
- The *SourceP* produces an infinite number of *Parts* and passes them on to *Station1*, which processes it.
- The *Transporter* drives from *Track2* onto the *Track*, accelerates, drives on to its first **Destination** *Station1* and decelerates in front of the *Sensor1* on the *Track*. There it waits until the *Part* is processed and *Station1* loads it onto the *Transporter*.
- The *Transporter* then accelerates again, drives on, decelerates in front of *Sensor2* at *Station2*, waits there and passes the transported *Part* to *Station2*.

Program the required actions:

- Define the properties of the *Transporter* in the **Entrance Control** of the *SourceT*:

```
@.XDim = 1
@.YDim = 1
@.StopAtDestination = true
@.AccelerationEnabled = true
@.Acceleration = 0.2
@.Deceleration = 0.2
@.Destination = Station1
```

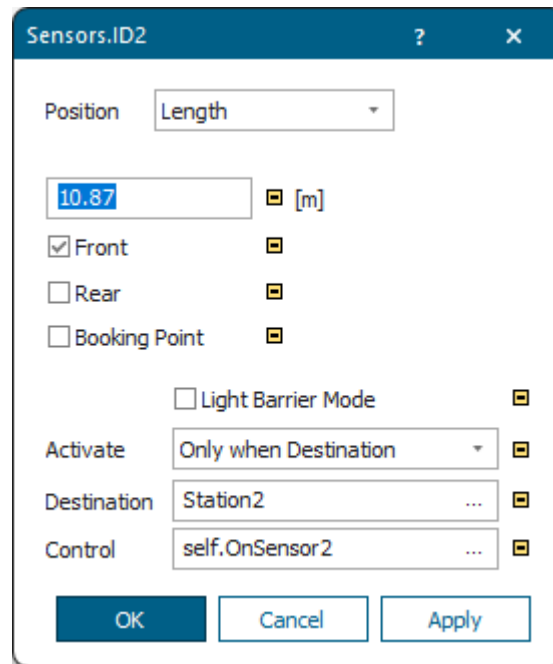
- Define the sensors at which the *Transporter* stops and waits on the *Track*.

The **Sensor Control** of *Sensor1* looks like this:

```
param SensorID: integer, Front: boolean, BookPos: boolean
var station = ?.sensorID(SensorID).Destination

waituntil station.Cont /= void and station.Cont.Finished
-- Wait until the station's content is not empty and the station's
-- content has finished processing.

station.Cont.move(@)
@.Destination = Station2
@.Stopped = false
```



The **Sensor Control** of *Sensor2* looks like this:

```
param SensorID: integer, Front: boolean, BookPos: boolean

var station = ?.sensorID(SensorID).Destination
var part    = @.Cont

part.move(station)

waituntil part.Location /= @
-- Wait until the part's location is not empty, i.e., until a part
-- is located there.

@.Destination = Station1
@.Stopped = false
```

See also

[Stop at Destination](#)

[StopAtDestination \[SimTalk\]](#)

Using Automatic Routing

Use Automatic Routing

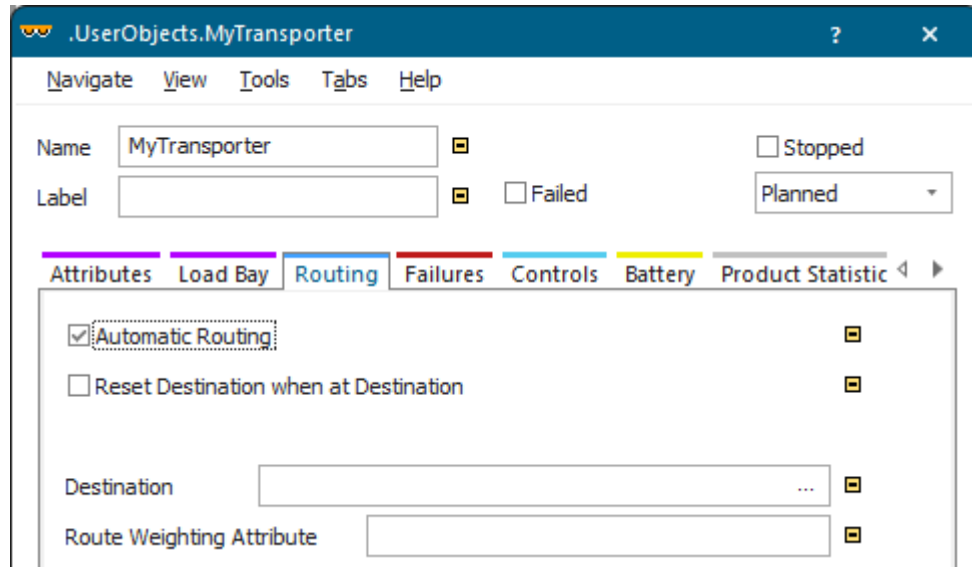
In this sample model we demonstrate how to use **automatic routing** to ensure that the parts always reach their destination object on the shortest possible route.

You can:

- **Work with Automatic Routing of the Transporter**

You find the respective settings on the tab **Routing** of the *Transporter*.

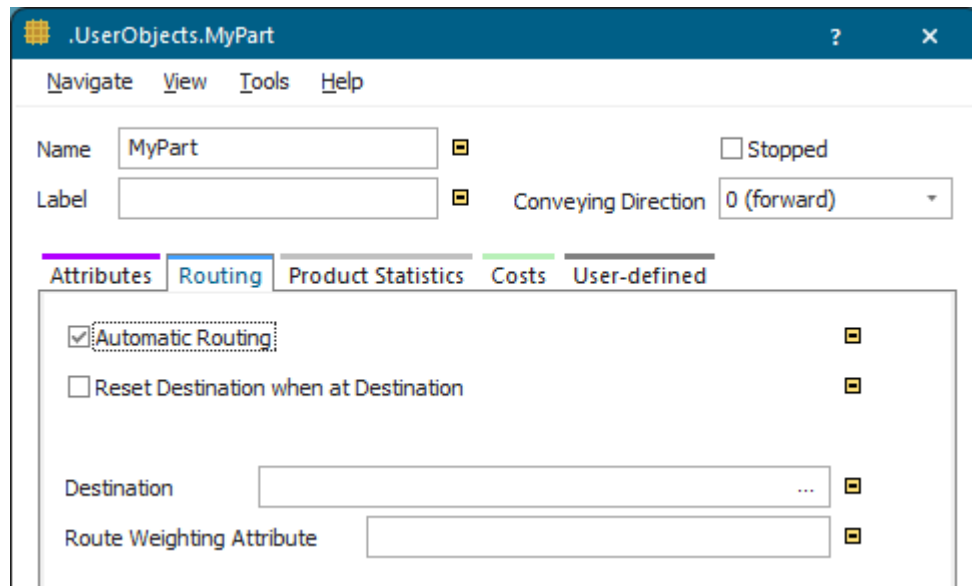
The *Transporter* finds its **Destination** along the **shortest** route.



- **Work with Automatic Routing of Part and Container**

You find the respective settings on the tab **Routing** of *Part* and *Container*.

The *Part* and the *Container* find their **Destination** along the **fastest** route.



Back to [Model Transport Systems](#)

Working with Automatic Routing of the Transporter

Work with Automatic Routing of the Transporter

In this sample model we demonstrate how a *Transporter* automatically finds its destination object. As soon as the destination object is assigned a reachable destination, *Plant Simulation* computes the route which describes the way on which the *Transporter* reaches its destination. The *Transporter* finds its **Destination** along the **shortest** route.

The *Source* named *SourceTransporters* creates *Transporters*, which wait for parts at the sensor of the *LoadingStation*.

In the **Sensor Control** we set that the *Transporter* waits until the *SourceParts* has produced a part of type *MyPart*. Once a part is located on the *LoadingStation*, this part is loaded onto the *Transporter*. Then we determine the destination object of the parts, i.e., one of the *unloading stations* *Unload1*, *Unload2*, or *Unload3*. When the *Transporter* has reached the destination object, the part will be unloaded and the *Transporter* returns to the *LoadingStation*, where it is loaded anew.

We will demonstrate how to:

- [Insert and Configure the Tracks](#)
- [Program the Control for Loading the Transporter](#)
- [Configure the Sources and the Transporter](#)
- [Program the Control for Unloading the Transporter](#)
- [Make the 3D Model Visually Pleasing](#)

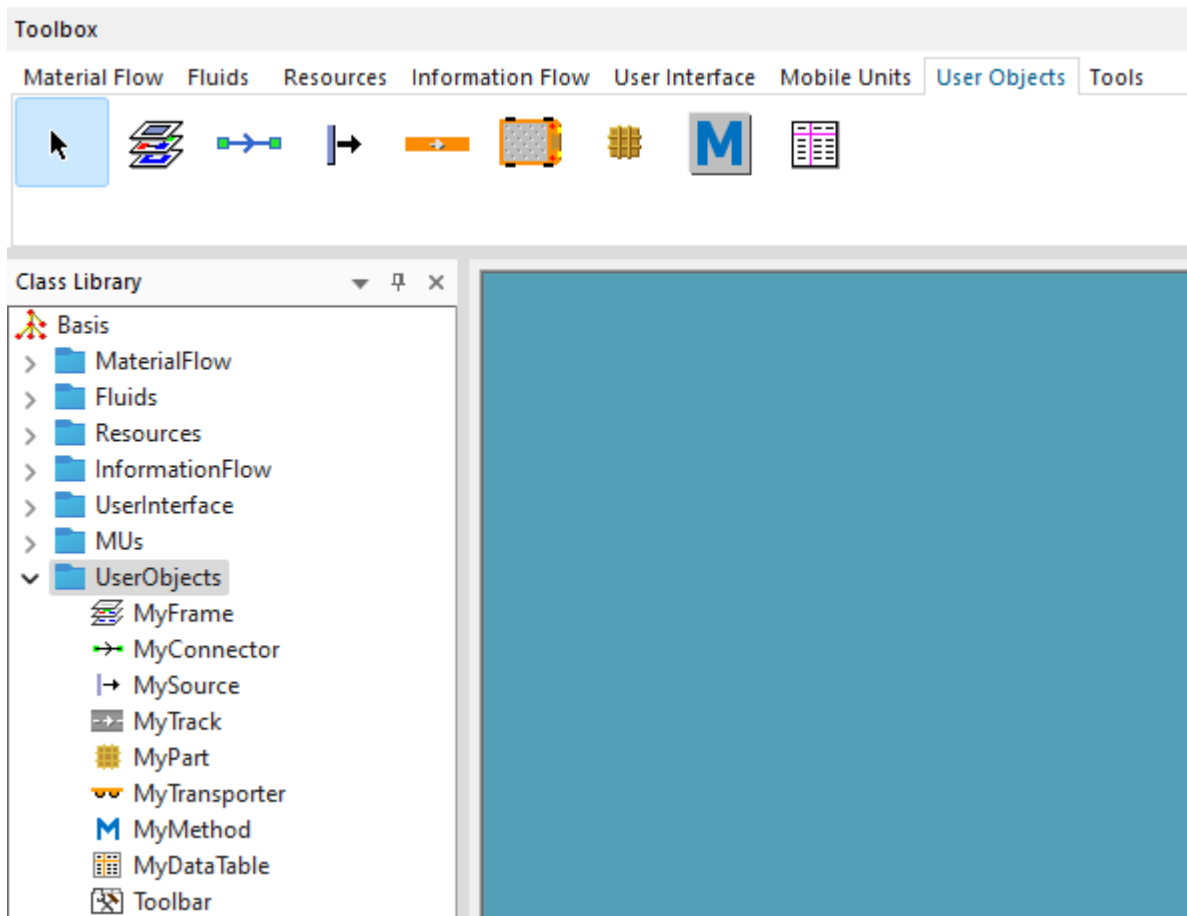
Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

[Back to Use Automatic Routing](#)

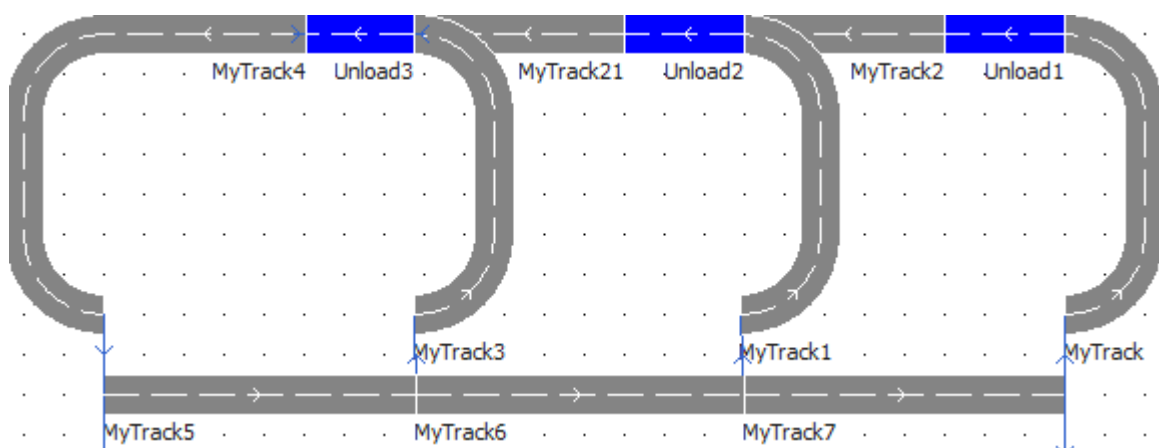
Insert and Configure the Tracks

Before we start modeling, we create the objects that we need as we do not want to change the built-in object classes.

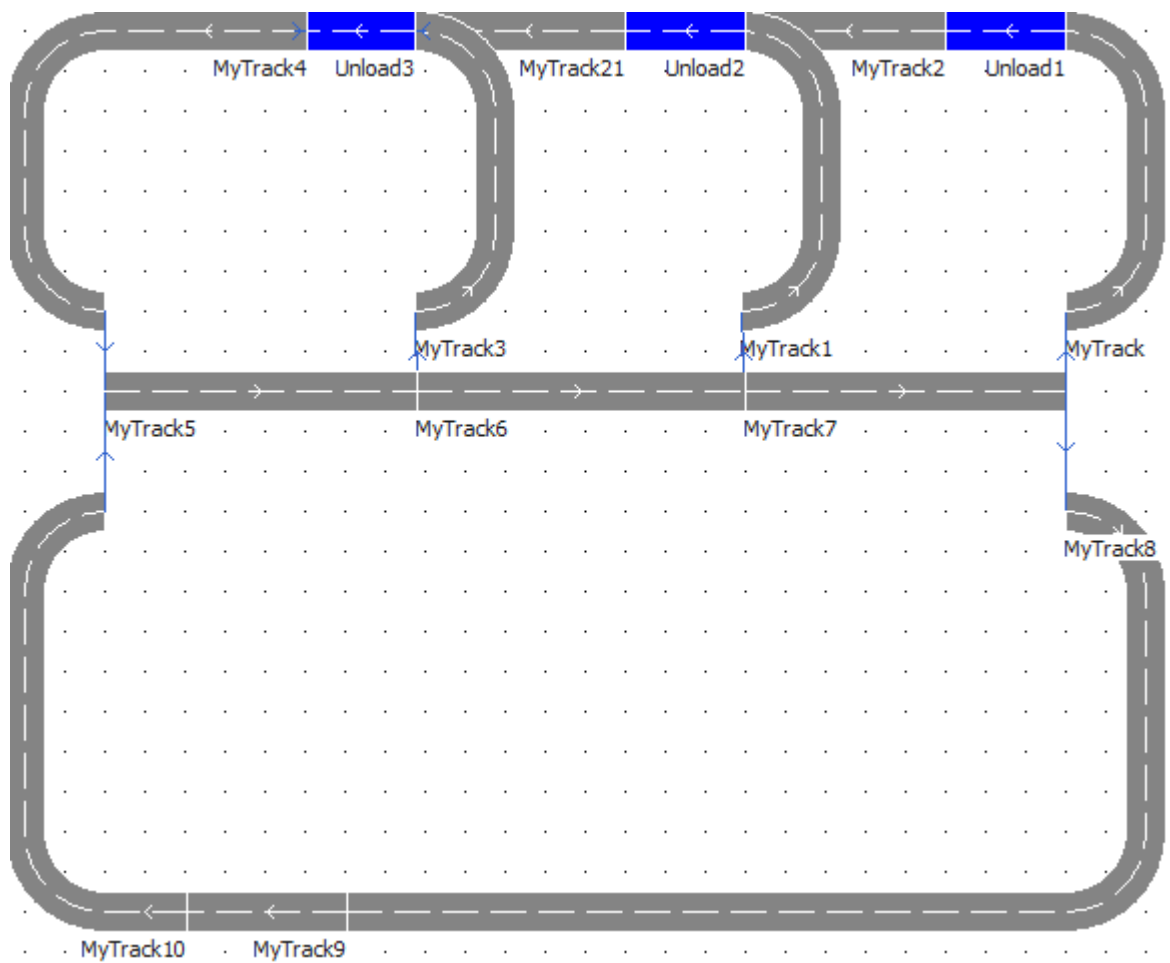
We duplicated the *Source*, the *Track*, the *Connector*, the *Transporter*, the *Part*, the *Method*, and the *DataTable* in their respective folders. *Plant Simulation* places the duplicated objects into the folder **UserObjects** in the *Class Library*. We renamed the objects and added them to the toolbar **User Objects** with drag-and-drop.



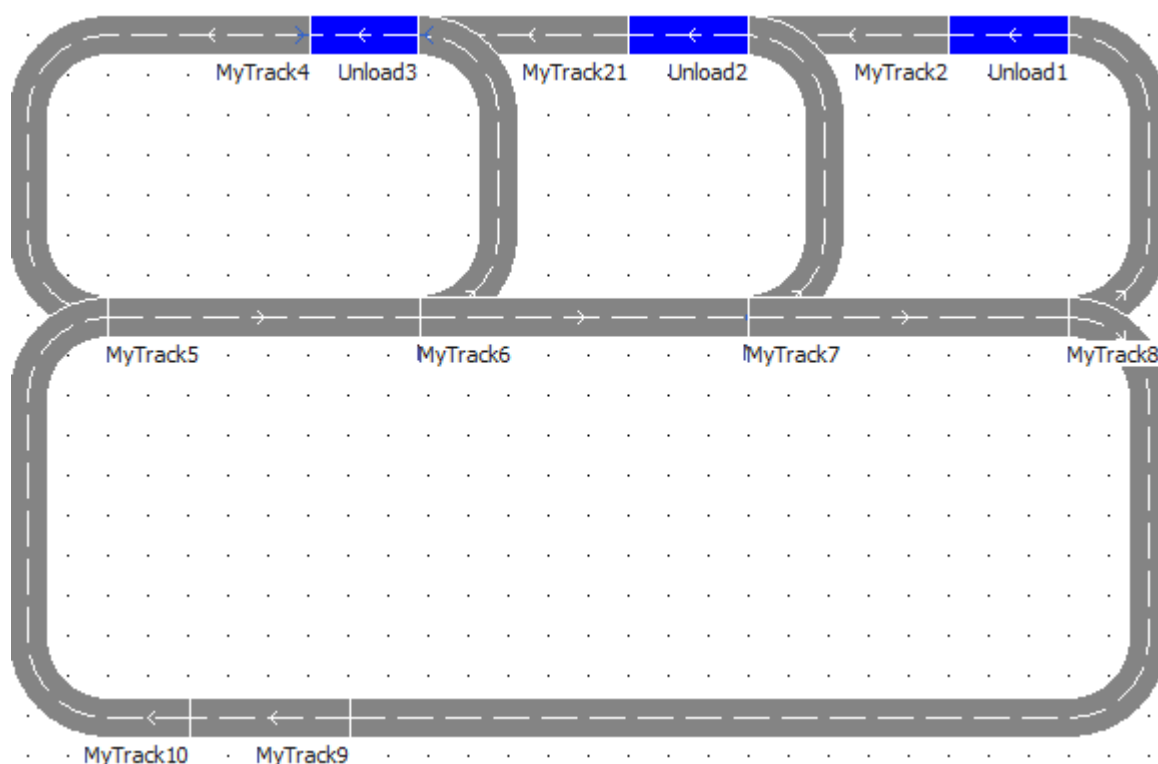
Now we model our route network on which the *Transporter* drives. As a part of the route network consists of three route loops, we first create a loop, copy it, and insert it twice. Then we connect the upper *Tracks* with *Connectors*.



We continue this until our route network is finished. Make sure that the individual route pieces are connected!



Then we move the individual loops so that they look like this:



Next we configure the *LoadingStation* and the three *unloading stations*, *Unload1*, *Unload2*, and *Unload3*. For this we renamed the respective route pieces as shown below. In addition we colored them on the tab **Curve > Color** to better recognize them.

Then we create a **sensor** on the *LoadingStation* at which the parts are going to be loaded onto the *Transporters*. Click the right mouse button on the route piece *LoadingStation* and select **Create Sensor**. We selected these settings:

Sensors.ID1 ? x

Position Length ▾

2.1 [m]

☒ Front

☐ Rear

☐ Booking Point

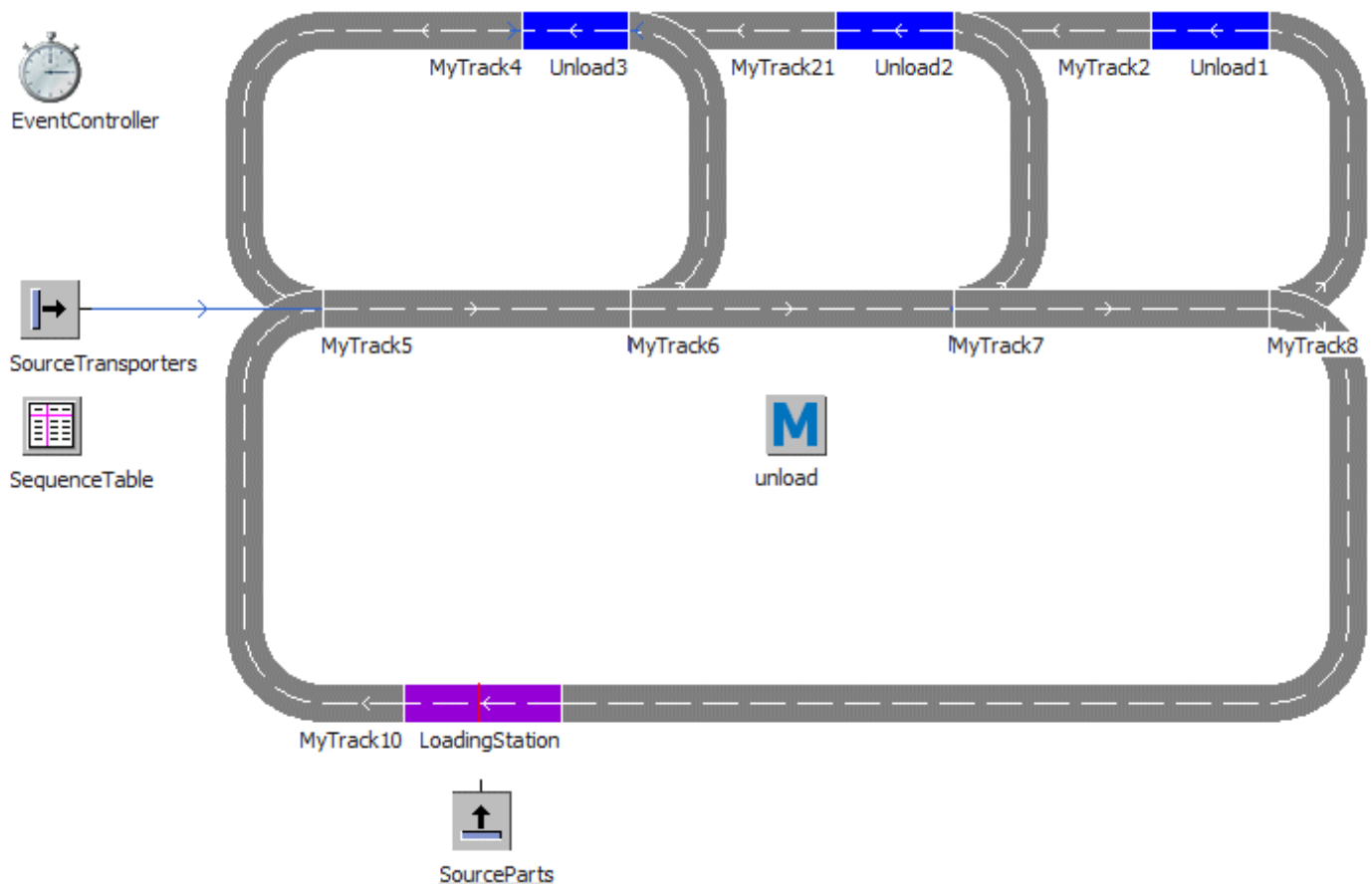
☐ Light Barrier Mode

Activate Always ▾

Control self.OnSensor1 ...

OK Cancel Apply

Finally we insert the two *Sources* that produce the *Transporters* and the *Parts*, the *SequenceTable*, and the method *unload*. Our model then looks like this:



Back to [Work with Automatic Routing of the Transporter](#)

Program the Control for Loading the Transporter

After we created the route network, the *LoadingStation*, and the *unloading stations*, we program the **Sensor Control** for loading the parts onto the *Transporter*.

In the **Sensor Control** we define that the *Transporter* waits until the *SourceParts* has produced a part of type *MyPart*. As soon as a part is located on the *LoadingStation*, it is loaded onto the *Transporter*. The anonymous identifier ? designates the part that is assigned to the contents of the calling object.

Next we determine the **Destination** with the instruction `@.Destination`. To do so, we differentiate the consecutive number of the *Transporter*, `@.ID`, using the modulo function (mod 2 or mod 3) for integer remainders. This way each third part receives an identical **Destination**. This is one of the *unloading stations* *Unload1*, *Unload2* or *Unload3*. After the **Destination** is assigned, each *Transporter* is assigned the method named *unload* as **Destination Control**, *DestCtrl*.

Our source code of the **Sensor Control** looks like this:

```
param SensorID: integer, Front: boolean

waituntil SourceParts.Occupied prio 1
SourceParts.Cont.move(@)
if (@.ID mod 2) = 0
    @.Destination := Unload2
elseif (@.id mod 3) = 0
    @.Destination := Unload3
else
    @.Destination := Unload1
```

```
end
@.DestCtrl := "unload"
```

Back to [Work with Automatic Routing of the Transporter](#)

Configure the Sources and the Transporter

In this step we configure the *SourceTransporters*, which produces the *Transporters* and the *SourceParts*, which produces the parts.

- As we always want to assign a **Destination** to the *Transporter* in our model, we selected the **MU Selection > Sequence** for the *SourceTransporters* and our *SequenceTable* as the required table:

We entered our *LoadingStation* as the **Destination** into the subtable **Settings**. Apart from that we use the default settings.

The screenshot shows the configuration window for **.Models.Model.SourceTransporters**. The window has a menu bar with **Navigate**, **View**, **Tools**, and **Help**. Below the menu bar, there are input fields for **Name** (SourceTransporters) and **Label**. There are also checkboxes for **Failed** and **Exit Locked**, and a dropdown menu for **Planned**. Below these fields, there are tabs for **Attributes**, **Failures**, **Controls**, **Exit**, **Statistics**, **Importer**, and **User-defined**. The **Attributes** tab is selected, showing the following settings:

- Operating Mode**: ☒ Blocking
- Time of Creation**: **Interval Adjustable** (dropdown), **Amount**: -1
- Interval**: **Const** (dropdown), 0
- Start**: **Const** (dropdown), 0
- Stop**: **Const** (dropdown), 0
- MU Selection**: **Sequence** (dropdown), ☐ Generate as Batch
- Table**: **SequenceTable** (dropdown)

object 1	integer 2	string 3	table 4
string MU	Number	Name	Attributes
1 *.UserObjects.MyTransporter	3	MyTransporter	Properties
2			
3			
4			
5			
6			
7			
8			

string 1	integer 2	boolean 3	string 4
string	Name of Attribute	Integer	Boolean
1	Destination		LoadingStation
2			

We connect the *SourceTransporters* with the object *MyTrack5* with a *Connector*.

- For the *SourceParts* we only selected our part named **MyPart** as the **MU**. Apart from that we use the default settings.

As we want to move the parts with a control to the *LoadingStation*, we do not need a *Connector* between the two stations.

.Models.Model.SourceParts

Name: ☐ Failed

Label: ☐ Planned ☐ Exit Locked

Attributes | Failures | Controls | Exit | Statistics | Importer | User-defined

Operating Mode: ☒ Blocking ☐

Time of Creation: ☐ Amount: ☐

Interval: ☐

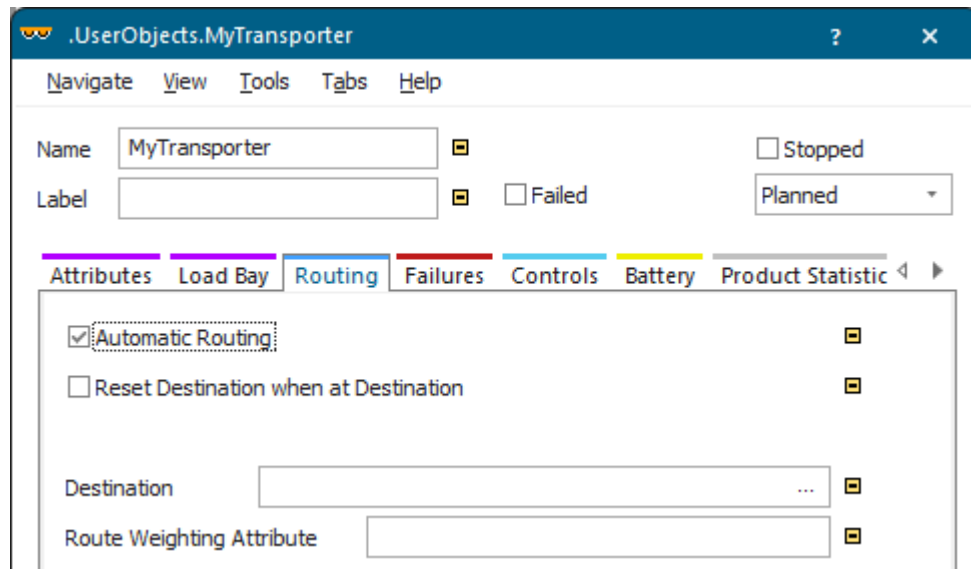
Start: ☐

Stop: ☐

MU Selection: ☐

MU: ☐

- For our *Transporter* named *MyTransporter* we activate **automatic routing** on the tab **Routing**.



- We use our *part* named *MyPart* with the default settings.

Back to [Work with Automatic Routing of the Transporter](#)

Program the Control for Unloading the Transporter

In this step we program our control for unloading the *Transporter*. We do this in the method named *unload*. This control is going to be called when the *Transporter* has reached its **Destination**.

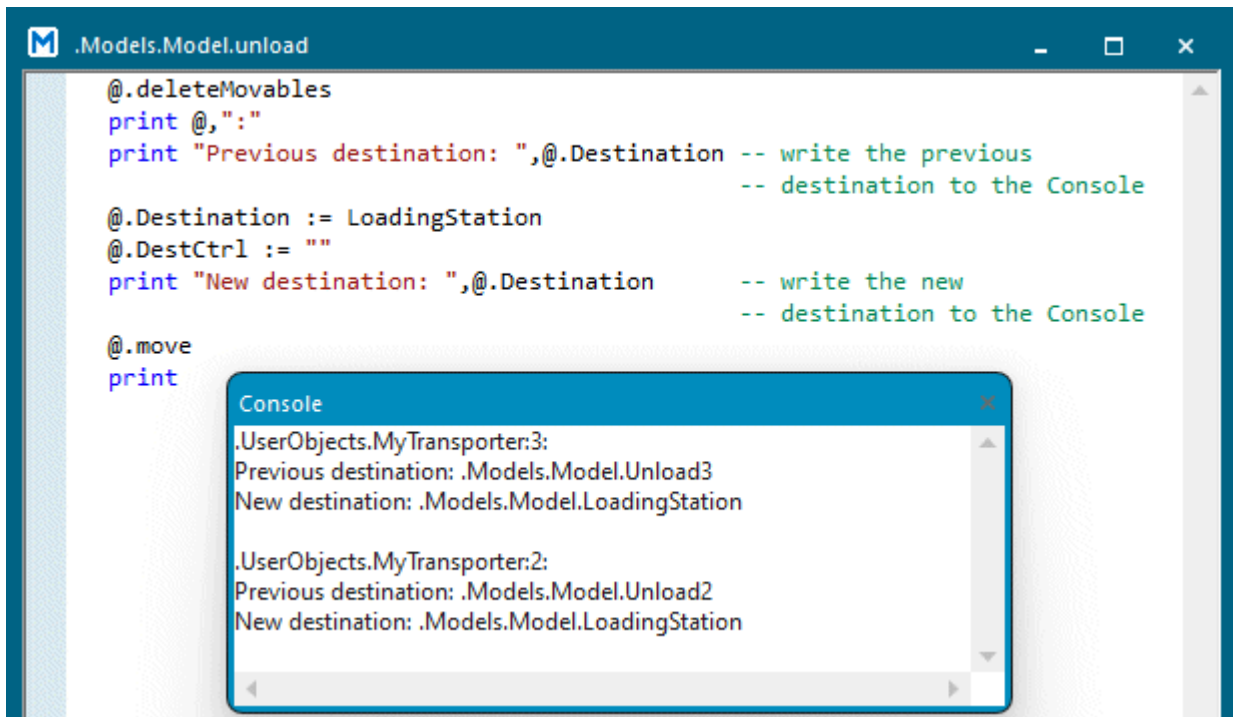
The *unload* control unloads the part from the bed of the *Transporter*, assigns the now empty *Transporter* to the *LoadingStation* as new **Destination** and removes the **Destination Control**, *DestCtrl*. In addition the control writes the name of the *Transporter*, the previous destination, and the new destination to the *Console*.

Our source code looks like this:

```
@.deleteMovable
print @, ":"
print "Previous destination: ", @.Destination // write the previous
                                              // destination to the Console

@.Destination := LoadingStation
@.DestCtrl := ""
print "New destination: ", @.Destination    // write the new
                                              // destination to the Console

@.move
```



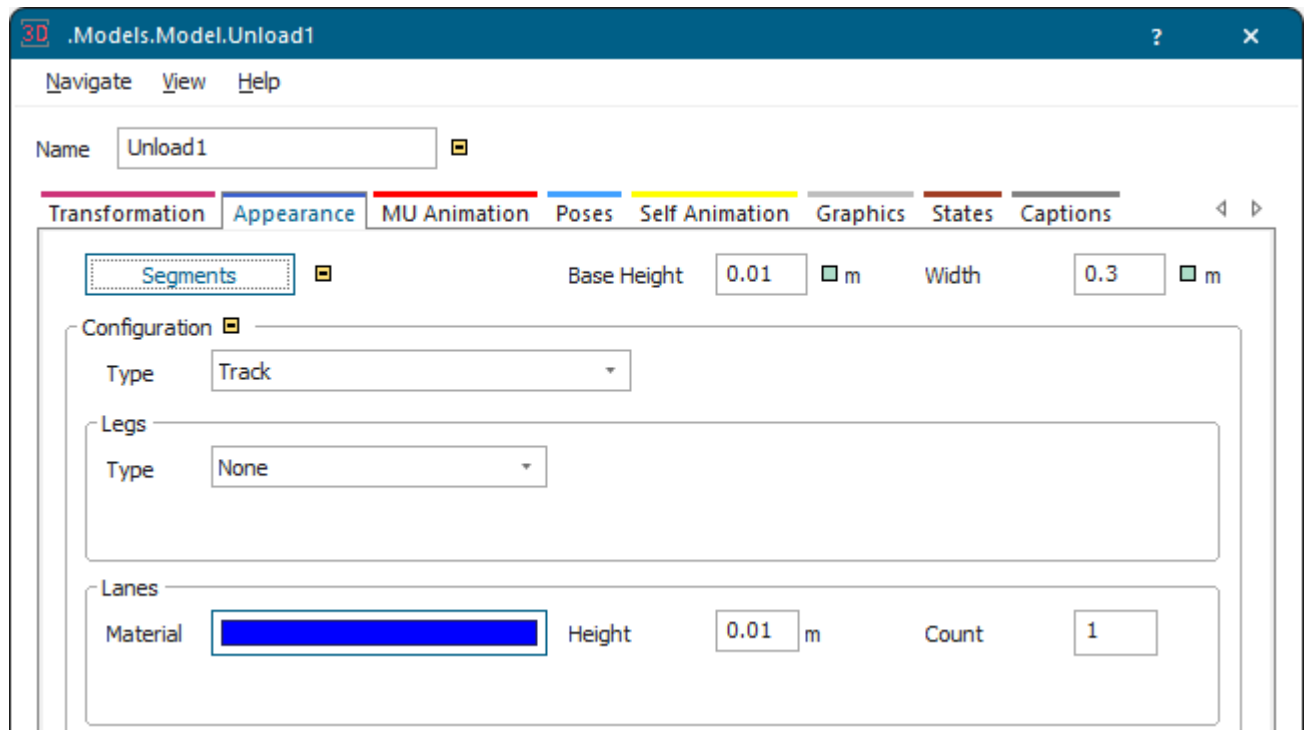
Back to [Work with Automatic Routing of the Transporter](#)

Make the 3D Model Visually Pleasing

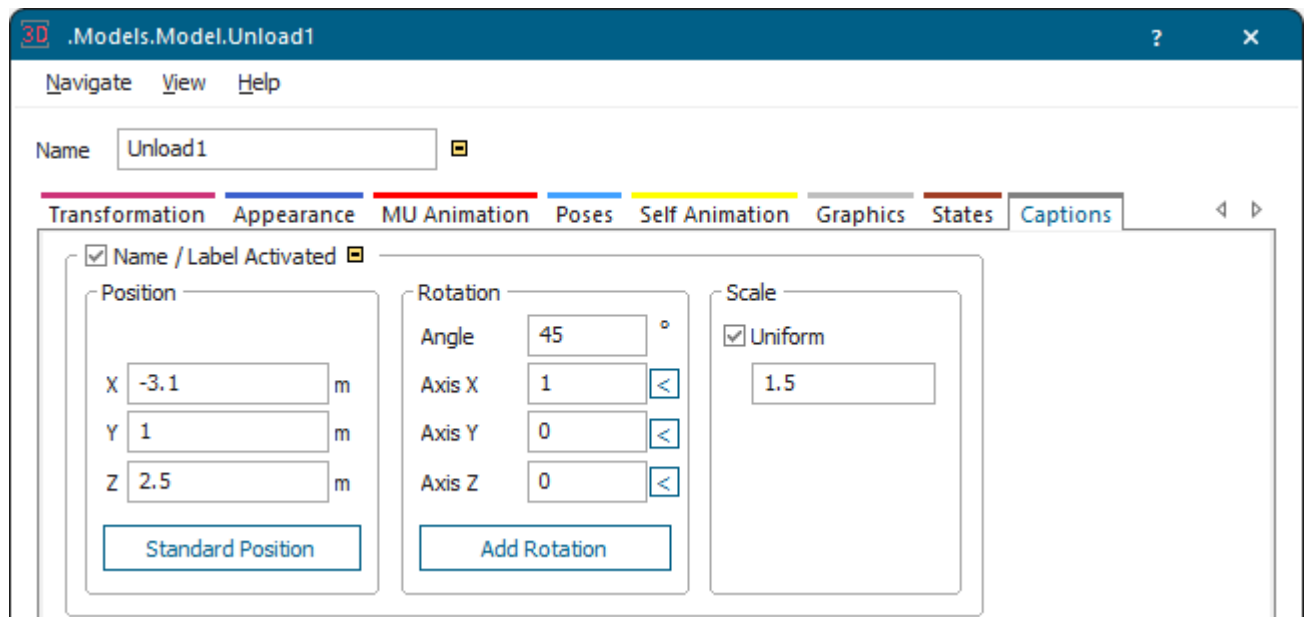
In this step we demonstrate how to make the 3D model visually pleasing.

[Use New Standard Graphics of the 2D Objects](#) shows how we replaced the outdated icon of the *Track* in our existing simulation model with the icon of the *Track* of version 15 and higher.

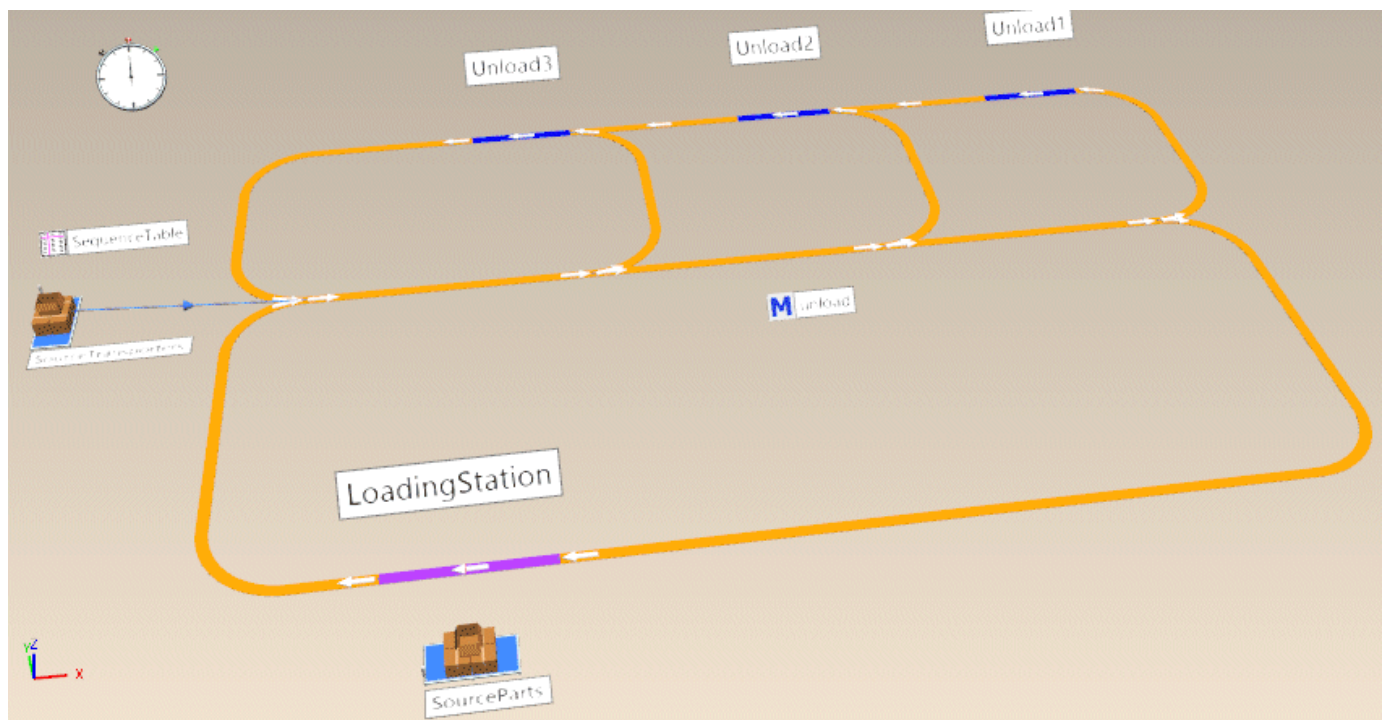
To better be able to tell the *LoadingStation* and the *unloading stations*—*Unload1*, *Unload2* and *Unload3*—apart from the other route pieces, we selected different colors for the lanes of these *Tracks* in 3D.



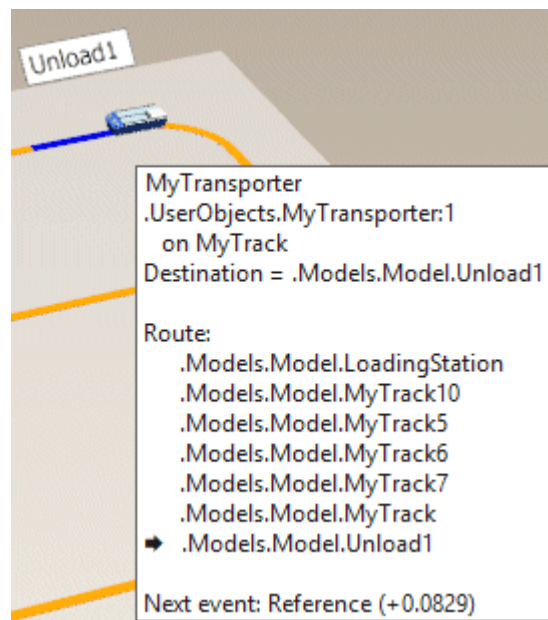
In addition we activated and displayed the **Captions** of the *LoadingStation* and of the *unloading stations*. We selected these and similar settings:



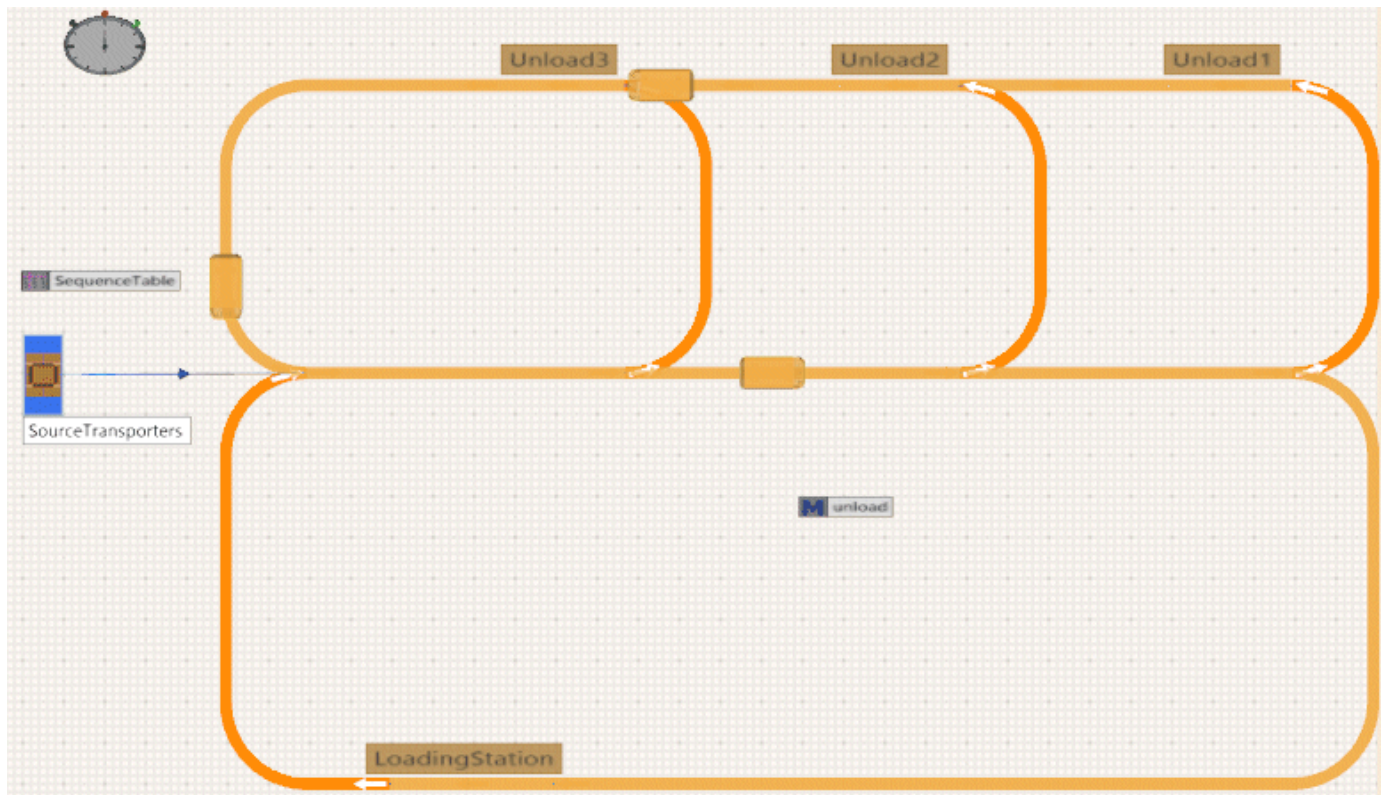
Click  **Names** on the tab **View** of the 3D ribbon tab to show the names of the objects. Start the simulation, reduce the simulation speed and watch how the *Transporters* move to the same *UnloadingStation* each third time and unload the parts there.



Drag the mouse over the *Transporter* to show its name, its current location, its destination, and the previous route in a *Tooltip*.



The command **View > Route to Destination** selects the objects that are part of the route to the destination in the *Frame*.



Back to [Work with Automatic Routing of the Transporter](#)

Working with Automatic Routing of Part and Container

Work with Automatic Routing of Part and Container

In this sample model we demonstrate how a *Pallet (Container)* automatically finds its destination by using its destination object. As soon as the destination object is assigned a reachable destination, *Plant Simulation* the route which describes the way on which the *Pallet* reaches its destination traveling on the *Conveyors*.

The *Part* and the *Container* find their **Destination** along the **fastest** route.

The *Source* named *SourcePallets* creates *Pallets*, which wait for parts at the sensor of the *LoadingStation*.

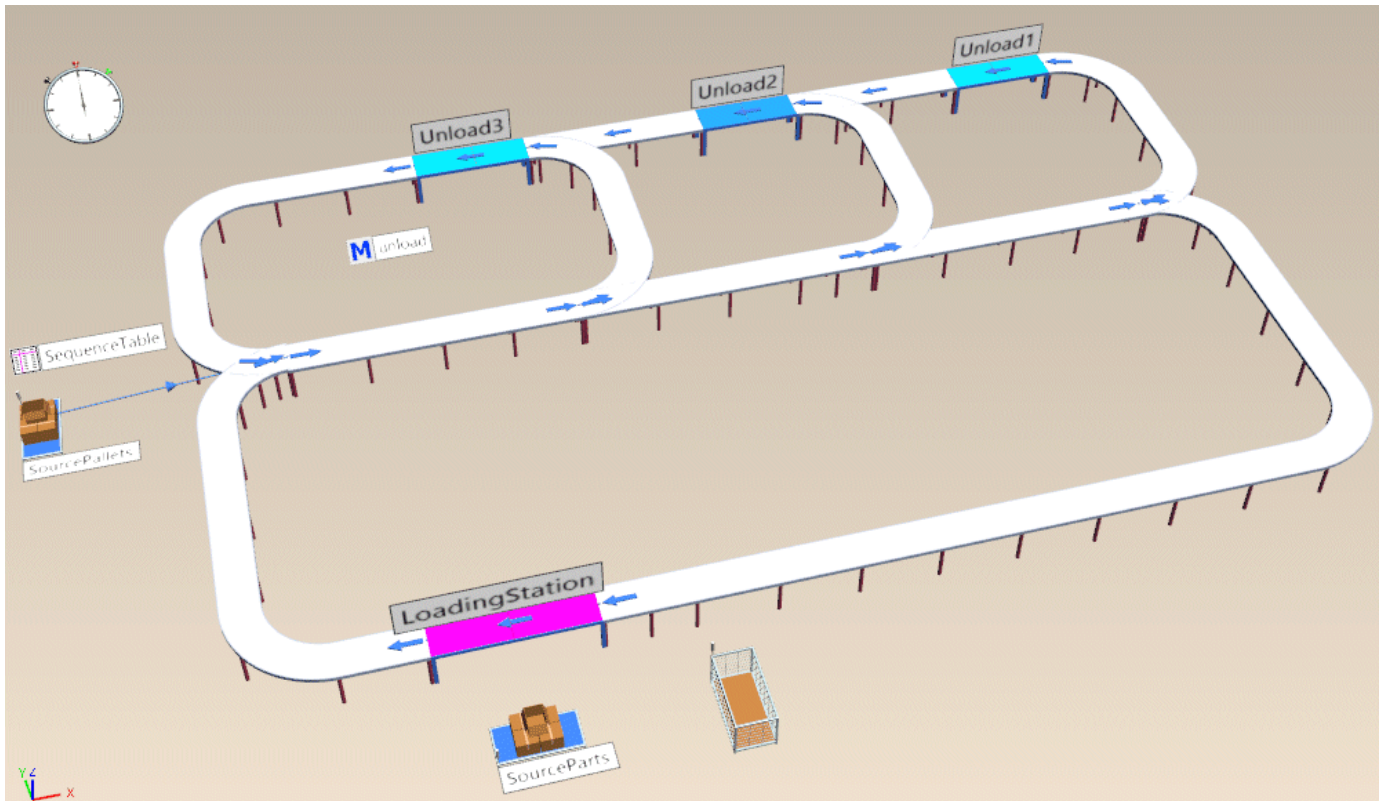
In the sensor control we set that the *Pallet* waits until the *SourcePallets* has produced a part of type *MyPart*. Once a part is located on the *LoadingStation*, this part is loaded onto the *Pallet*. Then we determine the destination object of the parts, i.e., one of the stations *Unload1*, *Unload2*, or *Unload3*. We inserted a sensor each into these stations. When the *Pallet* has reached the destination object, the part will be unloaded and the *Pallet* returns *LoadingStation*, where it is loaded anew.

We will demonstrate how to:

- [Insert and Configure the Conveyors](#)

- Configure the Unloading Stations
- Program the Control for Loading the Pallet
- Configure the Sources and the Pallet
- Program the Control for Unloading the Pallet
- Make the Model Visually Pleasing

Our simulation model looks like this:



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

Insert and Configure the Conveyors

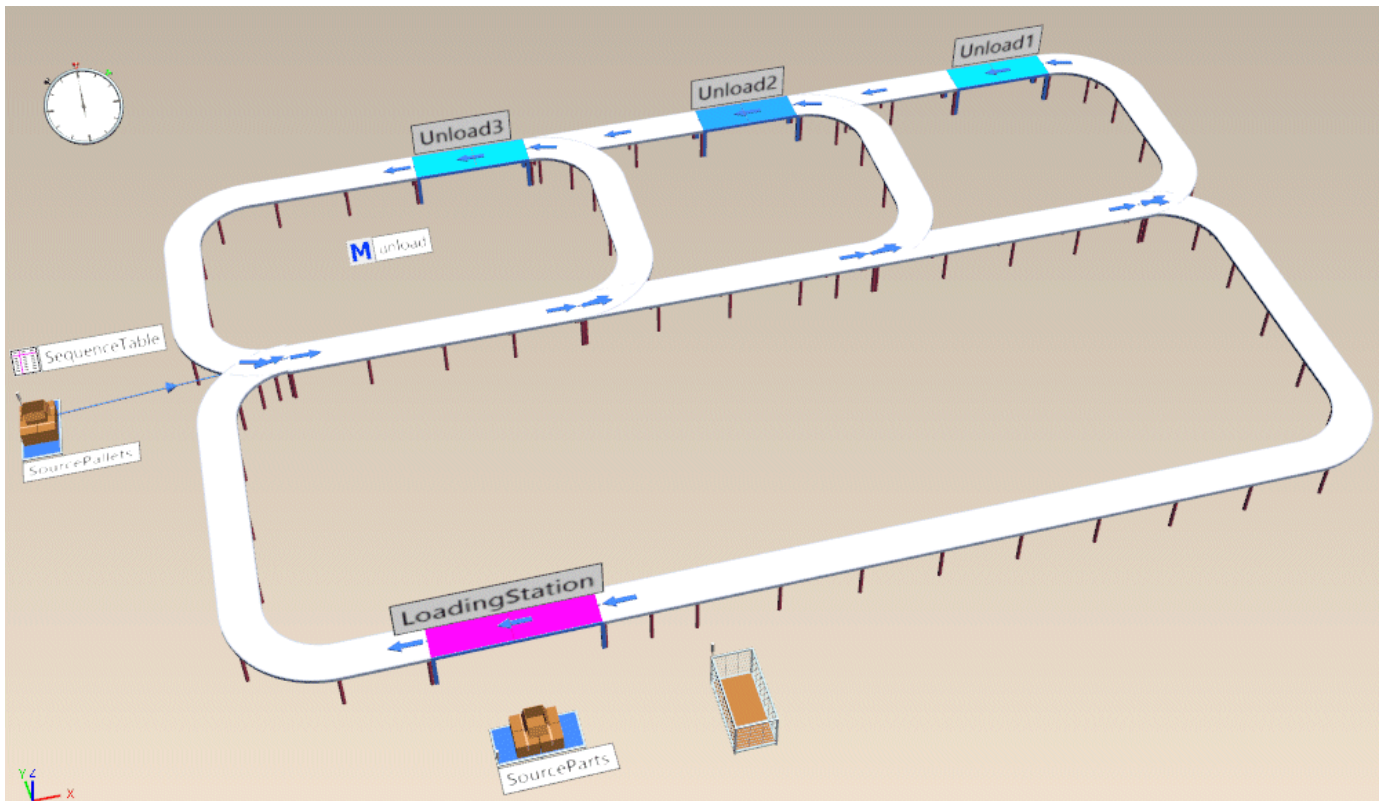
Before we start modeling, we create the objects that we need as we do not want to change the built-in object classes.

We duplicated the *Source*, the *Conveyor*, the *Connector*, the *Container*, the *Part*, the *Method*, and the *DataTable* in their respective folders. *Plant Simulation* places the duplicated objects into the folder **UserObjects** in the *Class Library*. We renamed the objects and added them to the toolbar **User Objects** with drag-and-drop.

Now we model our route network on which the *Container* travels. To simplify matters, we saved our model for automatic routing of the *Transporter* under a new name.

Then we held down **Alt**, dragged the object *MyConveyor* in the folder **User Objects** in the *Class Library* onto the object *MyTrack* and dropped it there. This replaced all instances of the object *MyTrack* with instances of the object *MyConveyor*. This way we do not have to manually create the route network. As we kept the changes to the instances of the *Track*, we now have to manually rename the individual route pieces, for example from **MyTrack1** to *MyConveyor1*, etc.

Next we configure the *LoadingStation* and the three *unloading stations*, *Unload1*, *Unload2*, and *Unload3*.



Then we change the **sensor** of the *LoadingStation* at which the parts are going to be loaded onto the *Transporters*. Click the right mouse button on the sensor of the *LoadingStation* and select **Open Sensor**. We selected these settings. As the **Destination** is the *LoadingStation* itself, *Plant Simulation* enters **self**.

Finally we configure our *Sources* that produce the *Pallets* and the *Parts*.

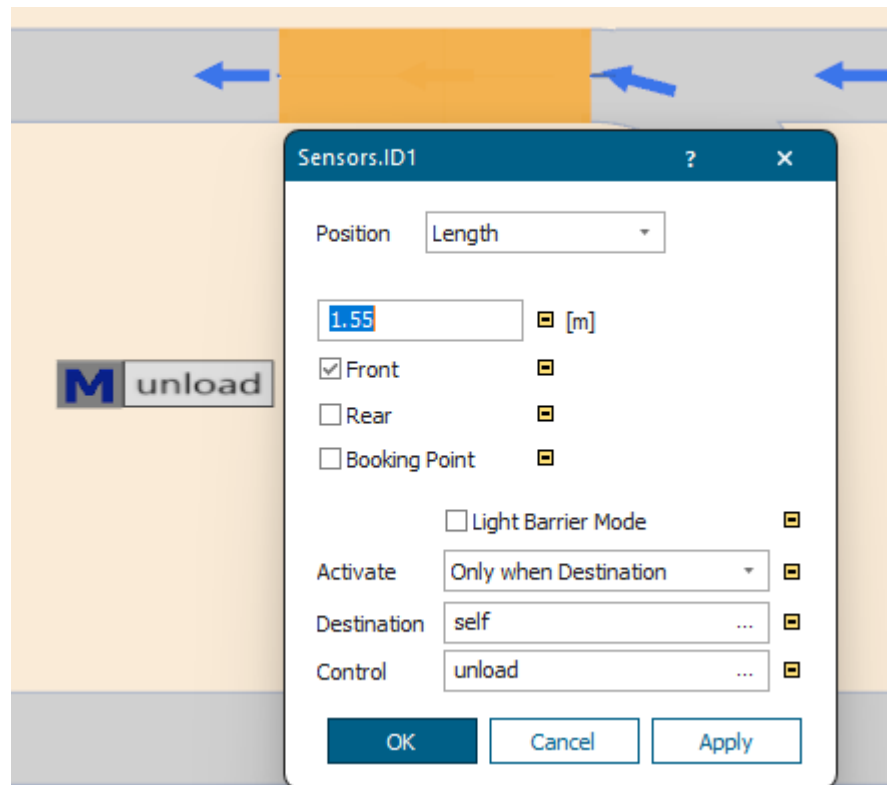
Back to [Work with Automatic Routing of the Transporter](#)

Configure the Unloading Stations

For each of the three *unloading stations*—*Unload1*, *Unload2*, and *Unload3*—we create a **sensor** each in their center at which the parts are going to unloaded.

As we only want to activate the sensor when the *Pallet* has the same **Destination** as the one we selected in the **Sensor**, we select **Only when Destination**. The **Destination** of the action, unloading the parts in our case, is the respective station. *Plant Simulation* enters **self** when we select the respective *unloading station*.

As **Control** we enter our method named *unload* in each case.



Back to [Work with Automatic Routing of the Transporter](#)

Program the Control for Loading the Pallet

After we created the route network, the *LoadingStation*, and the *unloading stations*, we program the **Sensor Control** for loading the parts onto the *Pallet*.

In the **Sensor Control** we define that the *Pallet* waits until the *SourceParts* has produced a part of type *MyPart*. As soon as a part is located on the *LoadingStation*, it is loaded onto the *Pallet*. The anonymous identifier ? designates the part that is assigned to the contents of the calling object.

Next we determine the **Destination** with the instruction `@.Destination`. To do so, we differentiate the consecutive number of the *Pallet*, `@.ID`, using the modulo function (mod 2 or mod 3) for integer remainders. This way each third part receives an identical **Destination**. This is one of the *unloading stations* *Unload1*, *Unload2* or *Unload3*. After the **Destination** is assigned, each *Pallet* is assigned the method named *unload* as **Destination Control**.

Our source code of the **Sensor Control** looks like this:

```
param SensorID: integer, Front: boolean

waituntil SourceParts.Occupied prio 1
SourceParts.Cont.move(@)
if (@.ID mod 2) = 0
    @.Destination := Unload2
elseif (@.id mod 3) = 0
    @.Destination := Unload3
else
    @.Destination := Unload1
end
```

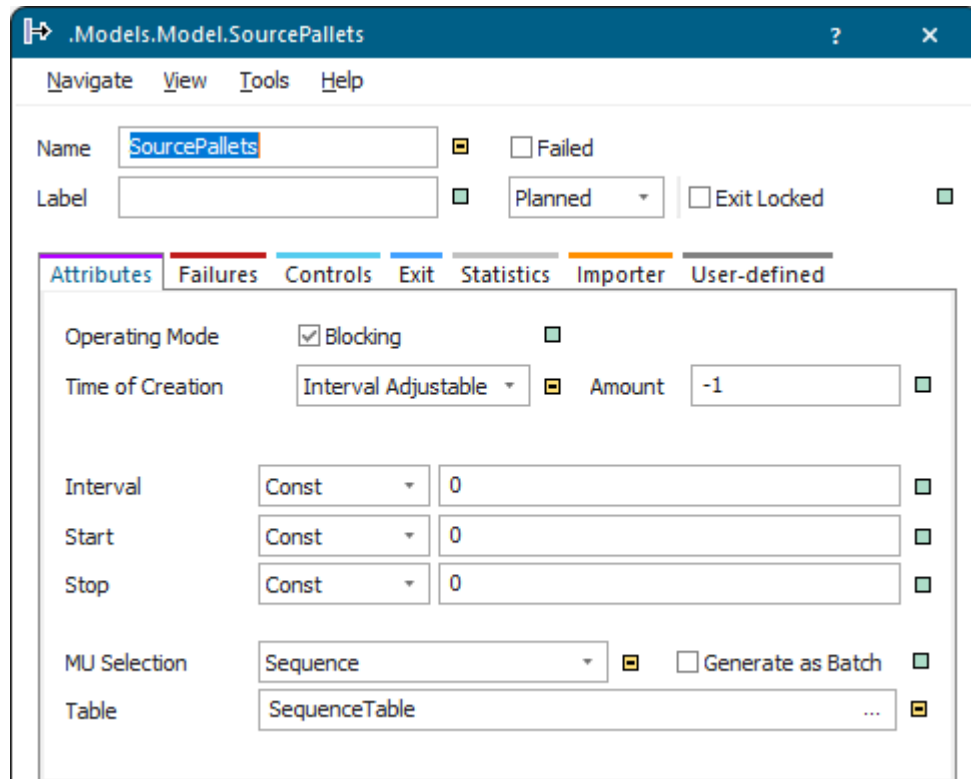
Back to [Work with Automatic Routing of the Transporter](#)

Configure the Sources and the Pallet

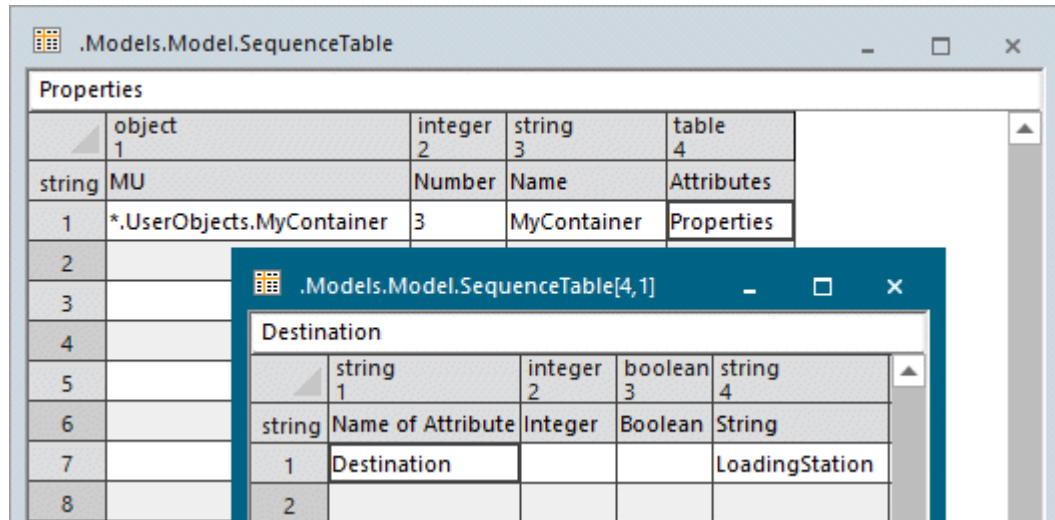
Now we configure the *SourcePallets*, which produces the *Pallets* and the *SourceParts*, which produces the *parts*.

Proceed as follows:

- As we always want to assign a **Destination** to the *Pallet* in our model, we selected the **MU Selection > Sequence** for the *SourcePallets* and for the required table our *SequenceTable*:



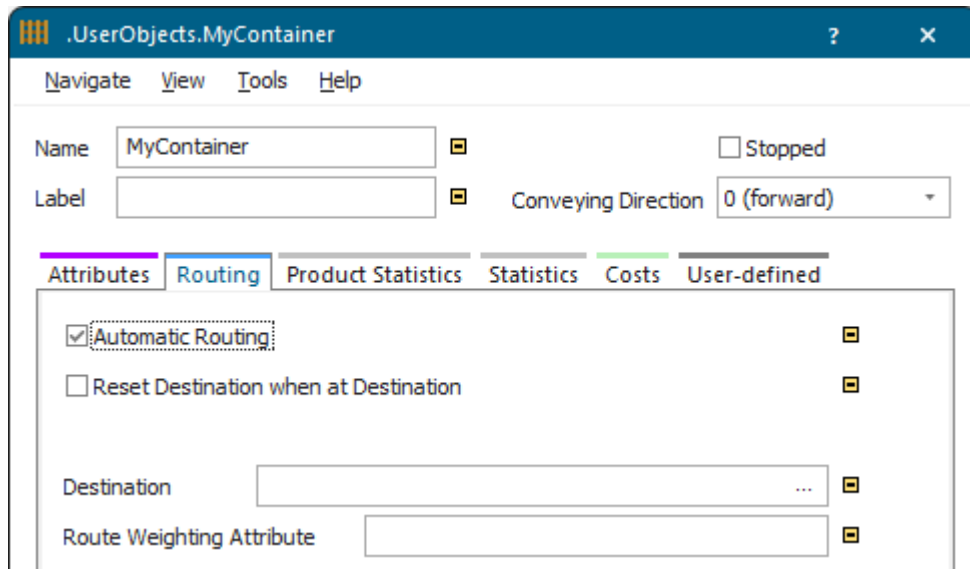
We entered our *LoadingStation* as the **Destination** into the subtable **Settings**. Apart from that we use the default settings.



- For the *SourceParts* already contains as **MU** our part named **MyPart**.

As we want to move the parts with a control to the *LoadingStation*, we do not need a *Connector* between the two stations.

- For our *Pallet* named *MyContainer* we activated **Automatic Routing** on the tab **Routing**.



- We use our *part* named *MyPart* with the default settings.

Back to [Work with Automatic Routing of the Transporter](#)

Program the Control for Unloading the Pallet

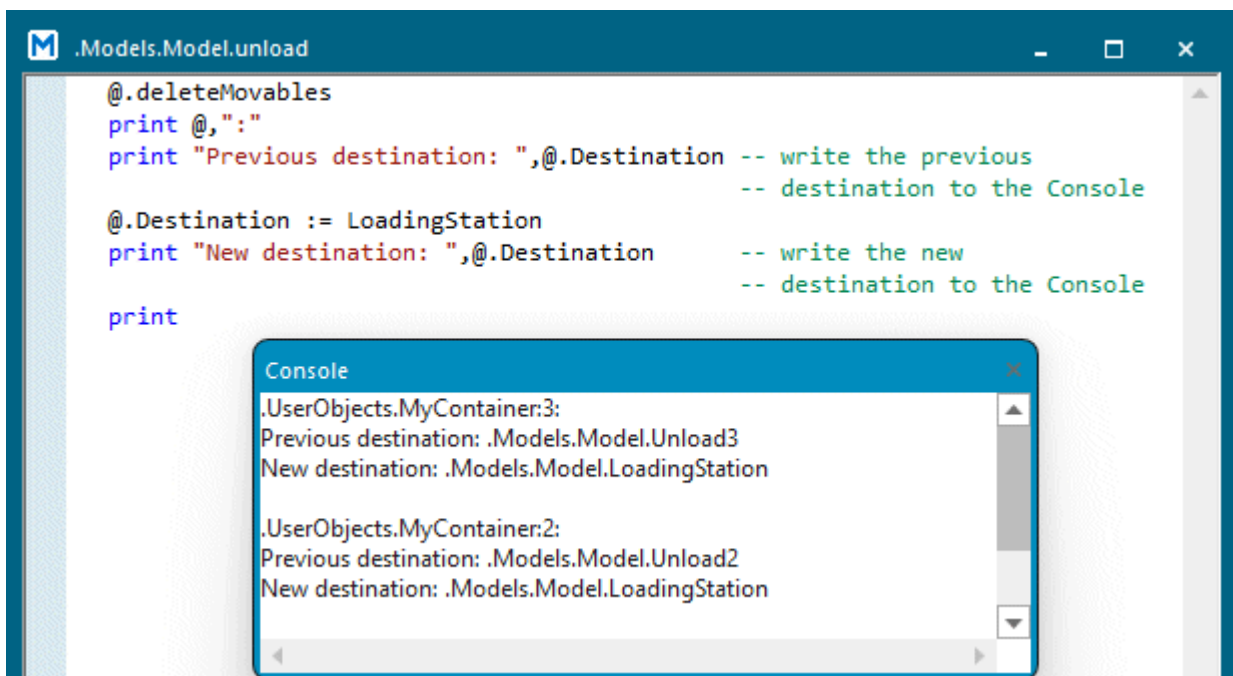
Finally we program our control for unloading the *Pallet*. We do this in the method named *unload*. This control is going to be called when the *Pallet* has reached its **Destination**.

The *unload* control unloads the part from the *Pallet*, assigns the now empty *Pallet* to the *LoadingStation* as new **Destination**. In addition the control writes the name of the *Pallet*, the previous destination and the new destination to the *Console*.

Our source code looks like this:

```
@.Cont.move(MyDrain)
print @, ":"
print "Previous destination: ", @.Destination // write the previous
                                              // destination to the Console

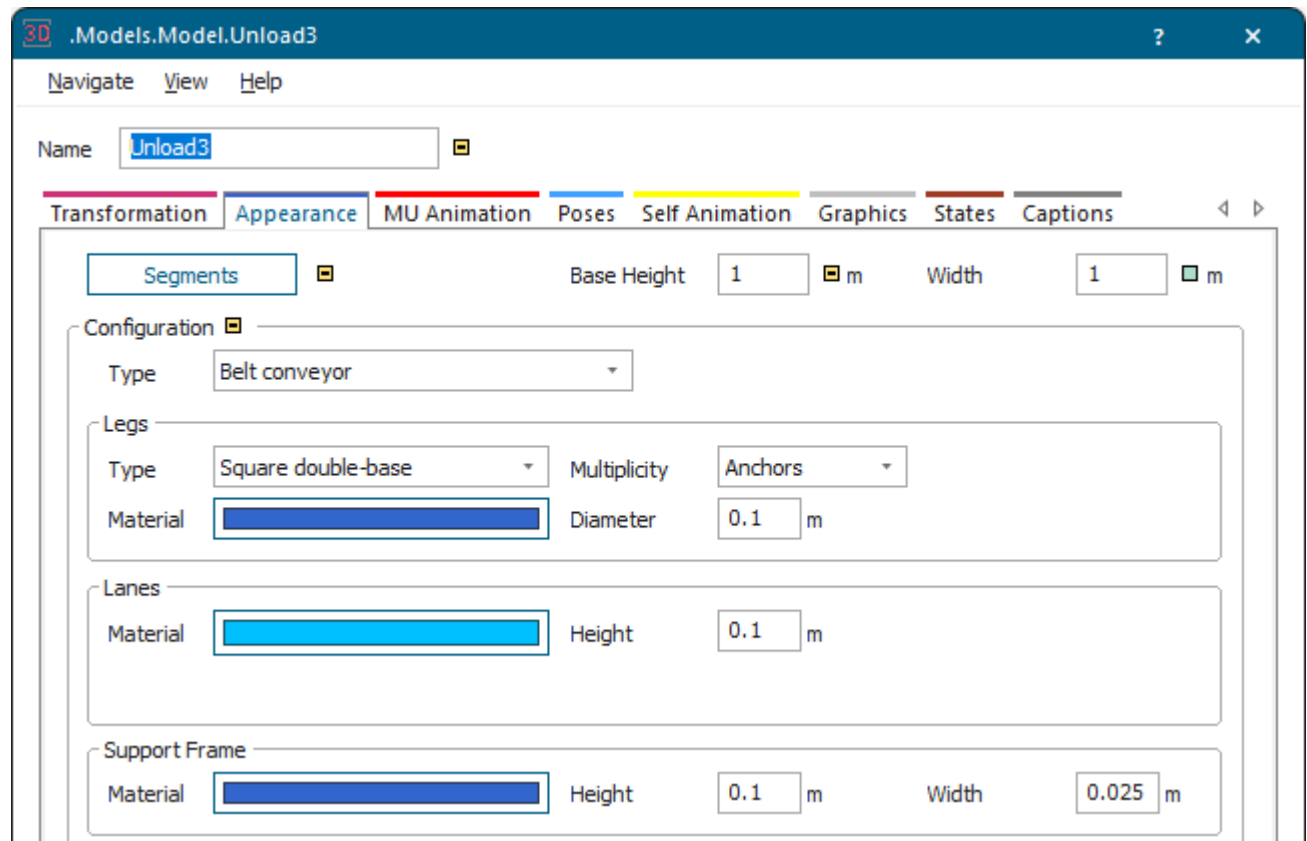
@.Destination := LoadingStation
print "New destination: ", @.Destination    // write the new
                                              // destination to the Console
print                                       // add an empty line
```



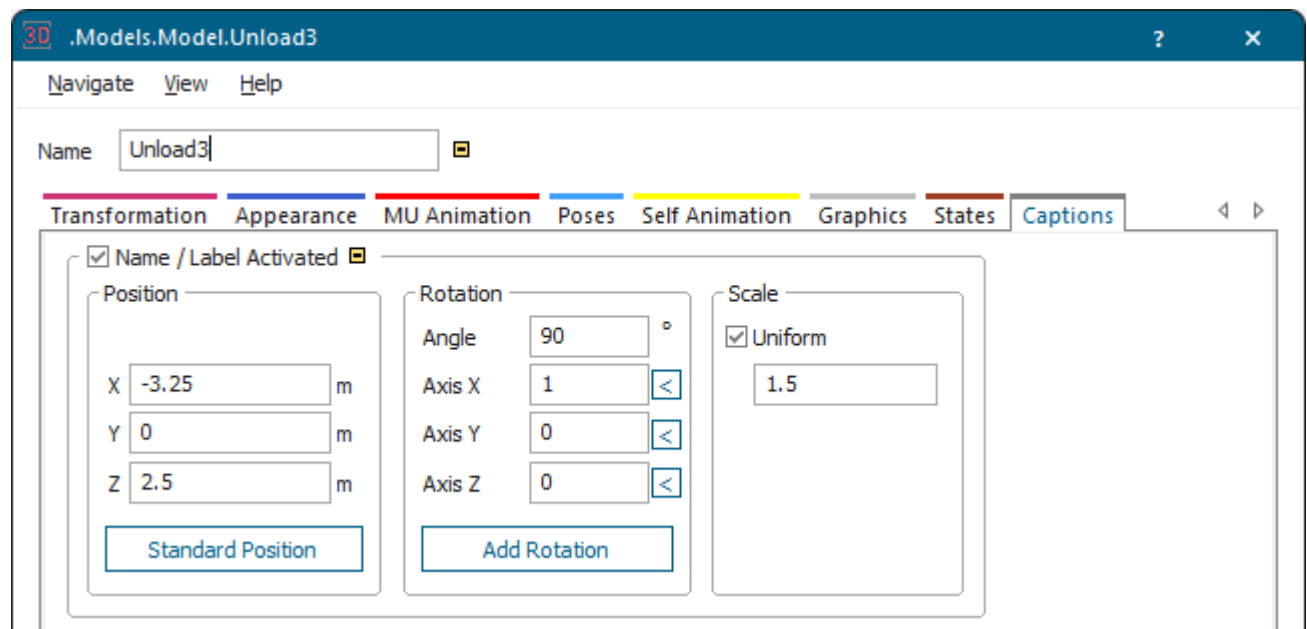
Back to [Work with Automatic Routing of the Transporter](#)

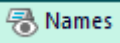
Make the Model Visually Pleasing

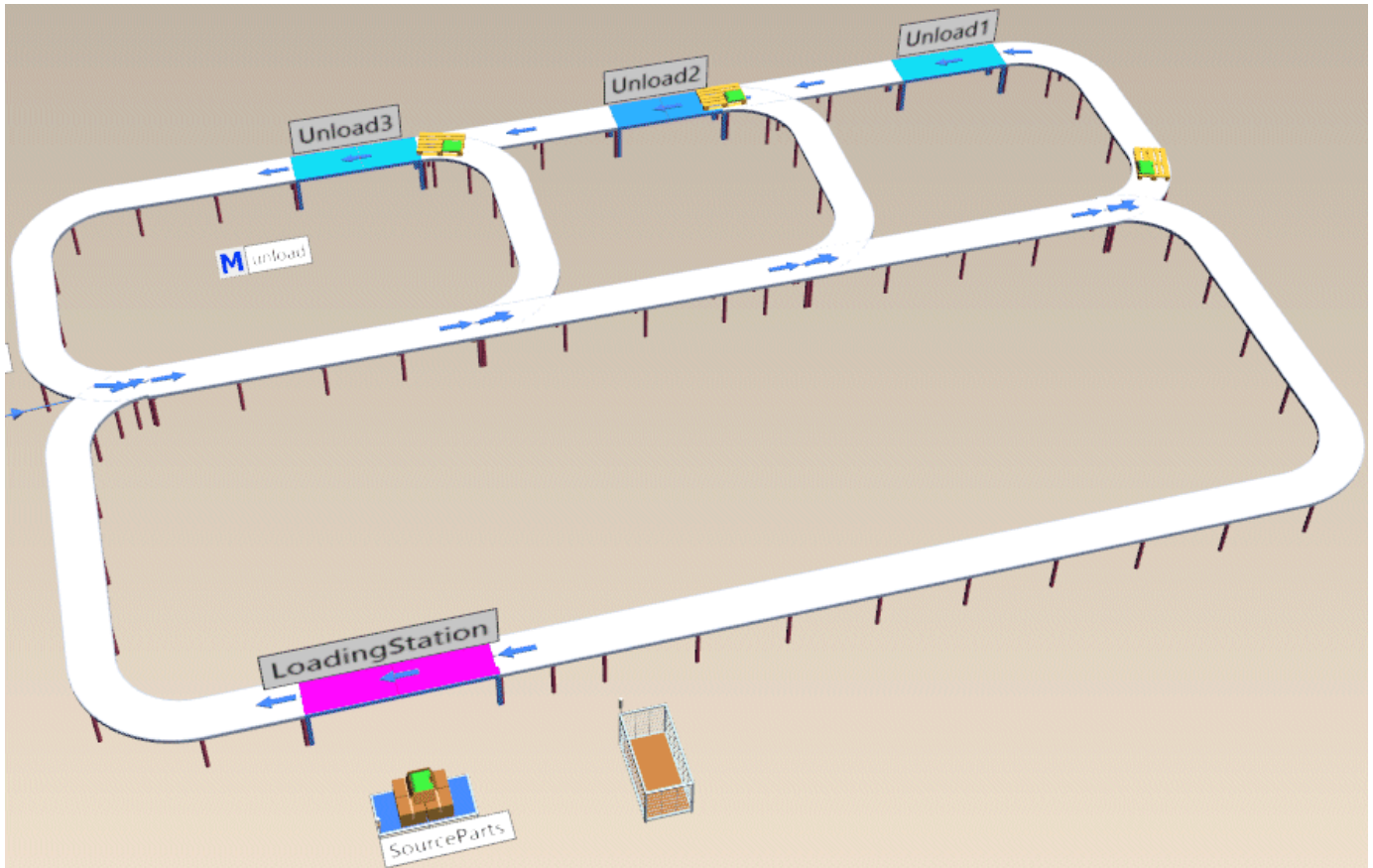
To better be able to tell the *LoadingStation* and the *unloading stations*—*Unload1*, *Unload2*, *Unload3*—apart from the other route pieces, we selected different colors for the lanes of these *Conveyors*.



In addition we activated and displayed the **Captions** of the *LoadingStation* and of the *unloading stations*. We selected these and similar settings:

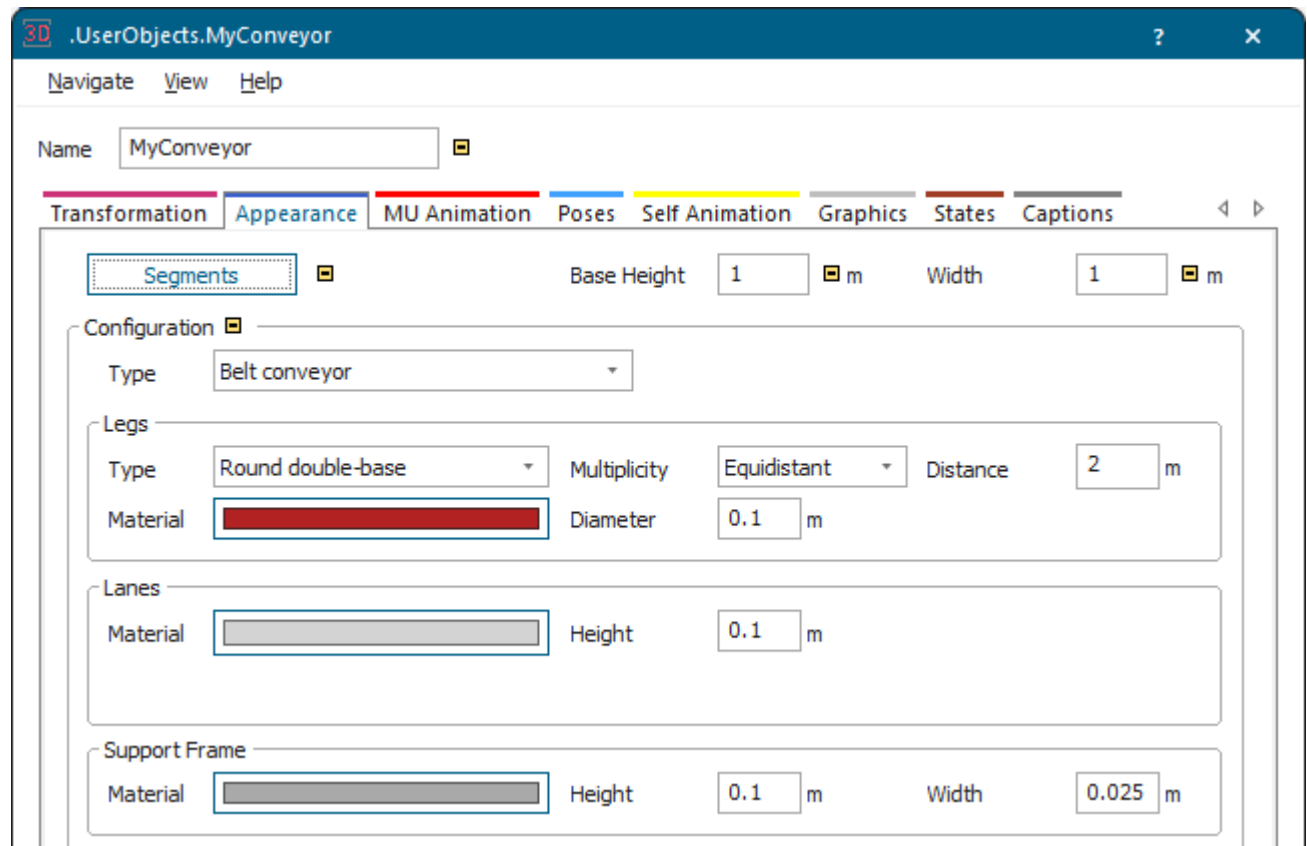


Click  on the **View** ribbon tab to show the names of the objects. Start the simulation, reduce the simulation speed and watch how the *Pallets* move to the same *UnloadingStation* each third time and unload the parts there.

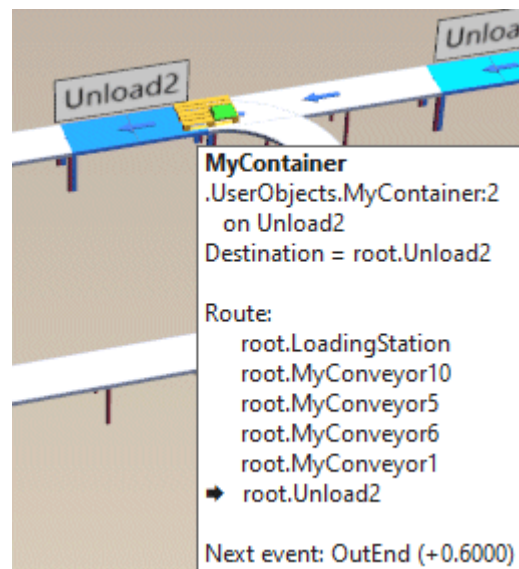


As we created our *Conveyors* by replacing the *Tracks* from another model with our *Conveyor*, we have to adjust the **Base Height** for each *instance* of the route piece individually, by entering 1 into the text box **Base Height**. While the *Tracks* are placed flush on the floor of the installation, the *Conveyors* usually are placed on legs elevated from the floor.

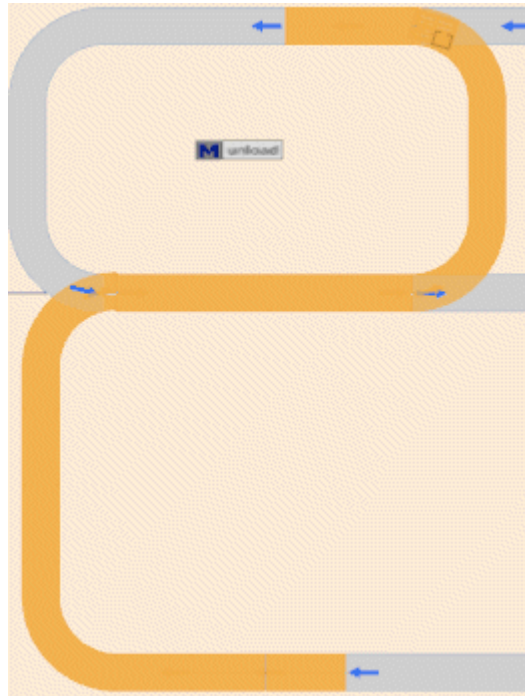
We also selected a number of other settings in the *class* of our *Conveyor*.



Drag the mouse over the *Pallet* to show its name, its current location, its destination, and the previous route in a *Tooltip*.



The command **View > Route to Destination [Part, Container]** selects the objects that are part of the route to the destination in the *Frame*.



Back to [Work with Automatic Routing of the Transporter](#)

Modeling an Automated Guided Vehicle System

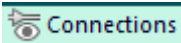
Model an Automated Guided Vehicle System (AGVS)

You can use the resource objects *AGVPool* and *Marker* to model *AGV systems* in your installation in which the AGV, starting at the *AGVPool*, covers the designated route along the *markers* you inserted. The AGV is not bound to a permanently installed route network, but drives freely within the space of the installation.

Note

When you insert *Markers* into your model, make sure that the *Markers* are **aligned**. This prevents *Plant Simulation* from computing unnecessary roundings between the *Markers*. Misaligned *Markers* might occur if you inserted *Markers* into your model while **Show Grid** was off.

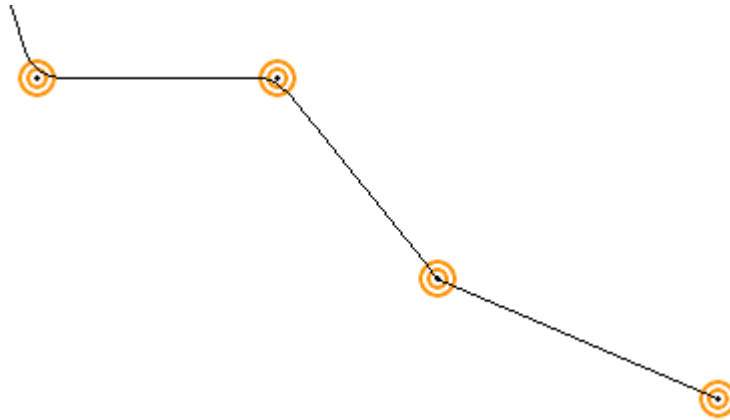
Note

To show or hide a *Marker*, click  **Show Connections**.

We show how the AGVs:

- **Cover a Route Along Omnidirectional Markers**

Here the AGV does not drive across the inserted *marker* but turns in front of the inserted *markers* according to the angle that *Plant Simulation* computed toward the next *marker*. The icon of an *omnidirectional marker* inserted into a *Frame* looks like this .



Set the route pattern which the AGV covers in the method **setRoute** of the *Transporter*:

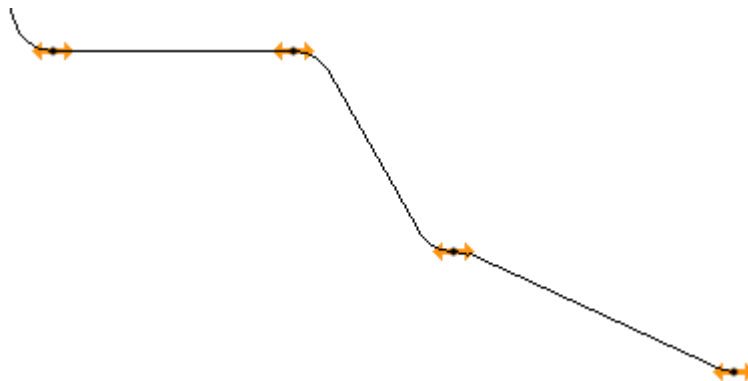
```
var AGV : object := AGVPool.Cont
AGV.setRoute([M1,M2,M3,M4]) // sets the route

waituntil AGV.DestinationWasReached

AGV.DestinationWasReached := false
// recognizes when the destination of the route was reached
print "The AGV is ready for the next route."
```

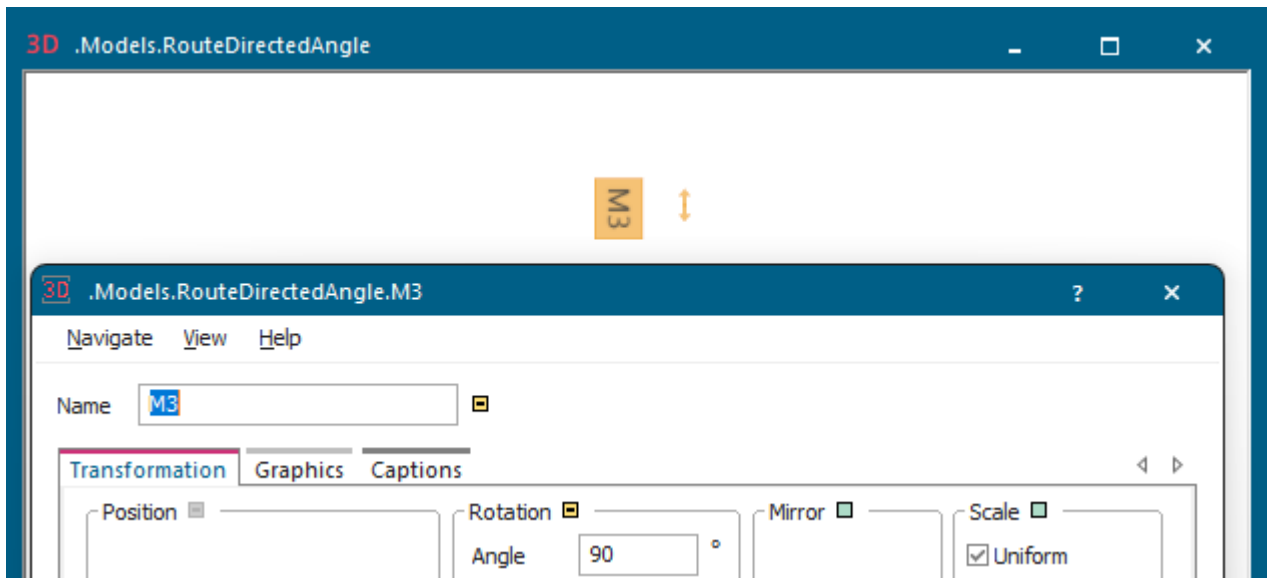
- **Cover a Route Along Directional Markers**

Here the AGV drives onto the *marker* and then rotates toward the *marker* around the default angle of 0° in the direction of the next *marker*. The icon of a *directional marker* inserted into a *Frame* with the default angle looks like this .

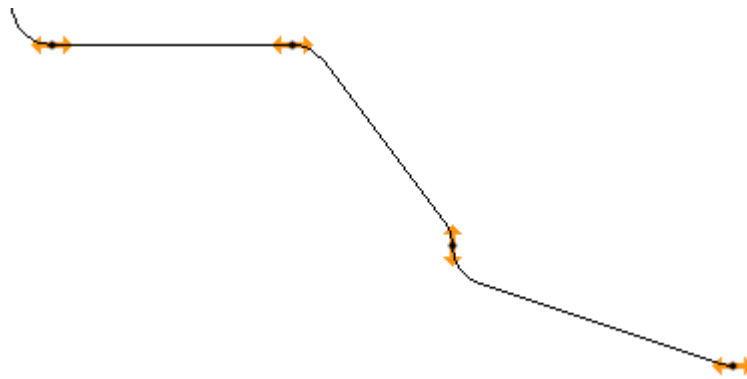


Cover a Route Along Directional Markers with the Defined Angle

Here the AGV drives toward the *marker* at an angle of 90°, which we defined, crosses it, and leaves it at an angle of 90°.



The icon of a *directional marker* inserted into a *Frame* with an angle of 90° looks like this .



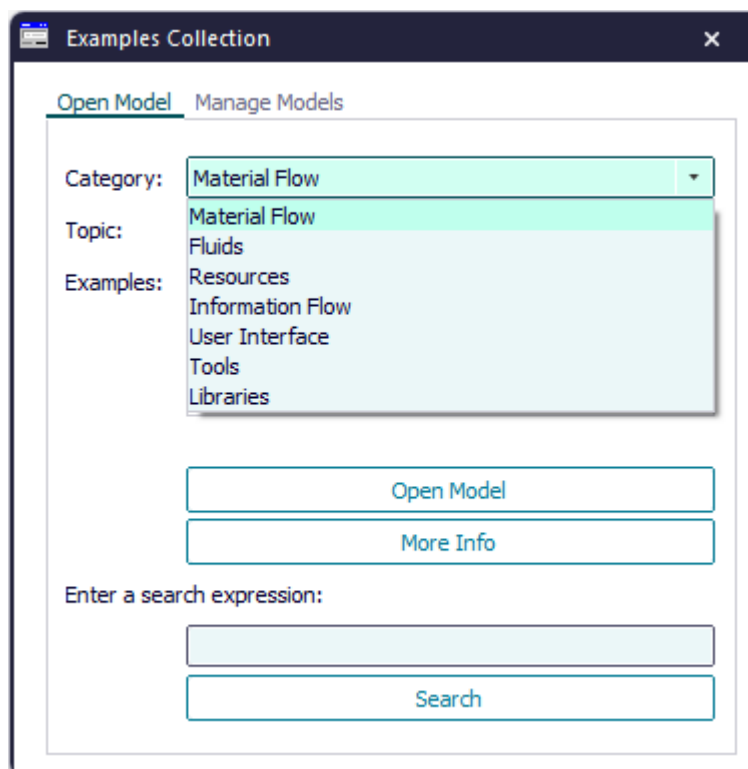
- **Cover a Route According to a Segments Table**

Here the AGV covers the route according to the route segments which we set in a table. To do so, we use the method `setRouteSegments`.

In addition, you can:

- **Load Parts Onto an AGV and Rotate Them**
- **Fine-position an AGV**
- **Prevent AGVs from Colliding**

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



See also [AGV](#)

See also [Marker](#) 

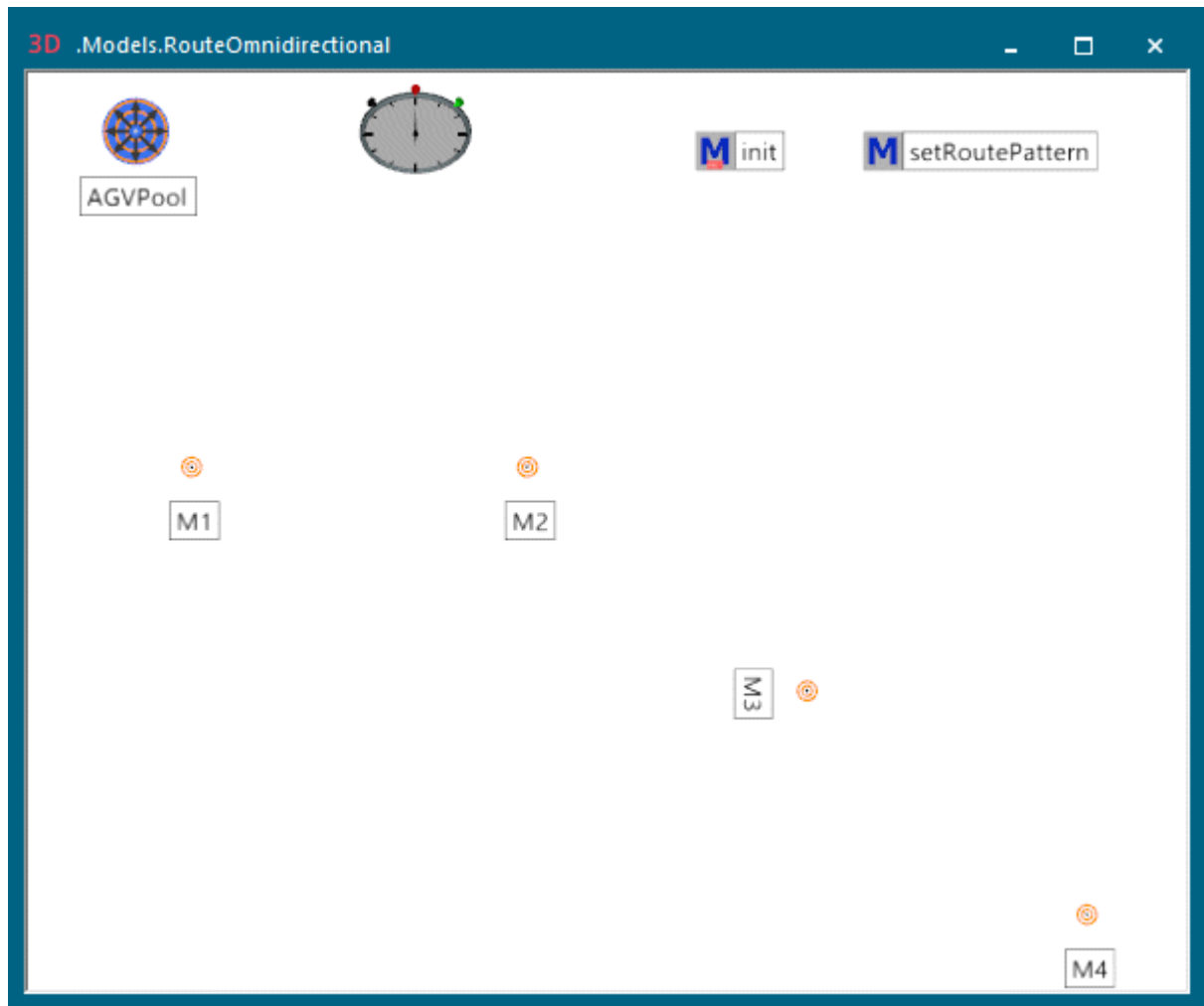
See also [AGVPool](#) 

Back to [Model Transport Systems](#)

Cover a Route Along Omnidirectional Markers

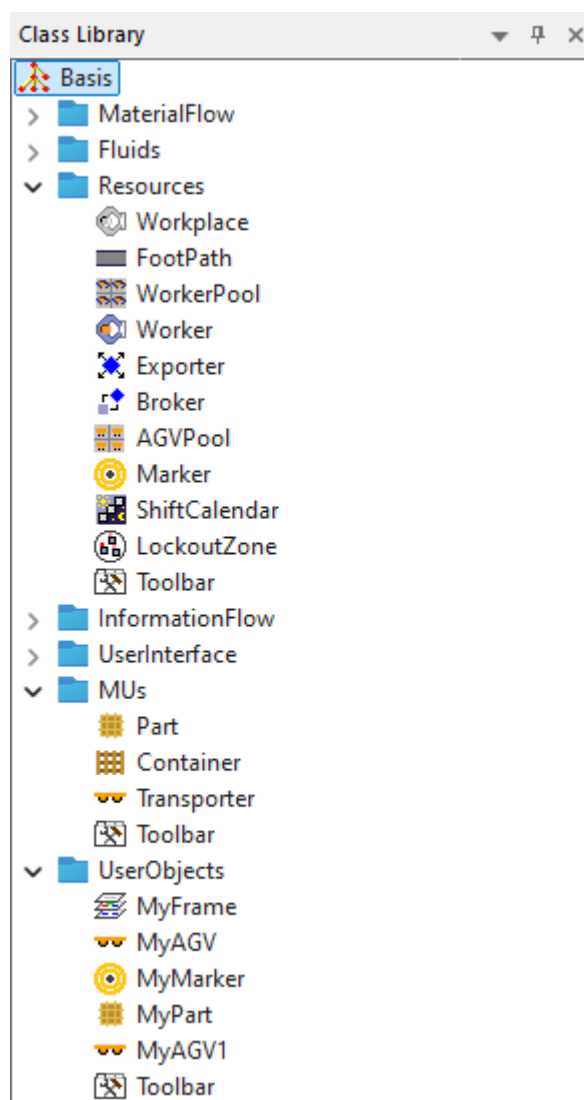
In this simple sample model the AGV covers the set route along *omnidirectional markers*. This means that it does not drive across the *markers* but turns in front of the *marker* toward the next *marker*. *Plant Simulation* computes the curve, which the AGV drives, according to the angle of the connecting line between the current *marker* and the next *marker*.

The finished simulation model looks like this:

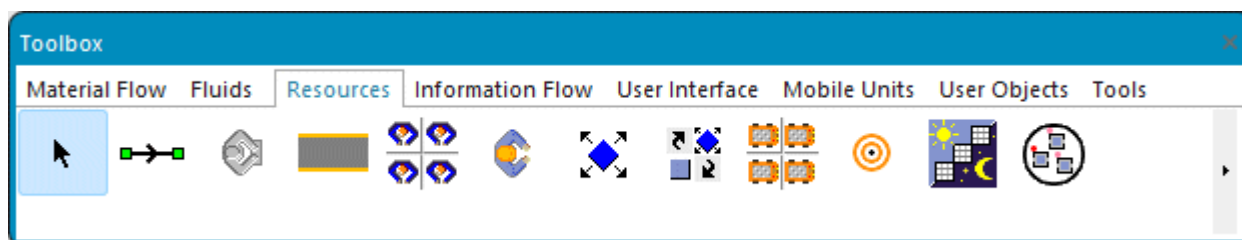


Proceed as follows to create the model:

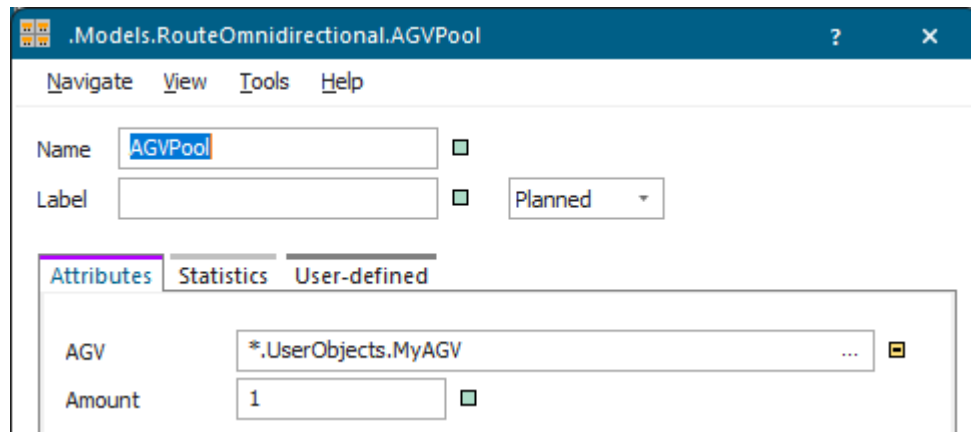
- Duplicate the objects *Transporter* in the folder **MUs** and *Marker* in the folder **Resources** in the *Class Library*. *Plant Simulation* places the duplicated objects into the folder **UserObjects**. We renamed our objects as shown below.



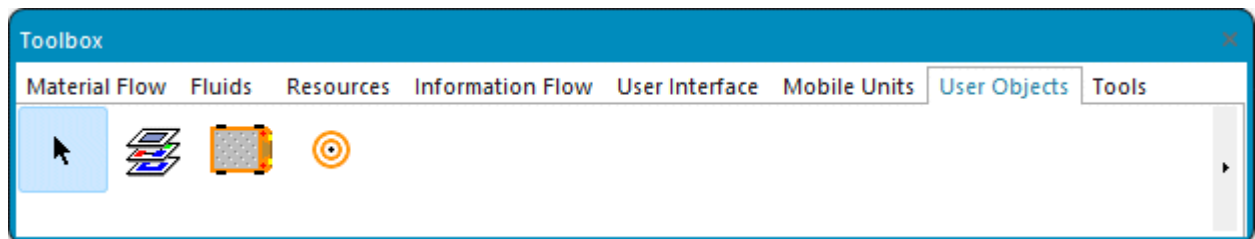
- Insert the **AGVPool** from the tab **Resources** in the *Toolbox*. The AGVs of the automated guided vehicle system are created in the *AGVPool*.



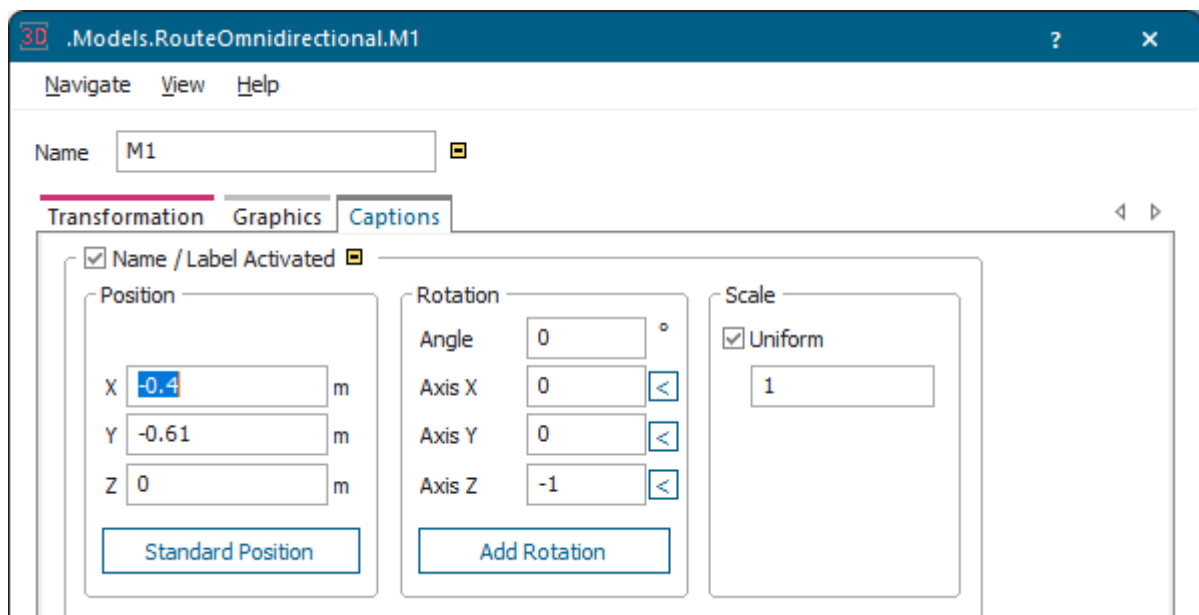
We use our duplicated AGV named MyAGV.



- Insert the *Markers* which form the route pattern into the *Frame*. We inserted four *Markers* and renamed them M1, M2, M3, and M4.



As we want to see these identifiers in the *Frame*, we selected the respective *Marker* and pressed the **spacebar**. Then we selected the settings below for the first *Marker* on the tab **Captions**. We repeated this for the remaining *Markers*.



- Set the route pattern which the AGV covers. You have to program this in a *Method*. We named our *Method* `setRoutePattern` and typed in this source code:

```
var AGV : object := AGVPool.Cont
AGV.setRoute([M1,M2,M3,M4]) // sets the route

waituntil AGV.DestinationWasReached

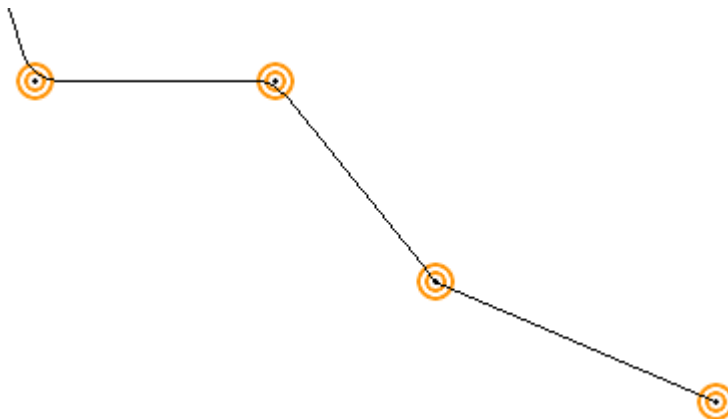
AGV.DestinationWasReached := false
// recognizes when the destination of the route was reached
print "The AGV is ready for the next route."
```

- To make AGV use the set route pattern, we typed in this source code into our *init-method*:

```
// set the route pattern
setRoutePattern()
```

- Run the simulation and watch how the AGV drives along the route and covers the individual *Markers* one after the other from **M1** to **M4**. To better follow the route, we entered the **Real-time** factor 3 in the *EventController*.

The AGV does not drive across the inserted *Markers* but turns in front of the inserted *Markers* according to the angle that *Plant Simulation* computed toward the next *Marker*. For our model it looks like this:



Back to [Model an Automated Guided Vehicle System \(AGVS\)](#)

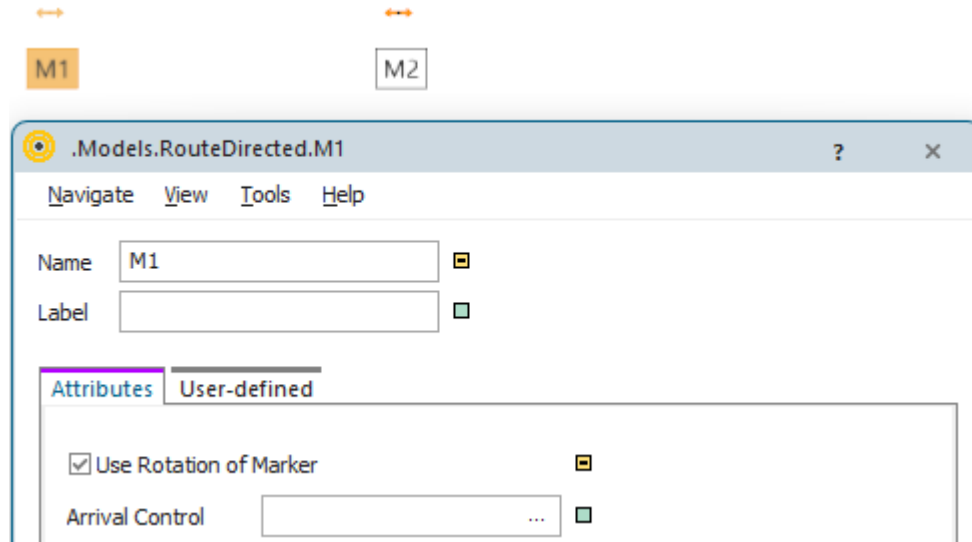
Cover a Route Along Directional Markers

In this sample model the AGV covers the route along *directional Markers* with the default angle.

Proceed as follows to create the model:

- Duplicate the model of the *omnidirectional markers*.
- Convert the *omnidirectional markers* to *directional markers*:

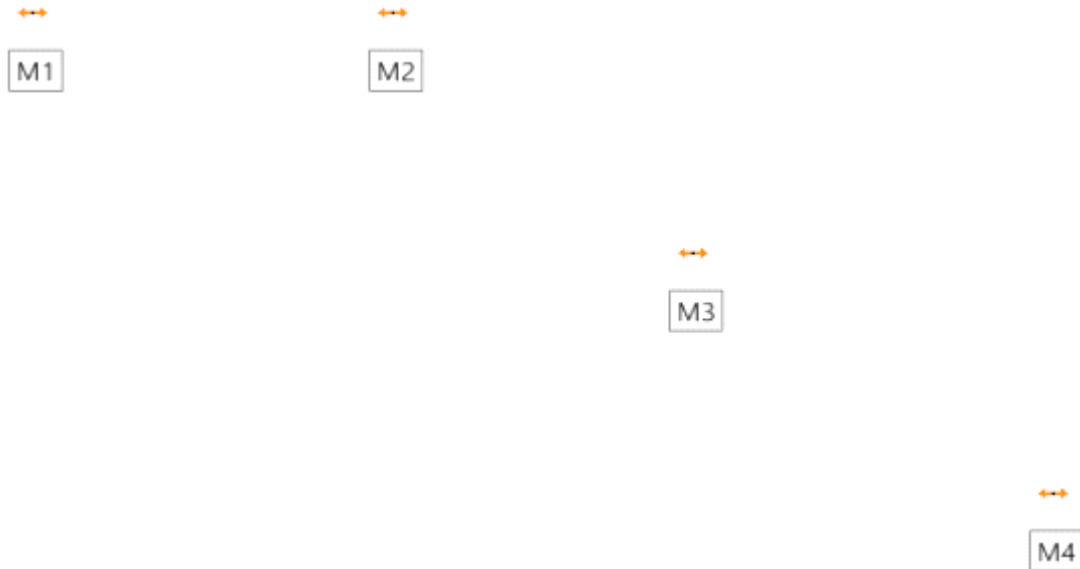
We double-clicked the Marker **M1**, selected **Use Rotation of Marker** on the tab **Attributes** and clicked **OK**. If you do not specify an angle, the *Marker* uses the default angle of 0°.



Plant Simulation now shows the *Marker* with this icon .

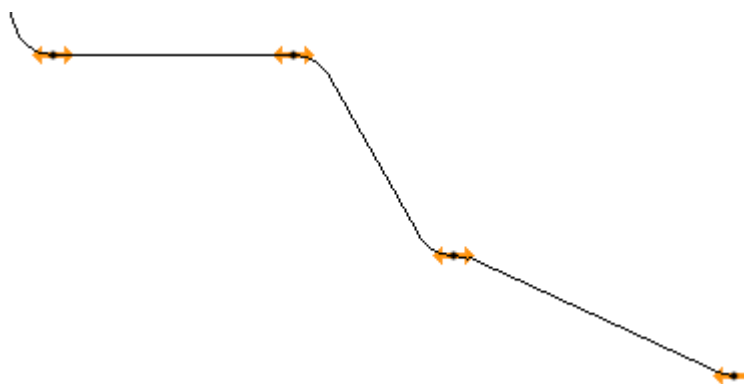
Repeat this for the remaining *Markers*.

In our example we use the default angle of 0 degrees for our *Markers*.



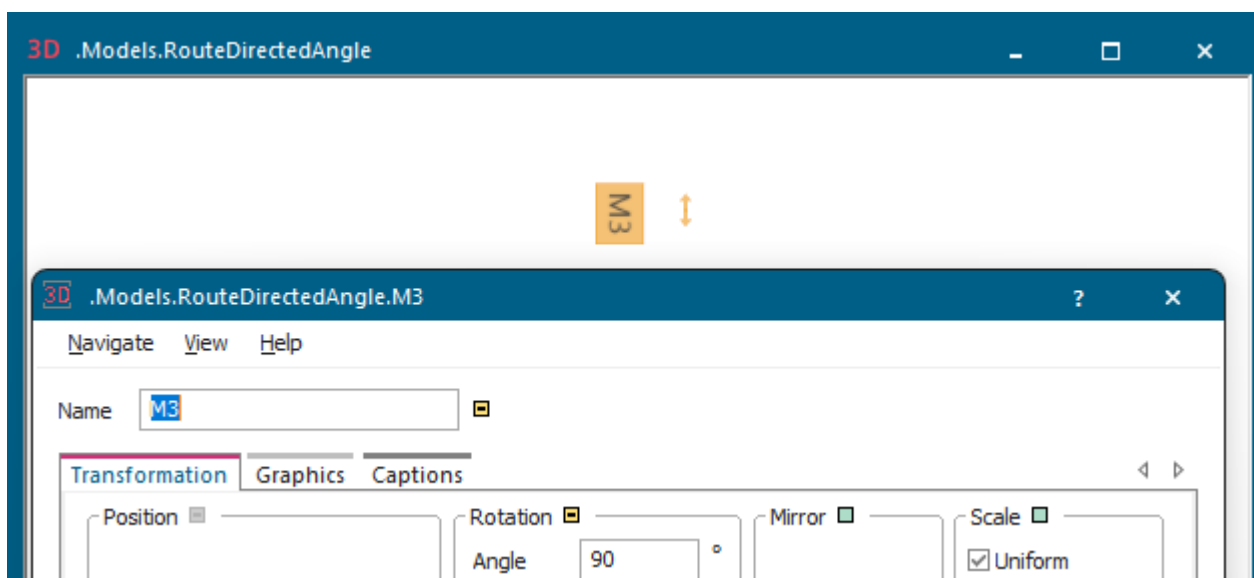
- Run the simulation and watch how the AGV drives along the route and covers the individual *Markers*. To better follow the route, we entered the **Real-time** factor 3 in the *EventController*.

The AGV drives across the *Markers* from **M1** to **M4** and turns on the *Marker* with the specified angle in the direction of the next *Marker*.



We can also use another rotation angle than the default angle of 0°:

- Click the *Marker* and press the **spacebar** to enter the **rotation angle** in the dialog **Edit 3D Properties**.
- Type the rotation angle into the text box **angle** in the group box **Rotation**. We typed in an angle of 90 degrees. Click **Apply** to show the changed settings in the *Frame* window.



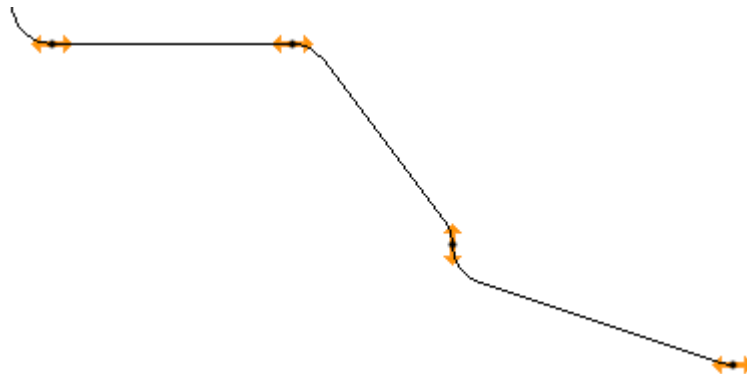
- Click **OK** when the angle meets your expectations and you want to apply it.

Plant Simulation now shows the *Marker* with this icon .



- Run the simulation and watch how the AGV drives along the route and covers the individual *Markers*. To better follow the route, we entered the **Real-time** factor 3 in the *EventController*.

The AGV covers the *Markers* from **M1** to **M4**. It drives toward the *Marker M3* at an angle of 90°, which we defined, crosses it, and leaves it at an angle of 90°.



Back to [Model an Automated Guided Vehicle System \(AGVS\)](#)

Cover a Route According to a Segments Table

In this example the AGV covers the route according to the route segments which we set in a table.

To do so, we use the method [setRouteSegments](#).

Proceed as follows to create the model:

- Insert the [AGVPool](#) from the tab **Resources** in the *Toolbox*. The AGVs of the automated guided vehicle system are created in the *AGVPool*.
- Insert a *Method* and program the source code so that it executes the following instructions:
 - We want the AGV created in the *AGVPool* to cover the segments of the route.

- We create a local *Variable* of data type *table* with four columns in which we set the segments of the route.
- We append a row for each segment to the table.
- We then tell the AGV to cover the segments of the route which we specified in the table named *Route*.
- We finally suspend method execution.

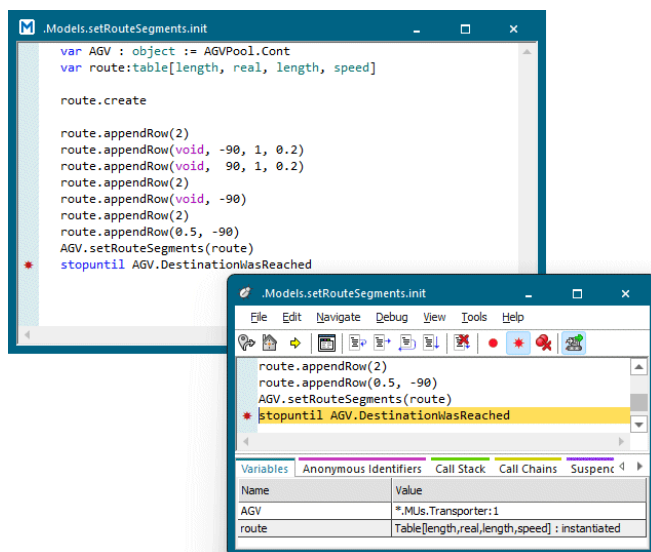
The respective source code looks like this:

```
var AGV : object := AGVPool.Cont
var route:table[length, real, length, speed]

route.create

route.appendRow(2)
route.appendRow(void, -90, 1, 0.2)
route.appendRow(void, 90, 1, 0.2)
route.appendRow(2)
route.appendRow(void, -90)
route.appendRow(2)
route.appendRow(0.5, -90)
agv.setRouteSegments(route)
stopuntil agv.DestinationWasReached
```

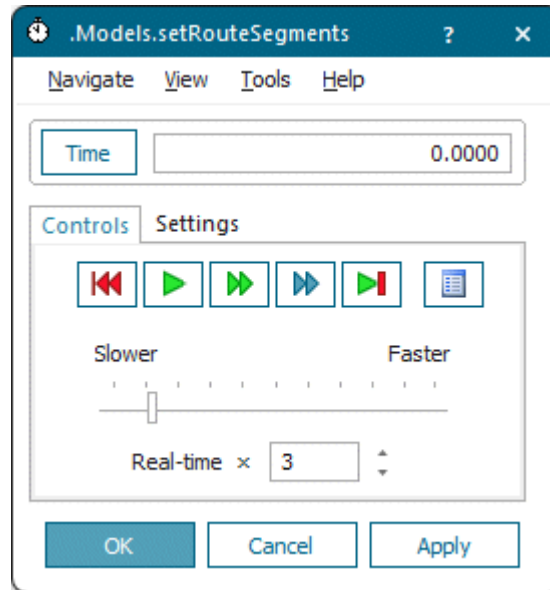
- Rename the *Method* to *init* so that the simulation will be run immediately after resetting the simulation model.
- Run the simulation and follow the movements of the AGV in the simulation model.
- To show the route segments in the created table, insert a break point in the last row of the source code and run the *Method*. Then double-click the cell **Value** in the column **Name** on the tab **Variables** in the *Method-Debugger*.



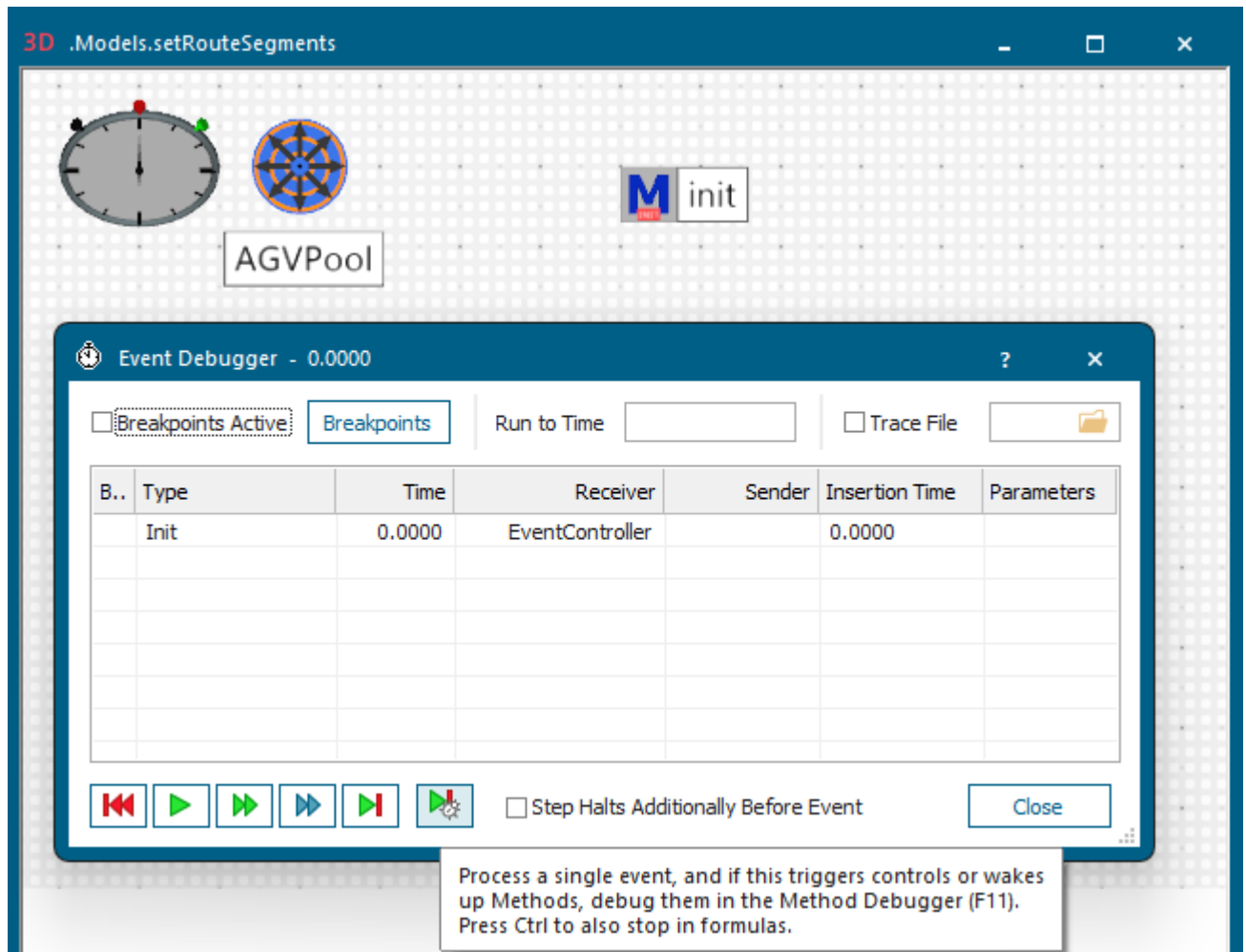
The image shows a table window titled `.Models.setRouteSegments.init.route`. It displays a table with 4 columns: `length[m]`, `real`, `length[m]`, and `speed[m/s]`. The table has 8 rows, with the first row being a header and the subsequent rows containing data. The data rows are numbered 1 through 8 in the first column.

	length[m]	real	length[m]	speed[m/s]
1	2			
2		-90.00	1	0.2
3		90.00	1	0.2
4	2			
5		-90.00		
6	2			
7	0.5	-90.00		
8				

- The first column contains the **Length** of the straight segment.
- The second column contains the **Angle** with which the curve changes its direction.
- The third column contains the **Radius** of the curved segment.
- The fourth column contains the **Speed** with which the AGV drives on this segment.
- To trace how the AGV drives on the individual segments, click .



Then click  in the **dialog box of the EventDebugger** to cover the segments one after the other.



Back to [System](#)

Load Parts Onto an AGV and Rotate Them

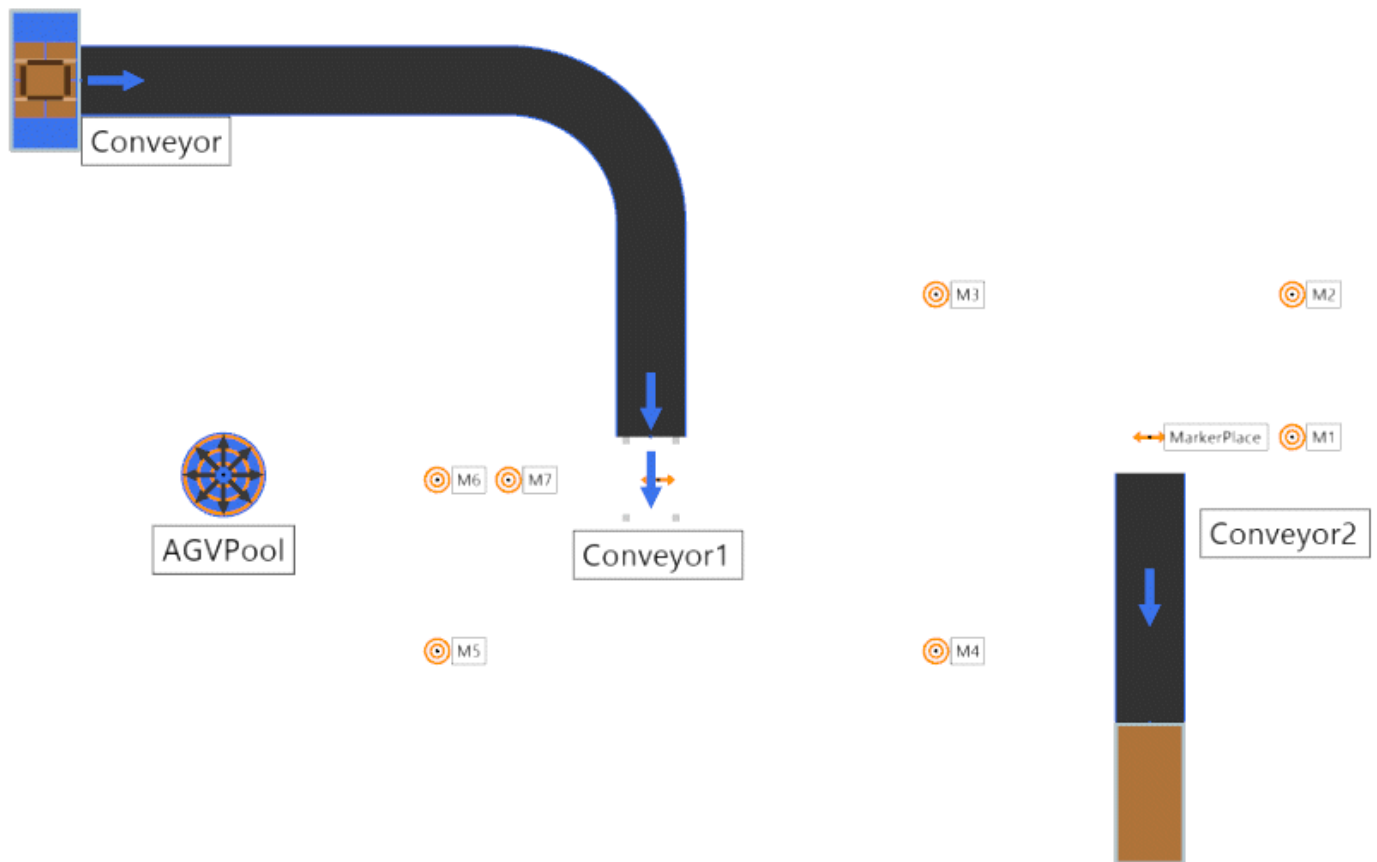
Load Parts Onto an AGV and Rotate Them

In this example we demonstrate how the AGV drives underneath a *Conveyor*, loads a pallet, and rotates it while driving toward its destination. After the AGV has placed the part, it drives back to the point of departure along the *Markers* and picks up the next pallet.

In our example we will:

- Insert the Material Flow Objects
- Create the Conveyor on Which to Place the Pallet
- Insert the Transport Components

The finished simulation model looks like this:



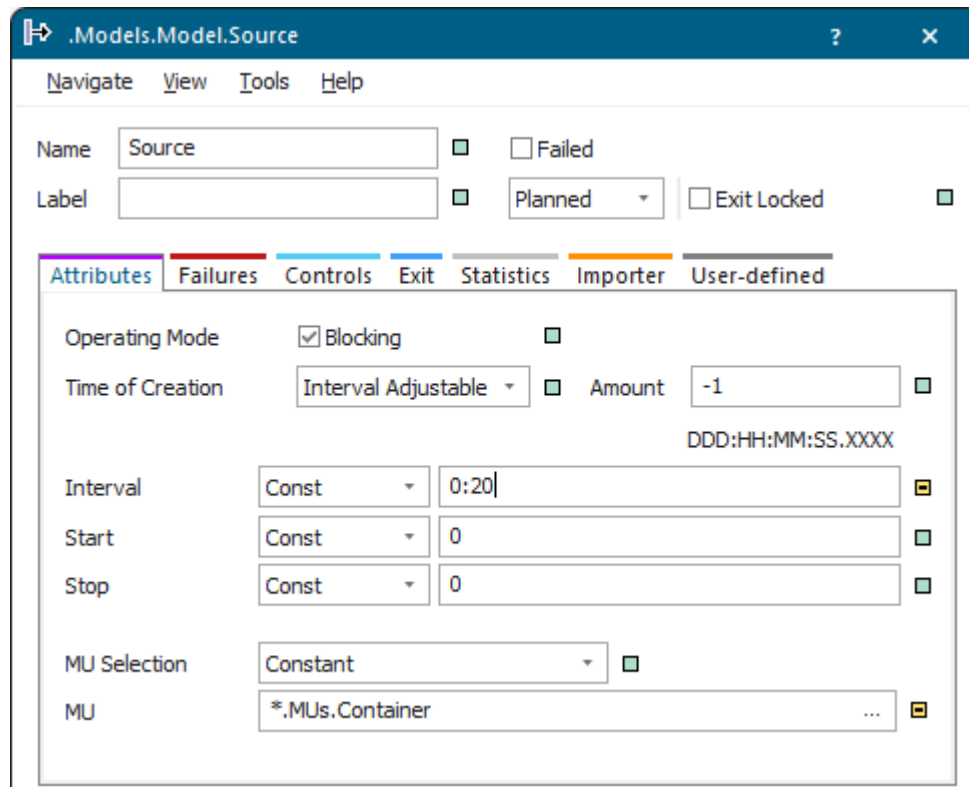
Back to [Model an Automated Guided Vehicle System \(AGVS\)](#)

Insert the Material Flow Objects

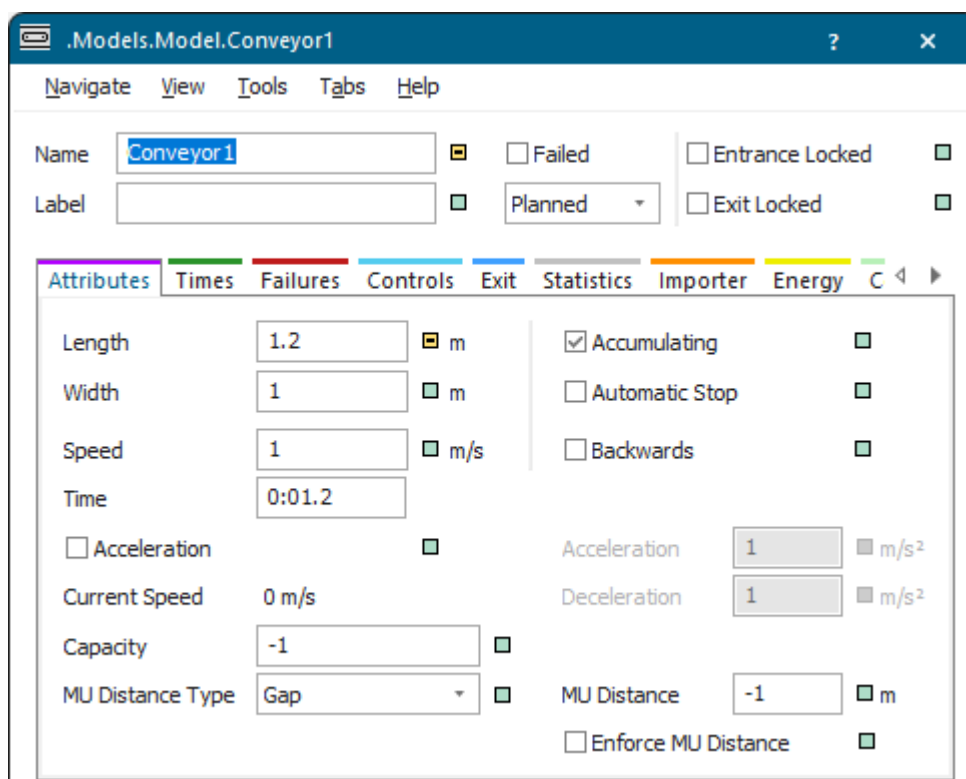
Insert the material flow objects which produce and transport the pallets.

Proceed as follows:

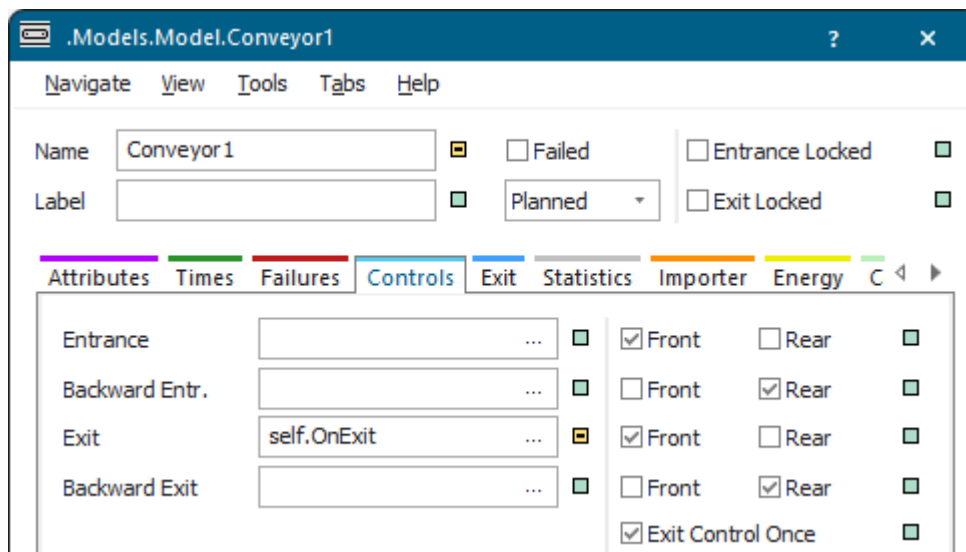
- The *Source* produces parts of type *Container*, our pallet, every 20 seconds.



- *Conveyor1* transports the pallets to the pick-up place. We use *Conveyor1* with its default settings.
- The pick-up place is *Conveyor1* underneath which the AGV drives and loads the pallet.
 - For *Conveyor1* we selected these simulation settings.



- We also created an **Exit Control** as a *user-defined attribute*. In it we set the route of the AGV and make it rotate the pallet.



We entered this source code:

```
waituntil AGVPool.NumIdleAGVs > 0
var agv := AGVPool.getIdleAGV

agv.setDim(1,1)
```

```
agv.setRoute([MarkerPick])

waituntil agv.DestinationWasReached

@.move(agv)
@.ConveyingDirection := 1

agv.setRoute([MarkerPlace])

wait 1
agv._3D.getObject(1).moveTo(90)

waituntil agv.DestinationWasReached
@.move(Conveyor2)

agv.setRoute([M1,M2,M3,M4,M5,M6,M7])
agv._3D.getObject(1).moveTo(00)

waituntil agv.DestinationWasReached
agv.IsIdle := true
```

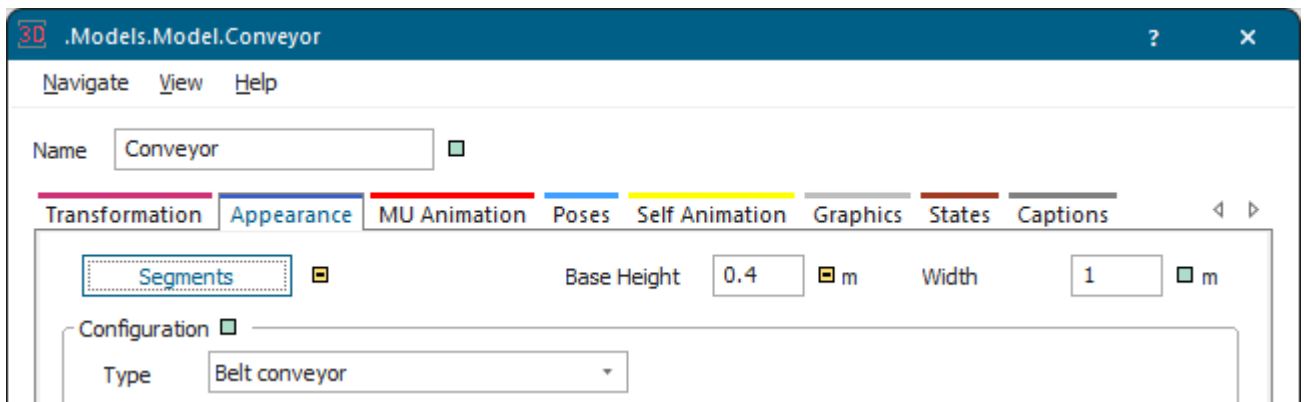
Back to [Load Parts Onto an AGV and Rotate Them](#)

Create the Conveyor on Which to Place the Pallet

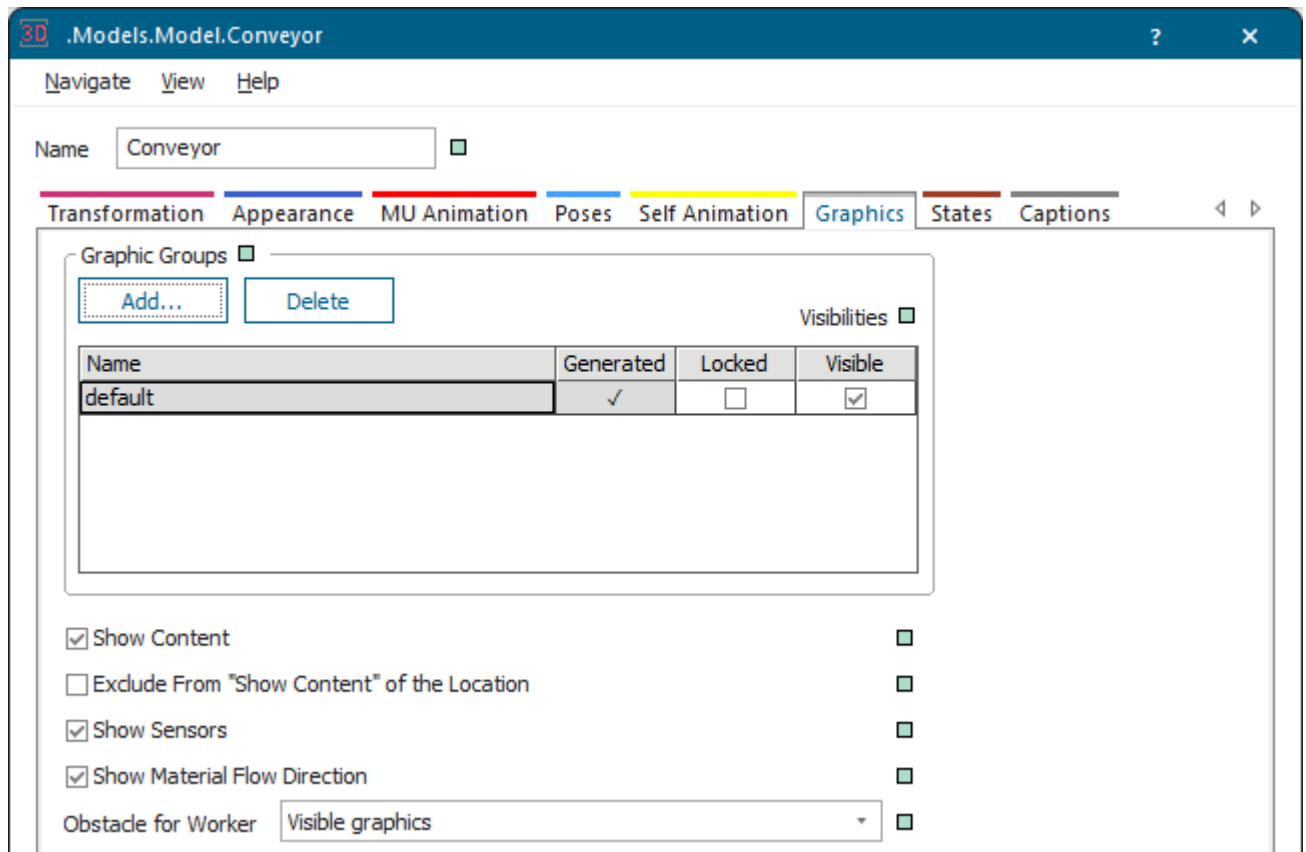
In this step we create the conveyor under which the AGV drives to pick the pallet up.

We proceeded as follows:

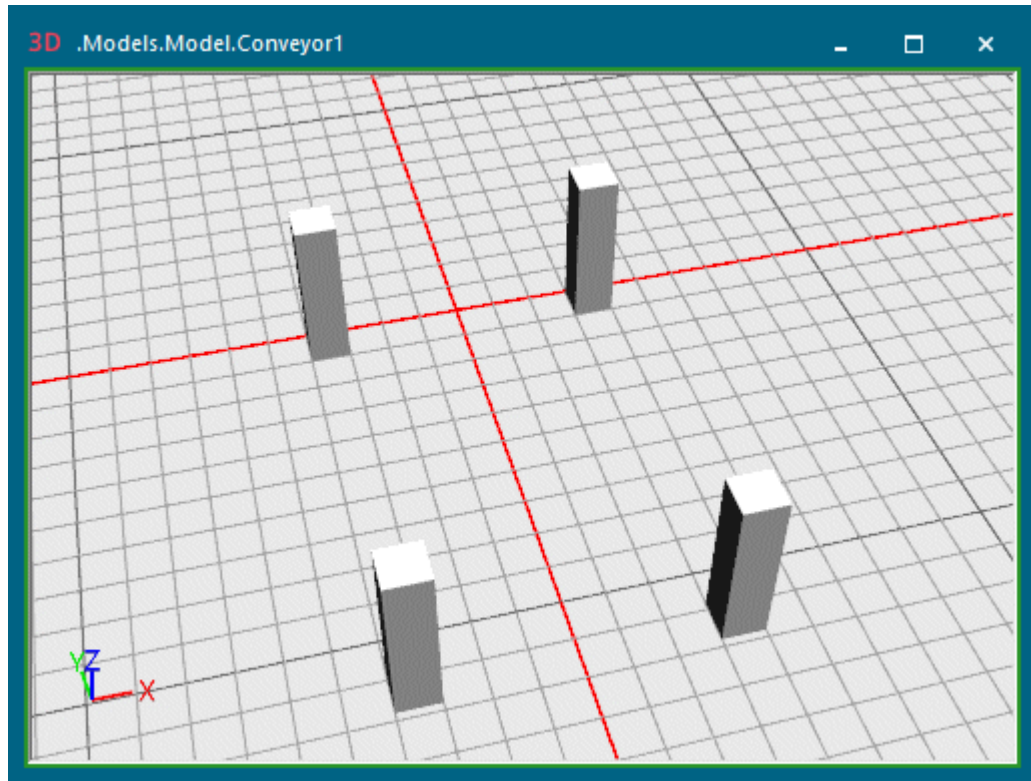
- We selected these 3D settings.
- We typed in a **Base Height** of 0.4 meters for our *Conveyor*.



As the AGV is to drive underneath *Conveyor1* to pick up the pallet, we added a new graphic group to it and named it *graphic*.



We then opened *Conveyor1* in a new window and created four posts with a height of 0.4 meters, which matches the **Base Height** of our *Conveyor*, and a width and a depth of 0.1 meters each.



- Insert another *Conveyor* onto which the AGV places the pallets. We use *Conveyor2* with its default settings.
- Insert a *Drain* which removes the pallets from the plant. We use the *Drain* with its default settings.

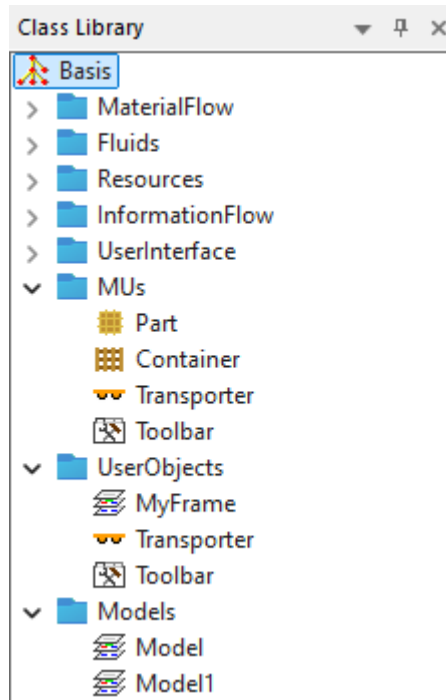
Back to [Load Parts Onto an AGV and Rotate Them](#)

Insert the Transport Components

In this step we insert the transport components.

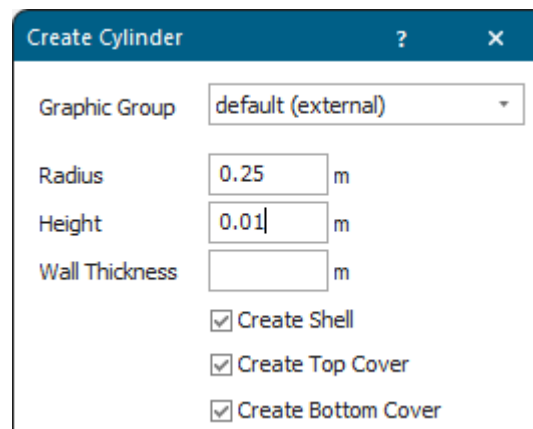
We proceeded as follows.

- As we want to change the properties of our AGV, we duplicate the *Transporter* in the folder *MUs*. This prevents that we change the built-in class of the *Transporter*.

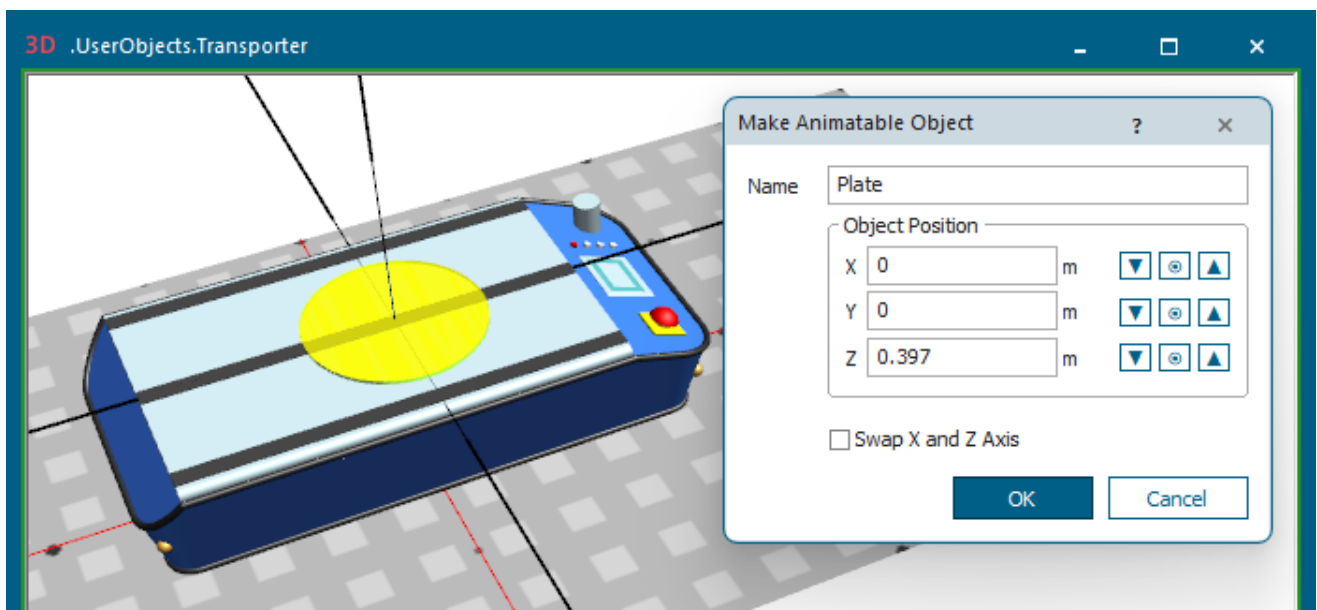


Now we create a plate on the loading space of the AGV, which rotates the loaded pallet:

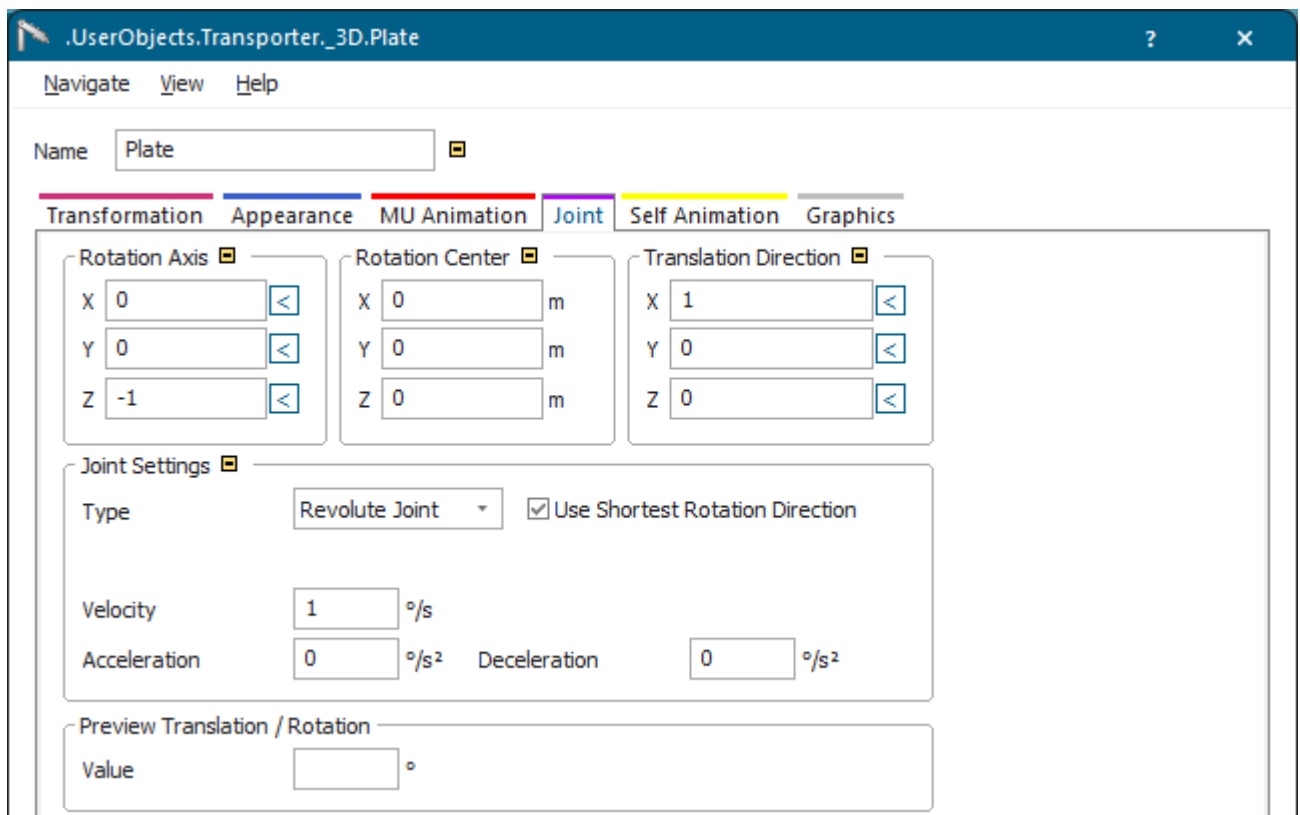
- Click the *Transporter* in the folder **UserObjects** with the right mouse button and select **Open in 3D**.
- Create a **Cylinder**. We selected these settings:



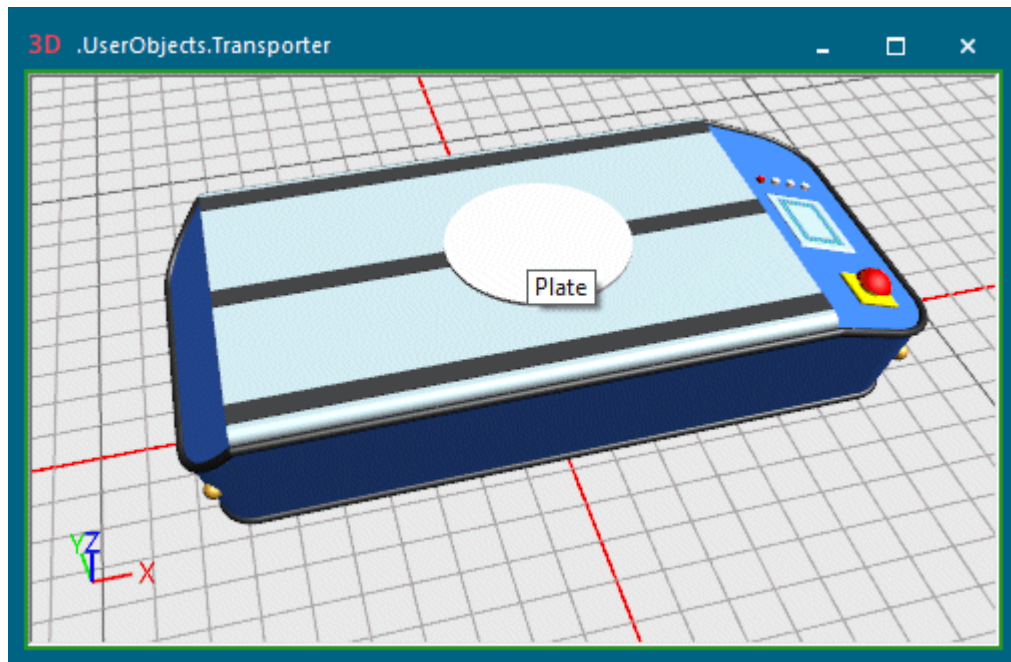
- Click the cylinder with the right mouse button and select **Make Animatable Object**. Type in **Plate** as the name. Click **OK**.



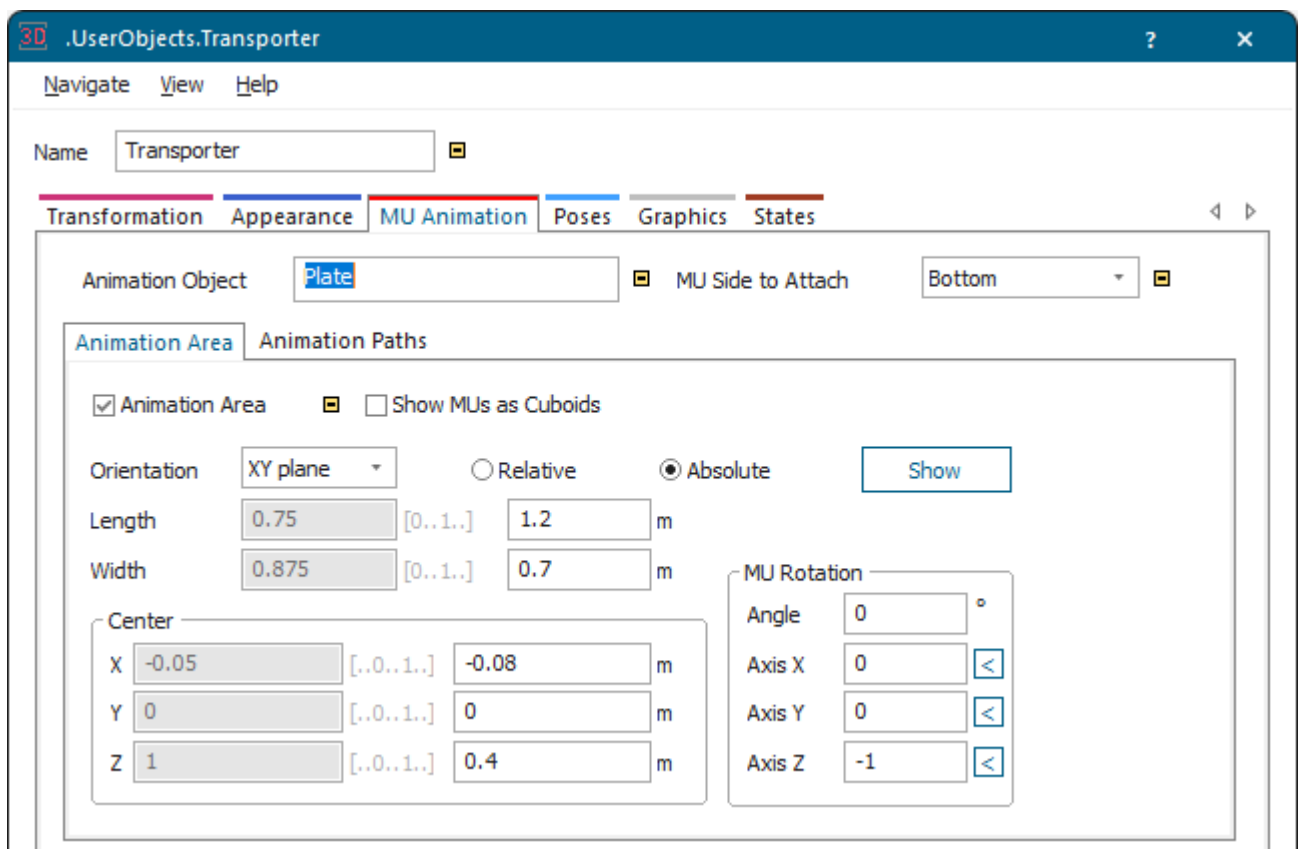
- This automatically opens this dialog with the **tab Joint**. Select **Revolute Joint** and type in 20 as the **Velocity**.



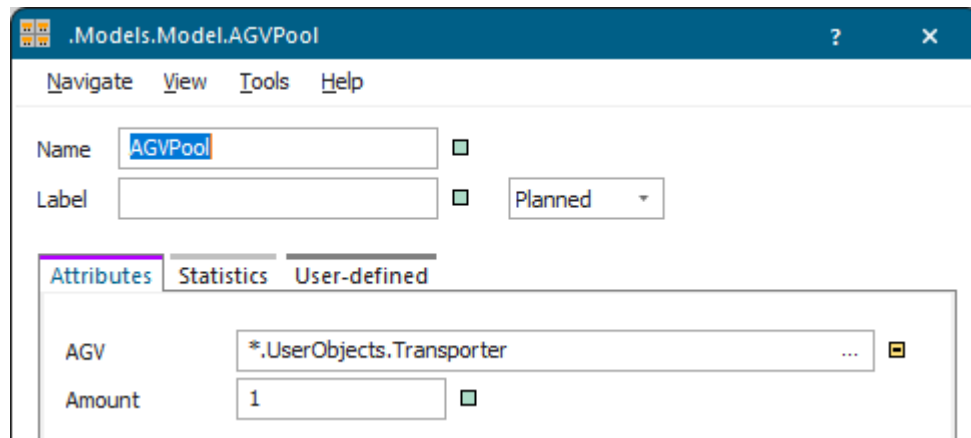
- Our *Transporter* then looks like this:



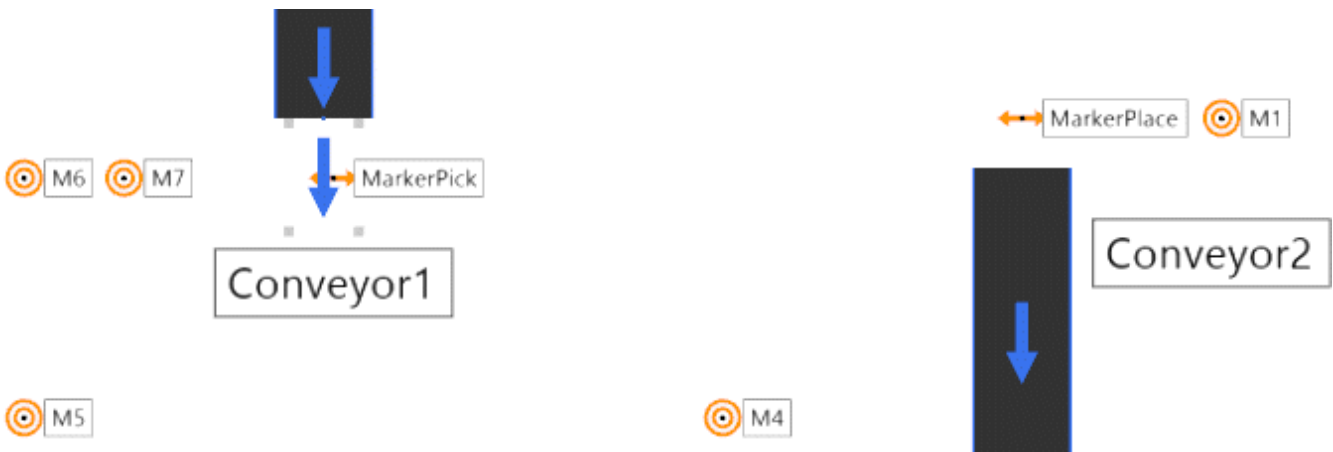
- We then typed in Plate as the **Animation Object** of the AGV.



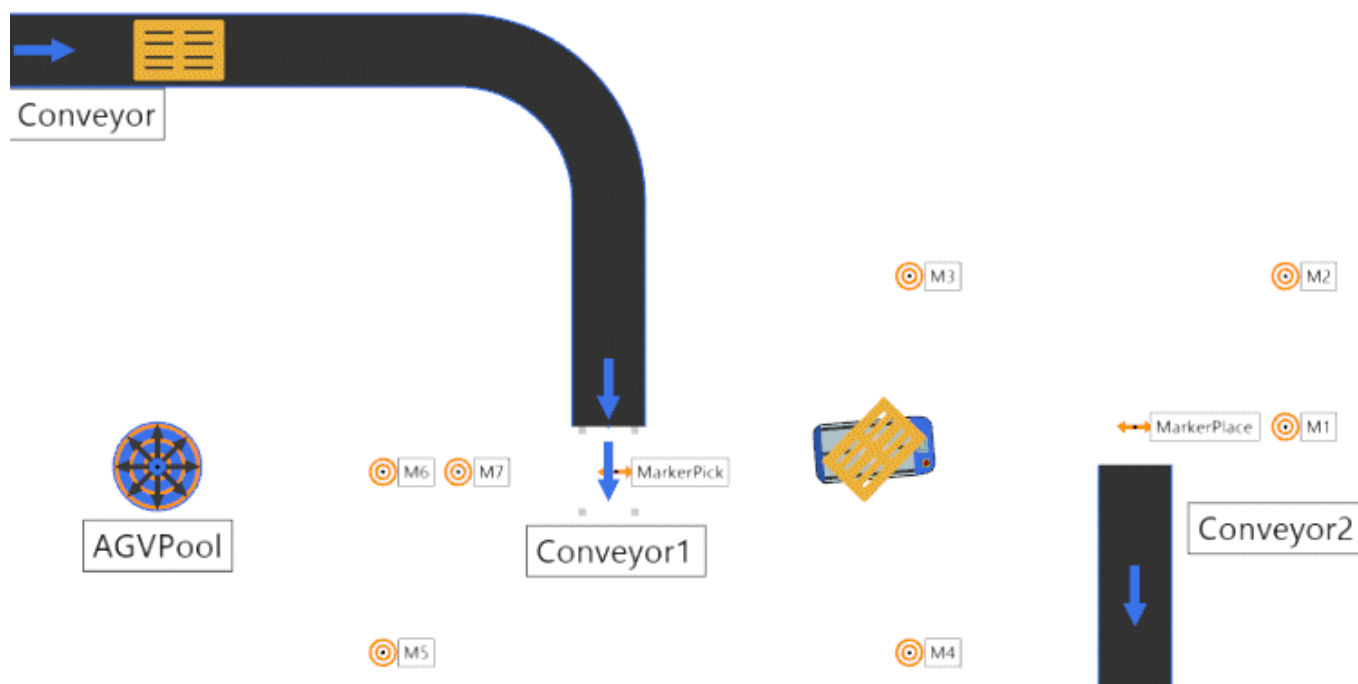
- Insert the *AGVPool* which creates the AGVs of the automated guided transport system. By default this is the *Transporter*.



- Insert the *Markers* along which the AGV drives.
The *Markers* M1 to M7 are omnidirectional *Markers*.
The *Markers* MarkerPick and MarkerPlace are directional *Markers*.



Run the simulation and watch how the AGV drives from the AGVPool underneath *Conveyor1* to *MarkerPick* and picks up the pallet there. On its way to *MarkerPlace* the pallet rotates on the AGV and is placed rotated on *Conveyor2* at the *MarkerPlace*. Then the AGV drives back to the pick-up place and repeats the process.



Back to Load Parts Onto an AGV and Rotate Them

Fine-position an AGV

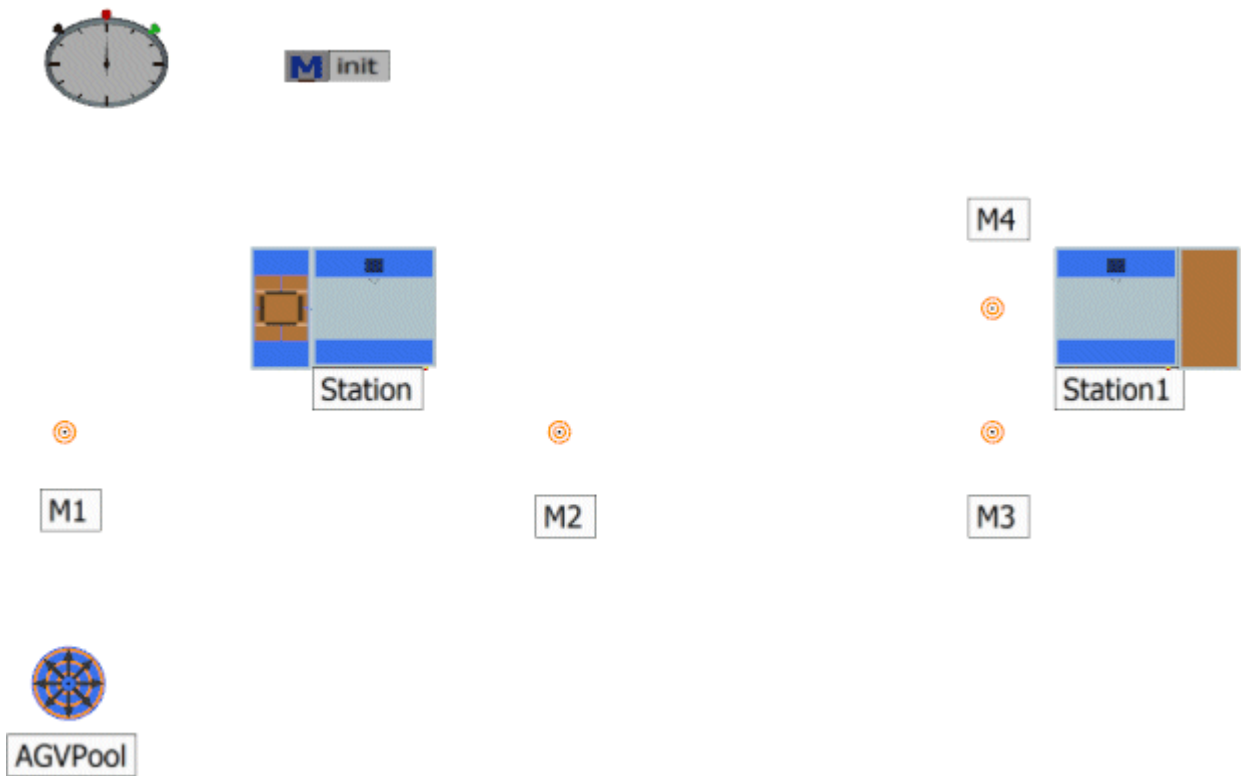
If the AGV is driving freely within the area of your model, you will sometimes have to fine-position it.

You can achieve this with the methods **rotate** and **drive** of the *Transporter*.

In our example the AGV drives from the *AGVPool* via the *Marker* named **M1** to the *Marker* named **M2**. There it rotates by 90° to the left and drives 2 meters upward. Afterward it moves sideways in a 90° angle 1.6 meters to the left, and loads the part from the *Station*. Then it moves sideways in a 90° angle 1.6 meters to the right and drives backward to the *Marker* **M2**.

On the *Marker* **M2** the AGV rotates 90° to the right and drives via the *Marker* named **M3** to the *Marker* named **M4**. There it unloads the part onto the *Station1* and drives back to the *AGVPool* via the *Markers* **M2** and **M1**.

The finished simulation model looks like this:



We use the *material flow objects* and the *resource objects* with their default settings. We fine-position the AGV in our *init method* by using the methods *setRoute*, *rotate*, and *drive* of the *Transporter*.

The source code of our *init method* looks like this. The comments describe what the instructions accomplish.

```
var AGV := AGVPool.Cont

AGV.setRoute([M1, M2])           // AGV drives to the marker M2
stopuntil AGV.DestinationWasReached

AGV.rotate(-90)                  // AGV rotates 90° to the left
stopuntil AGV.DestinationWasReached

AGV.drive(2)                     // AGV drives 2m upward
stopuntil AGV.DestinationWasReached

AGV.drive(1.6, -90)              // AGV moves sideways in a
stopuntil AGV.DestinationWasReached // 90° angle 1.6 m to the left

var MU := Station.Cont
waituntil MU.Finished           // MU waits until it's processed
wait 2                          // MU waits 2 seconds
MU.move(AGV)                    // MU moves to the AGV
wait 2                          // AGV waits 2 seconds

AGV.drive(1.6, 90)              // AGV moves sideways in a
```

```

stopuntil AGV.DestinationWasReached // 90° angle 1.6 m to the right
AGV.drive(-2)                        // AGV drives 2m downward
stopuntil AGV.DestinationWasReached

AGV.rotate(90)                       // AGV rotates 90° to the right
stopuntil AGV.DestinationWasReached

AGV.setRoute([M3, M4])               // AGV drives across M3 to M4
stopuntil AGV.DestinationWasReached

waituntil Station1.empty             // AGV waits until Station1 is
wait 2                              // empty, waits 2 seconds
MU.move(Station1)                   // MU moves to Station1
wait 2                              // AGV waits 2 seconds

AGV.setRoute([M2, M1, AGVPool])      // AGV drives via the markers M2
// and M1 back to the AGVPool

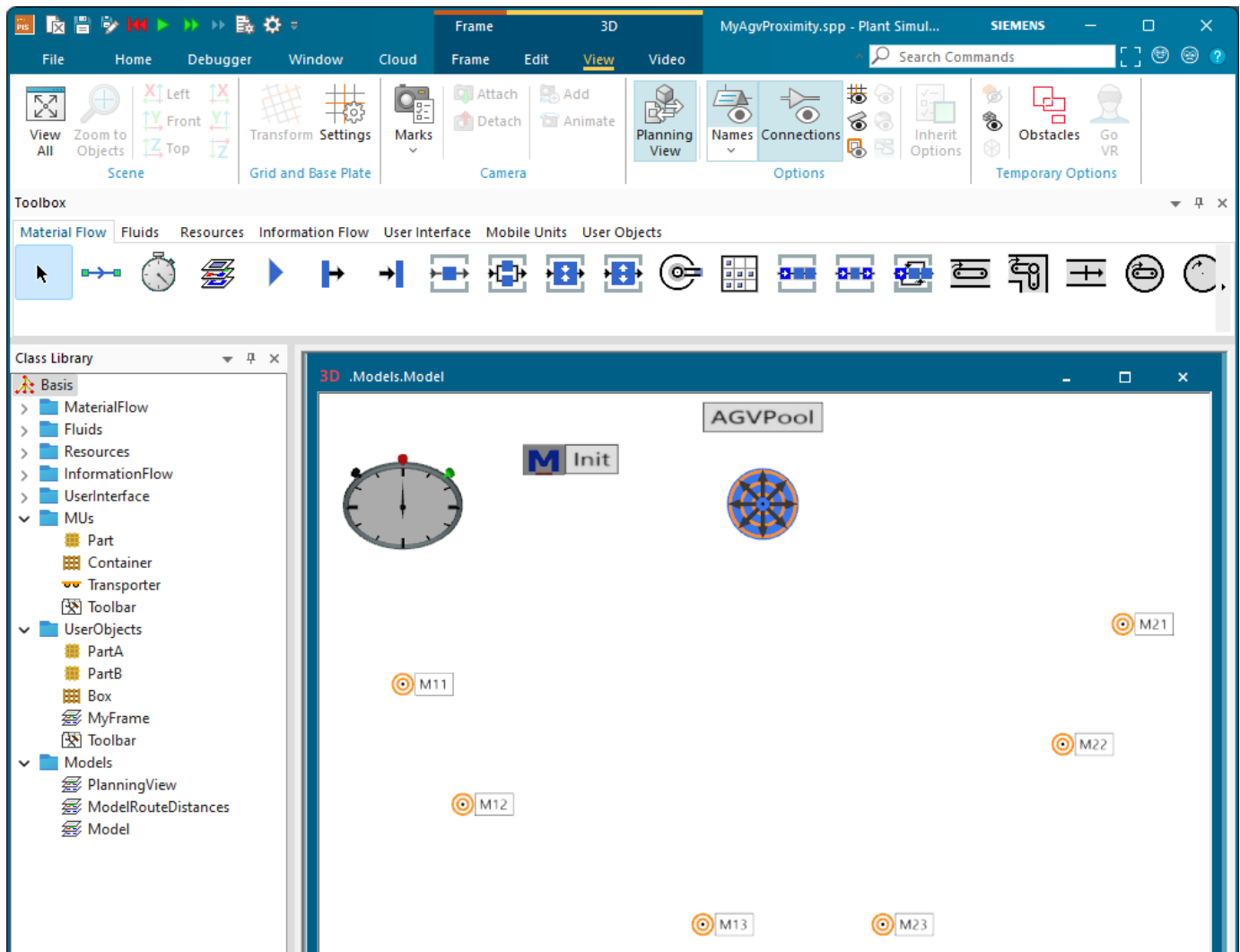
```

Back to [Model Transport Systems](#)

Prevent AGVs from Colliding

In this model we demonstrate how to use the settings of the **Distance Control** of the *Transporter* to prevent two AGVs from colliding.

The finished simulation model looks like this.

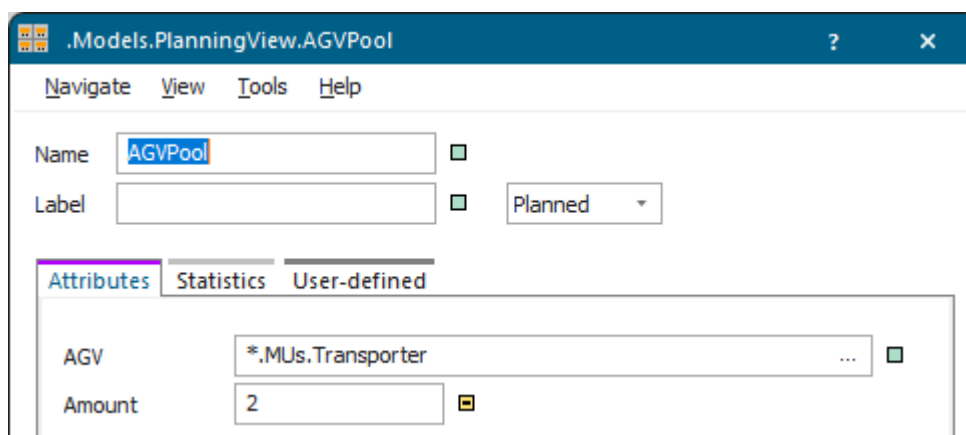


Proceed as follows to create the model:

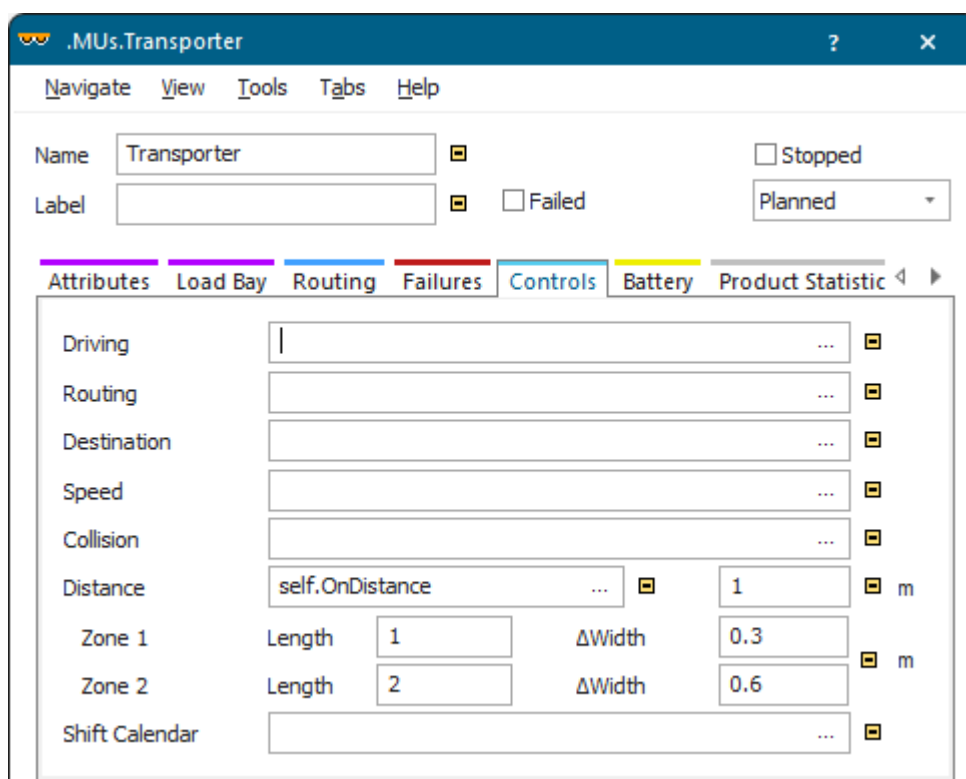
- Insert an *AGVPool* that creates the AGVs.
- Insert four *Markers* each for each of the two *Transporters*. *Transporter1* covers the markers *M11*, *M12*, *M13*, and *M14*. *Transporter2* covers the markers *M21*, *M22*, *M23*, and *M24*.
- Insert an *init* control that finds an idle AGV in the *AGVPool* that can do the job. In addition we set the routes of the AGVs.

Then configure the objects.

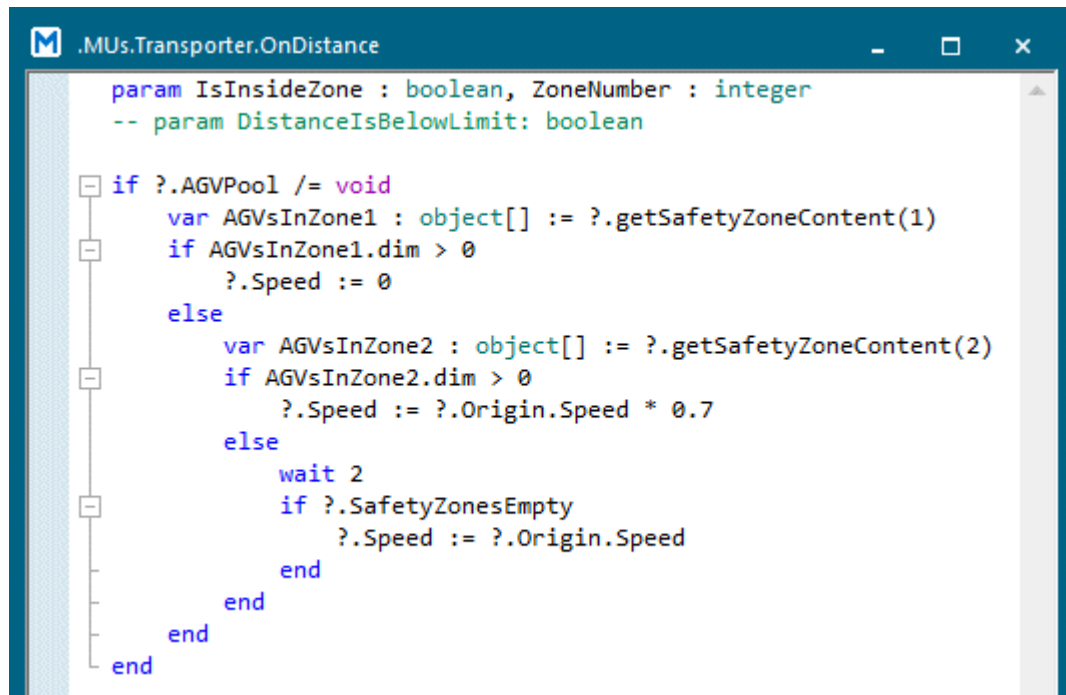
- We use the default setting of the *AGVPool*, meaning that the **AGV** is the *Transporter* in the folder **MUs**.



- We use the default settings for the **Safety Zones** of the *Transporter*.



- We use the default settings of the **Distance Control** that we created.



- We programmed the *init control* that finds an idle AGV in the *AGVPool* and sets the routes which the AGVs cover.

```

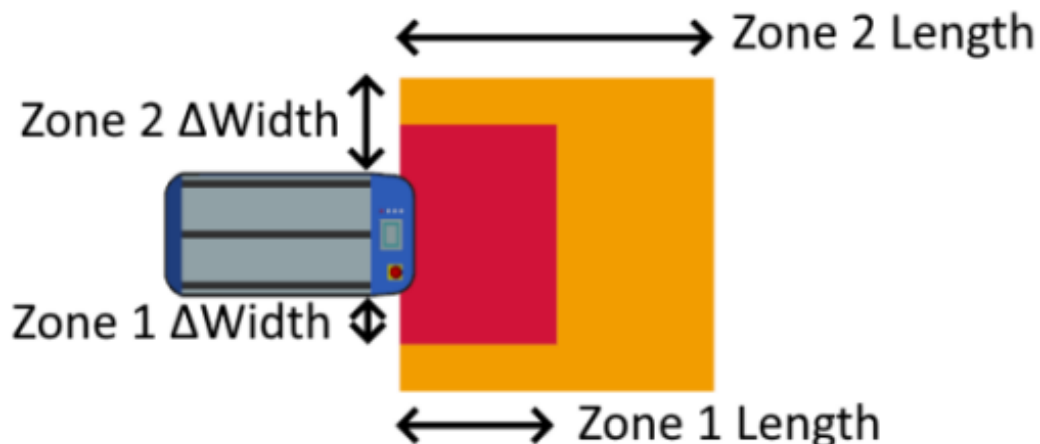
var AGV1 := AGVPool.getIdleAGV
AGV1.setRoute([M11, M12, M13, M14])

wait 2

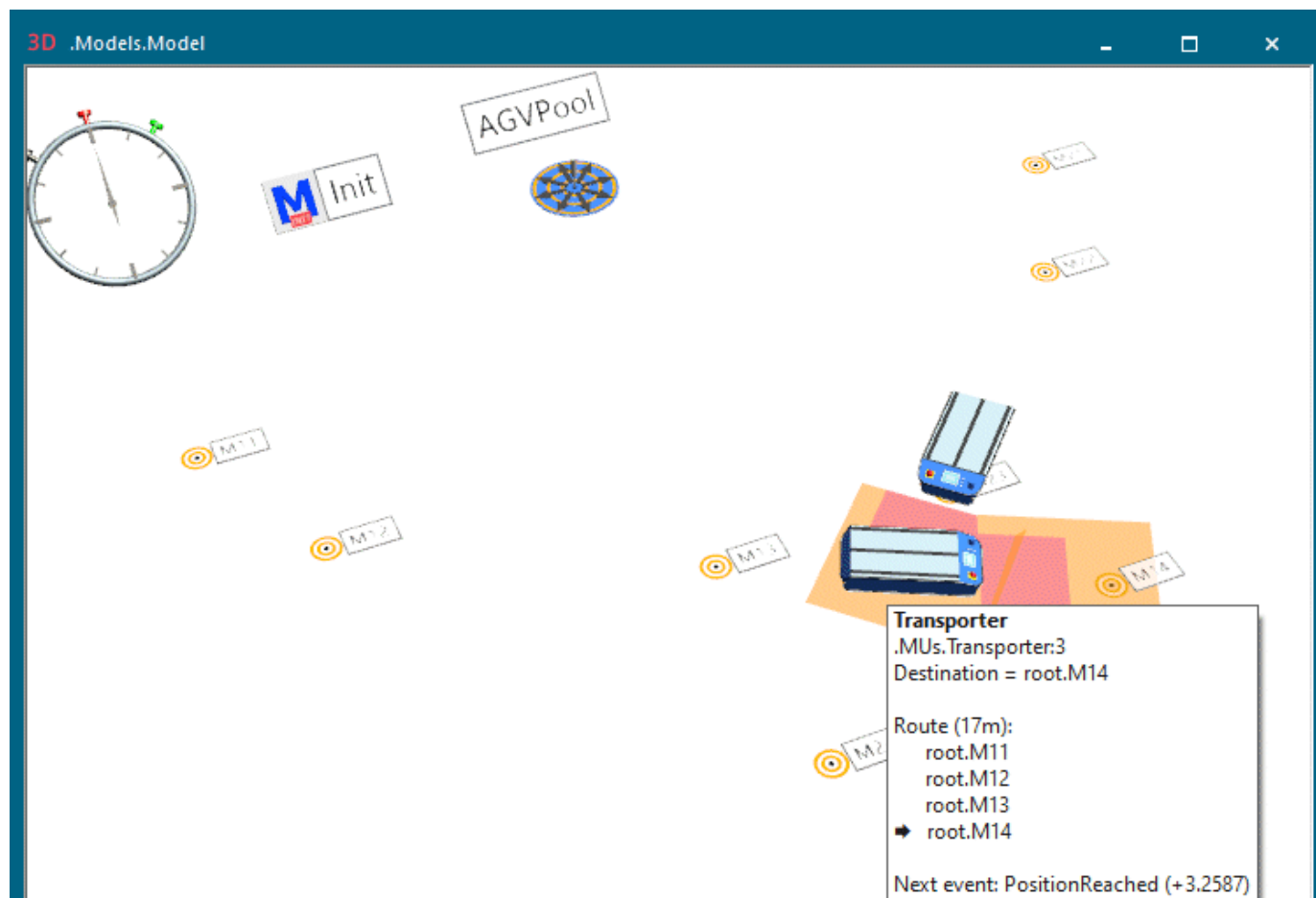
var AGV2 := AGVPool.getIdleAGV
AGV2.setRoute([M21, M22, M23, M24])

```

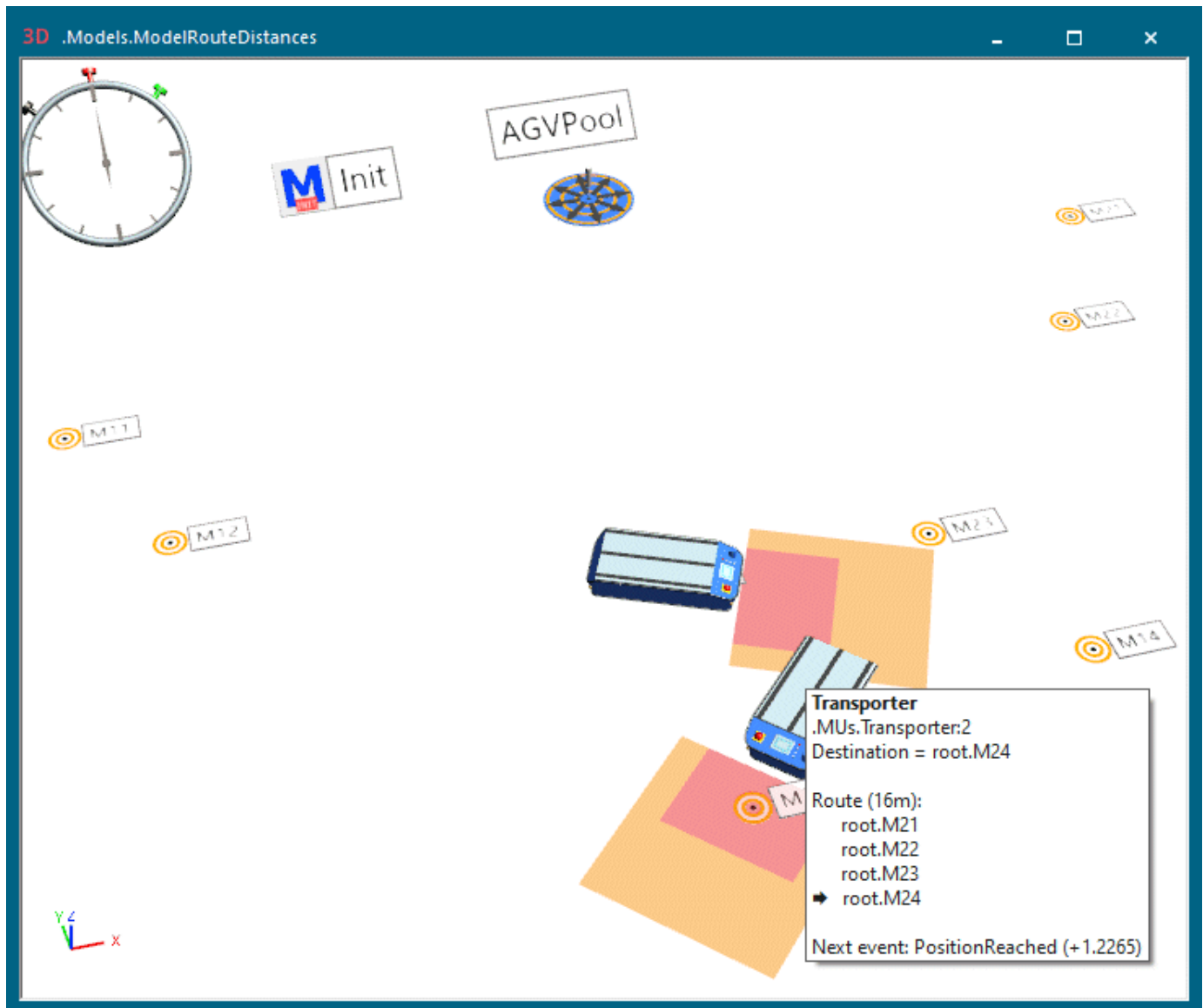
When you run the simulation, you can watch how *Transporter2* slows down once *Transporter1* enters its **Safety Zone 2** and then stops when it enters **Safety Zone 1**. *Transporter2* continues on its **route** once *Transporter1* passed by safely.



To show the defined **Safety Zones** of the *Transporter*, select **Edit 3D Properties** > **Tab Appearance** > **Show Safety Zones**.



When you move the markers *M11* and *M12* to the left in the **X-Direction**, meaning that *Transporter2* now arrives first at the point where the routes cross, *Transporter1* slows down and stops to let *Transporter2* pass. This shows that the **safety zone settings** and the **Distance Control** reliably prevent the AGVs from colliding.



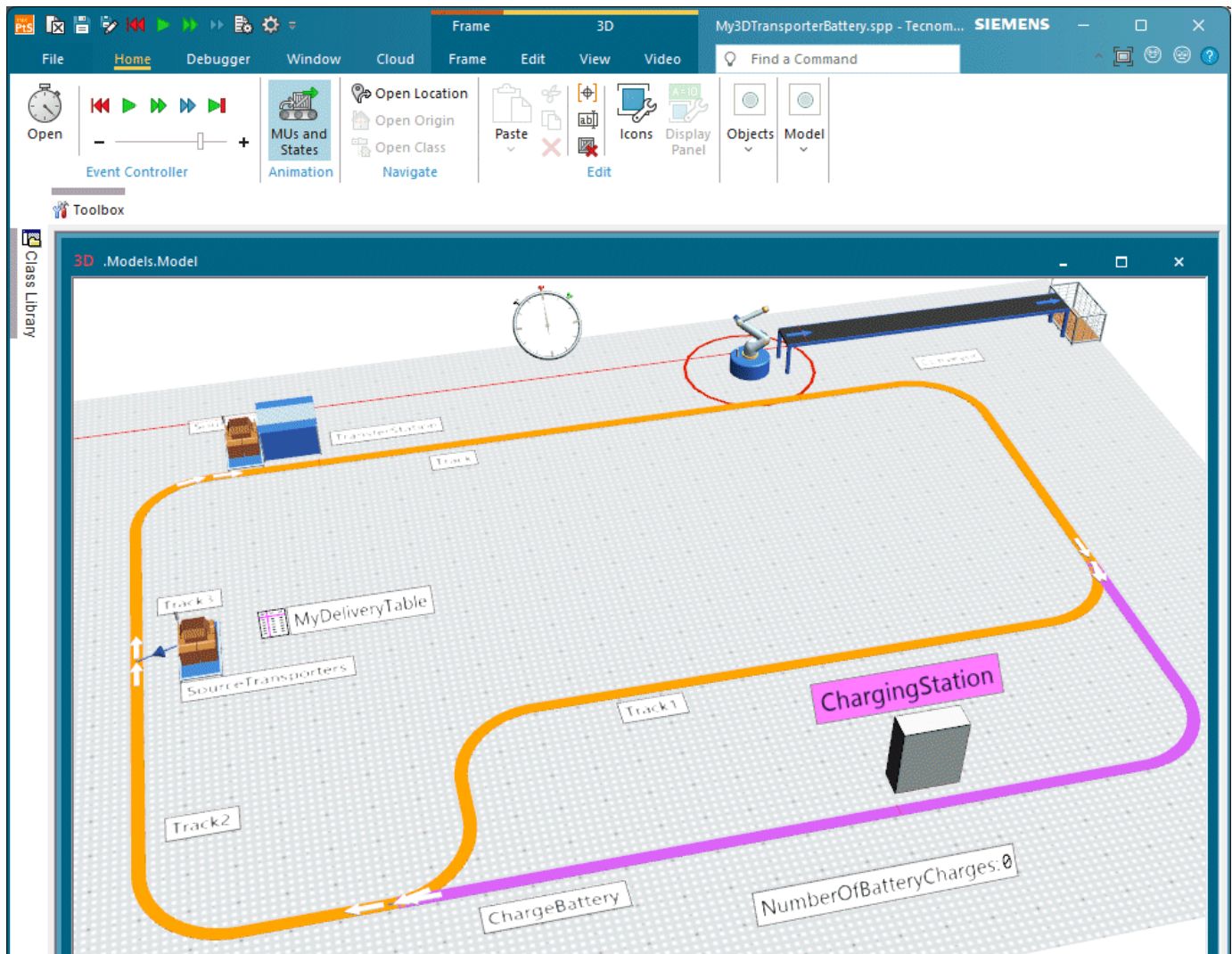
Back to [Model Transport Systems](#)

Modeling a Battery-powered Transporter, Basic

Model a Battery-powered Transporter, Basic

In this sample model we demonstrate how easy it is to quickly model a *battery-powered Transporter*. The *Transporters* drive on a *Track* moving parts from one point to another. The **Exit Control** of the *Track* detects the charge of the battery; if enough power is left, it continues moving parts, if not, it drives onto the *Track* to the charging station, which charges the battery.

The finished simulation model looks like this:



First we duplicated the *Transporter* in the *Class Library* and renamed it to *TransporterBattery*.

- We selected these settings on the tab **Battery**:

.UserObjects.TransporterBattery

Attributes View Tools Tabs Help

Name: TransporterBattery ☐ Stopped ☐ Failed

Attributes Load Bay Routing Failures Controls **Battery** Product Statistics

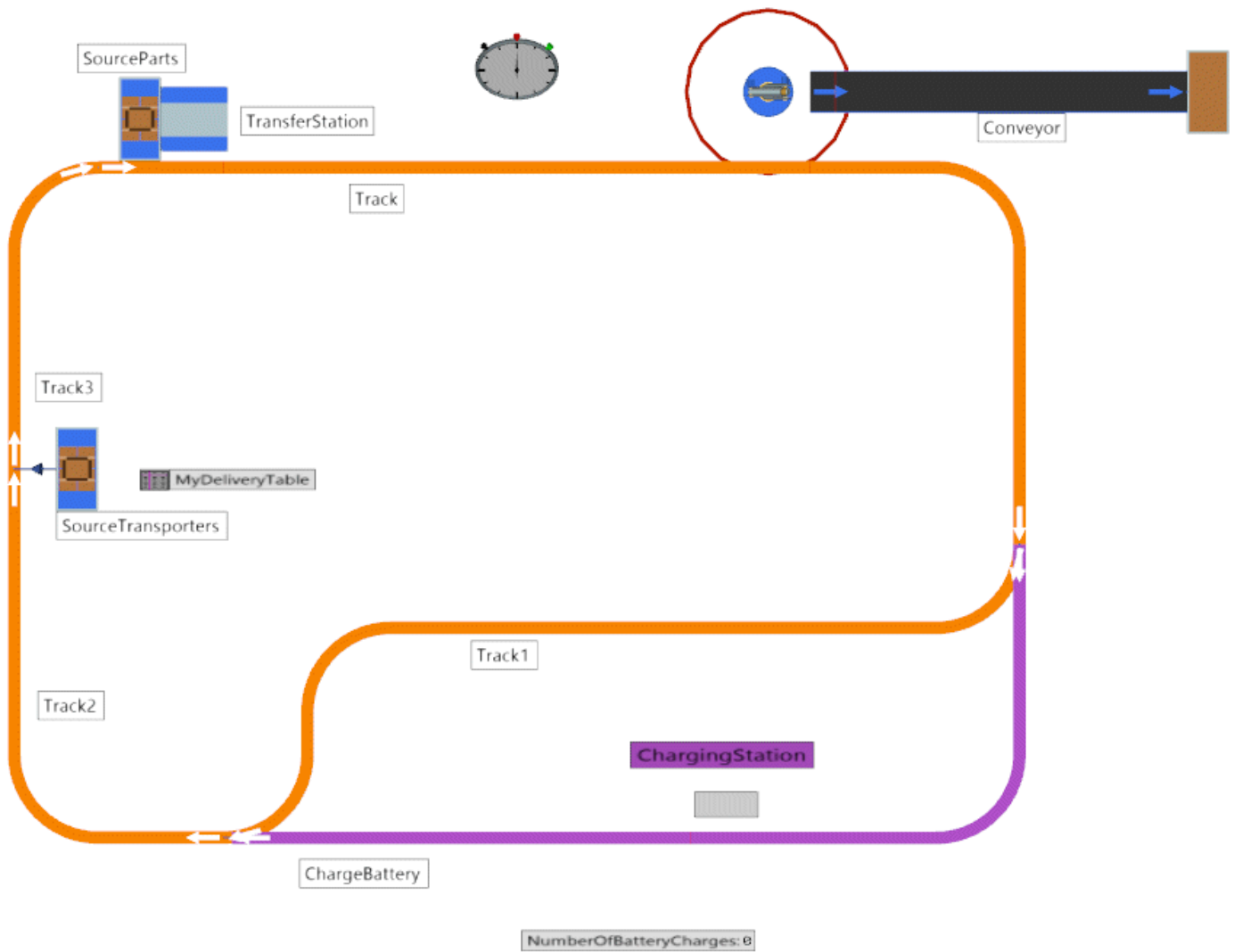
☒ Battery

Charge	3	Ah		
Basic Consumption	1	A	Driving Consumption	8 A
Capacity	8	Ah	Reserve	2 Ah
Charge Current	40	A		
Charge Control	self.OnCharge			...

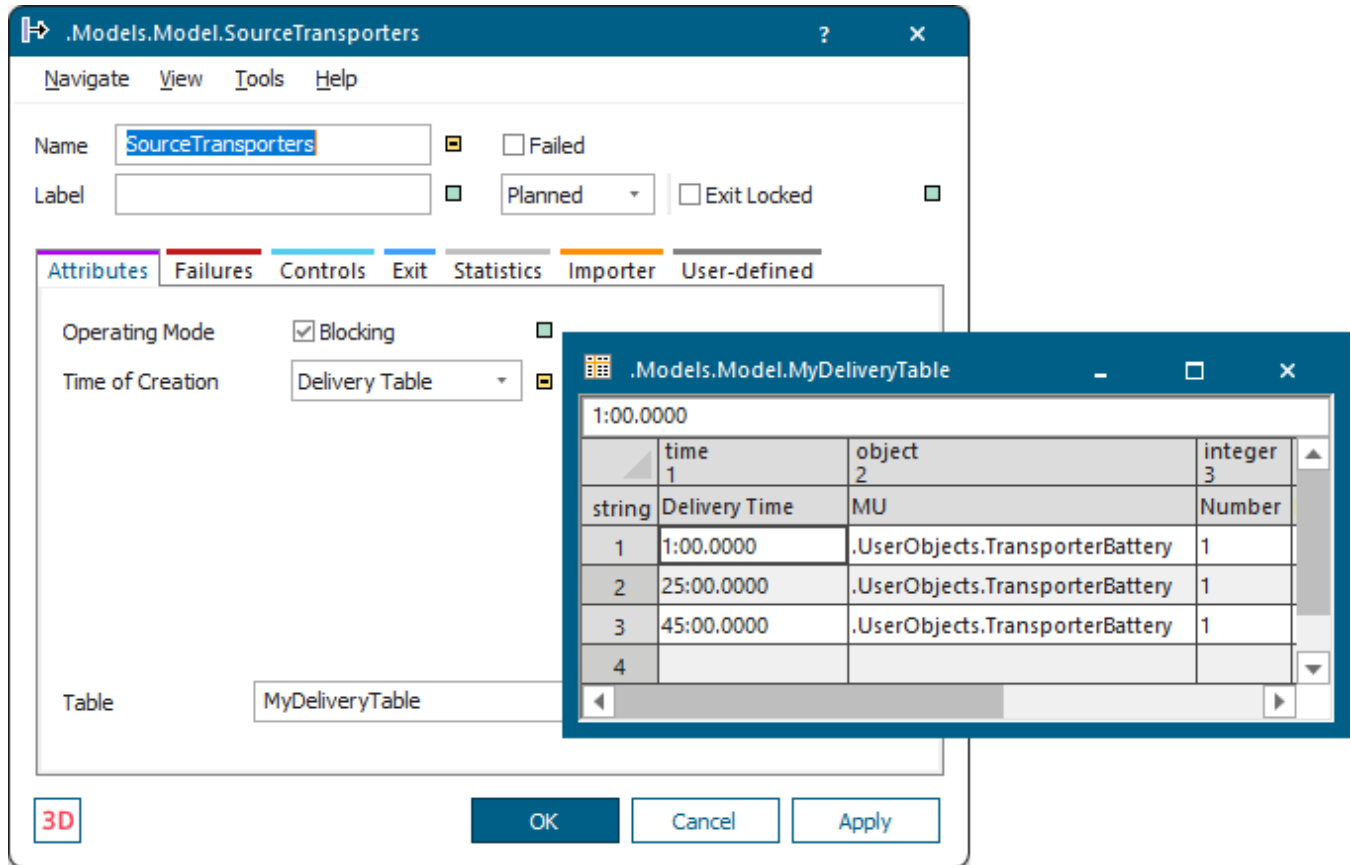
- We typed in these instructions into the **Charge Control**:

```
if @.BatCharge >= @.BatCapacity
    @.Stopped := false
end
```

Then we modeled the material flow and configured the material flow objects:



- We selected these settings in the *Source* which feeds the *Transporters* into the plant.

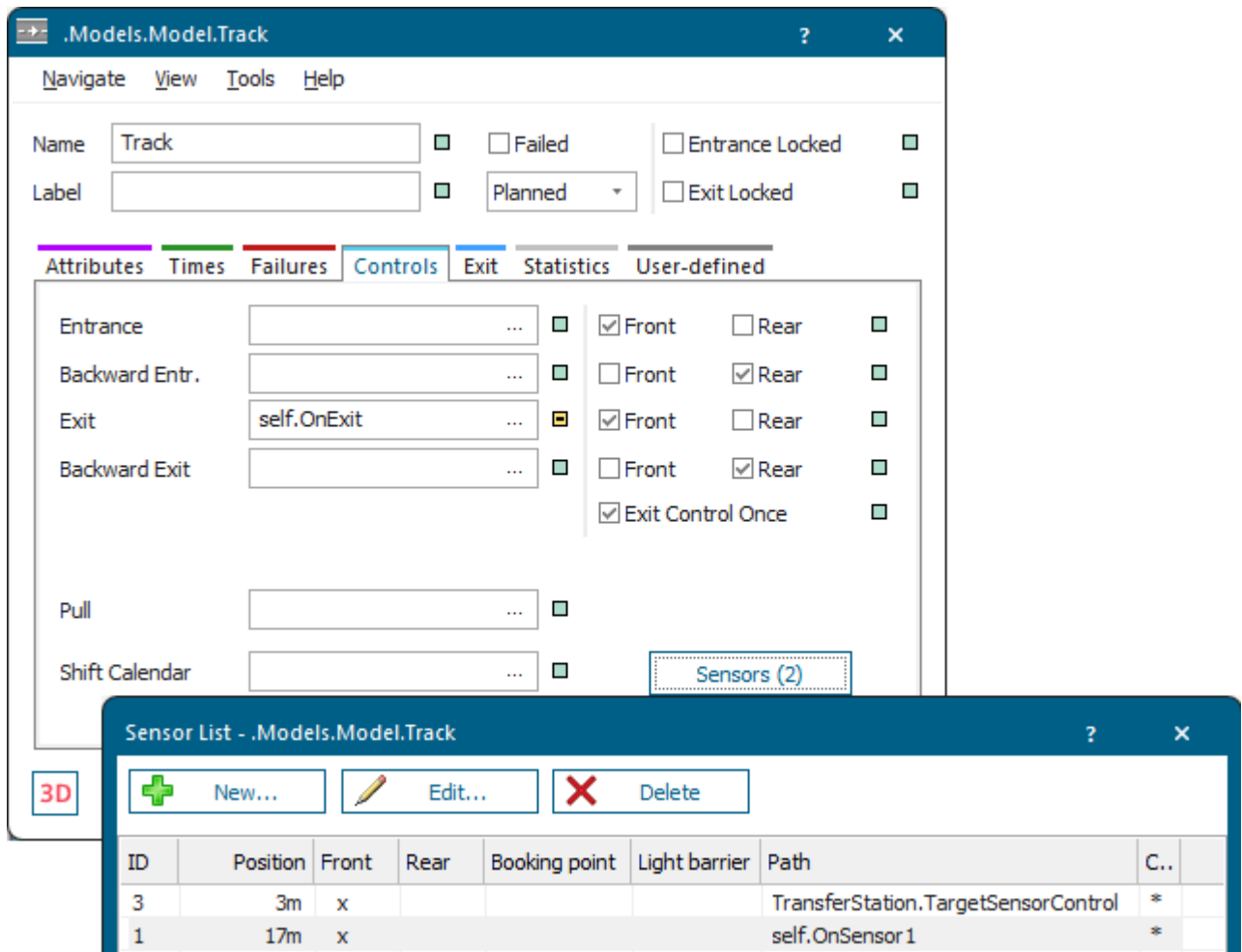


- We created an **Exit Control** in the object *Track*, which sends a *Transporter* with a low battery charge on the *Track*, ChargeBattery in our case, that leads to the charging station.

We typed in this source code:

```
if @.BatCharge <= @.BatReserve
  @.move(1)
else
  @.move(2)
end
```

- We created two **Sensors** on the object *Track* and selected these settings.



At the sensor with the **ID 3** the *TransferStation* loads the parts onto the loading space of the *Transporter*.

At the sensor with the **ID 1** the *PickAndPlace* robot unloads the parts from the loading space of the *Transporter* and places them on the *Conveyor* at the sensor. We typed in these instructions into the **Sensor Control** *OnSensor1*.

```
param SensorID: integer, Front: boolean, BookPos: boolean

// Activating the sensor moves the parts on to the designated
// destination, namely the PickAndPlace robot.

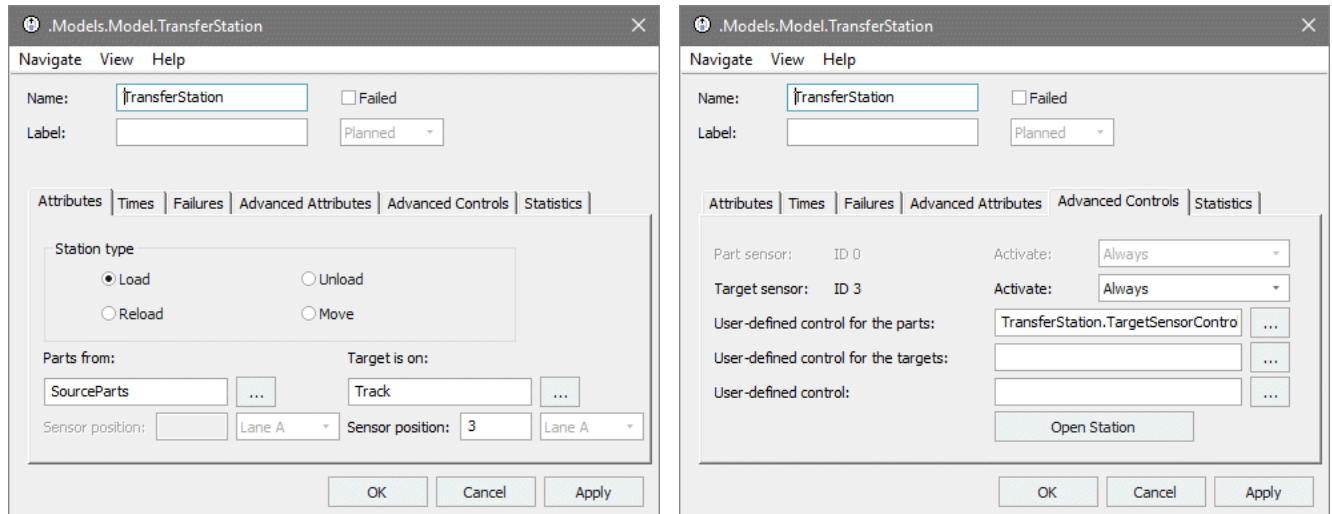
@.Stopped := true

var Destination:object := PickAndPlace

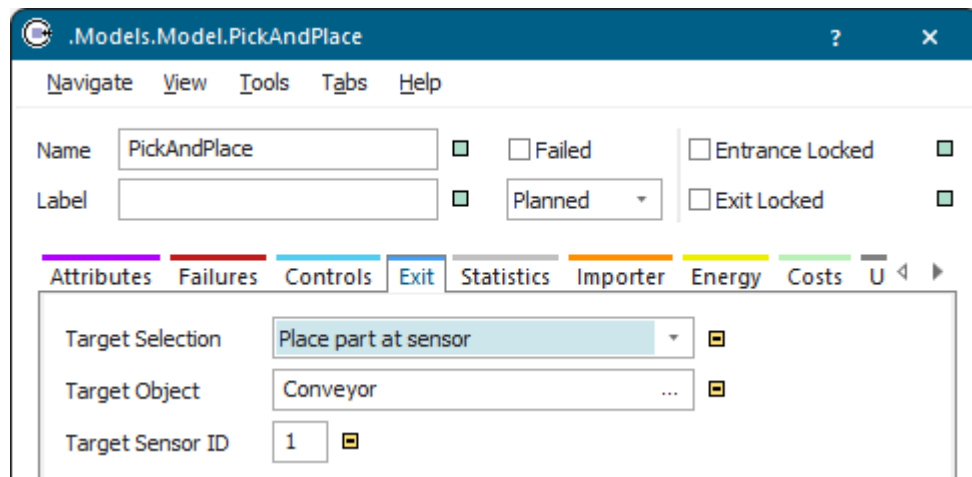
while not @.Empty
  var MU: object := @.cont
  if not MU.move(Destination)
    stopuntil MU.Location /= @ and not Destination.IsLoading
  end
```

```
end
@.Stopped := false
```

- The *SourceParts*, which produces the parts that the *Transporter* transports, we use with the default settings.
- For the *TransferStation* we selected these settings:



- For the *PickAndPlace* robot, which unloads the parts from the *Transporter* and places them on the *Conveyor*, we use a **Loading Time** and an **Unloading Time** of 2 minutes each and these settings on the tab **Exit**.



- For the *Conveyor* we created this sensor at which the *PickAndPlace* robot places the parts.

Sensors.ID1

Position: Length

0.6 [m]

☒ Front

☐ Rear

☐ Booking Point

☒ Light Barrier Mode

Activate: Always

Control: ...

- We created a **Sensor** in the *Track* named *ChargeBattery* and selected these settings:

.Models.Model.ChargeBattery

Name: ChargeBattery

Label:

☐ Failed ☐ Entrance Locked

☐ Planned ☐ Exit Locked

Attributes Times Failures **Controls** Exit Statistics User-defined

Entrance	...	☒ Front	☐ Rear	☒
Backward Entr.	...	☐ Front	☒ Rear	☒
Exit	...	☒ Front	☐ Rear	☒
Backward Exit	...	☐ Front	☒ Rear	☒
		☒ Exit Control Once		☒
Pull	...	☒		
Shift Calendar	...	☒		

Sensors (1)

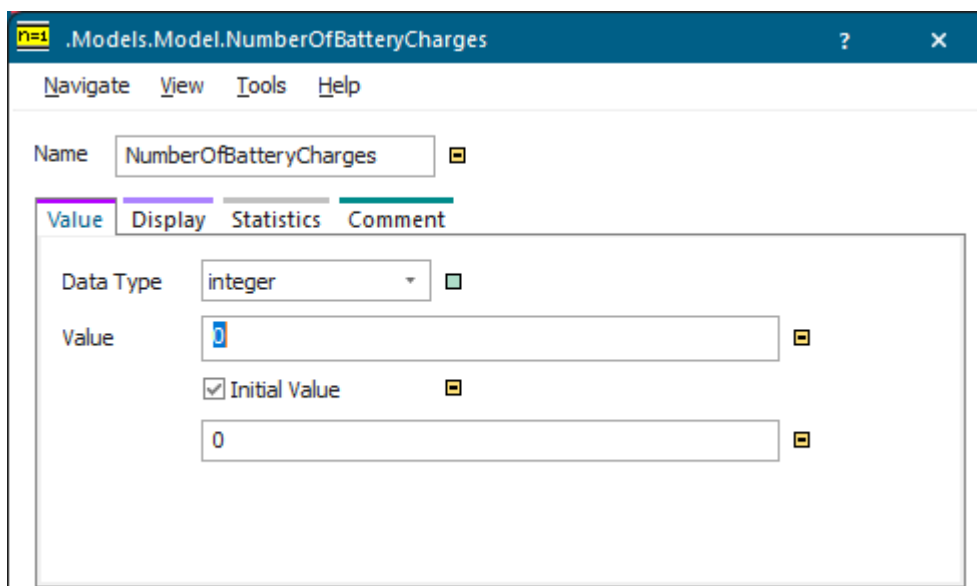
Sensor List - .Models.Model.ChargeBattery

ID	Position	Front	Rear	Book...	L...	Path	C..
1	14m	x				self.OnSensor 1	*

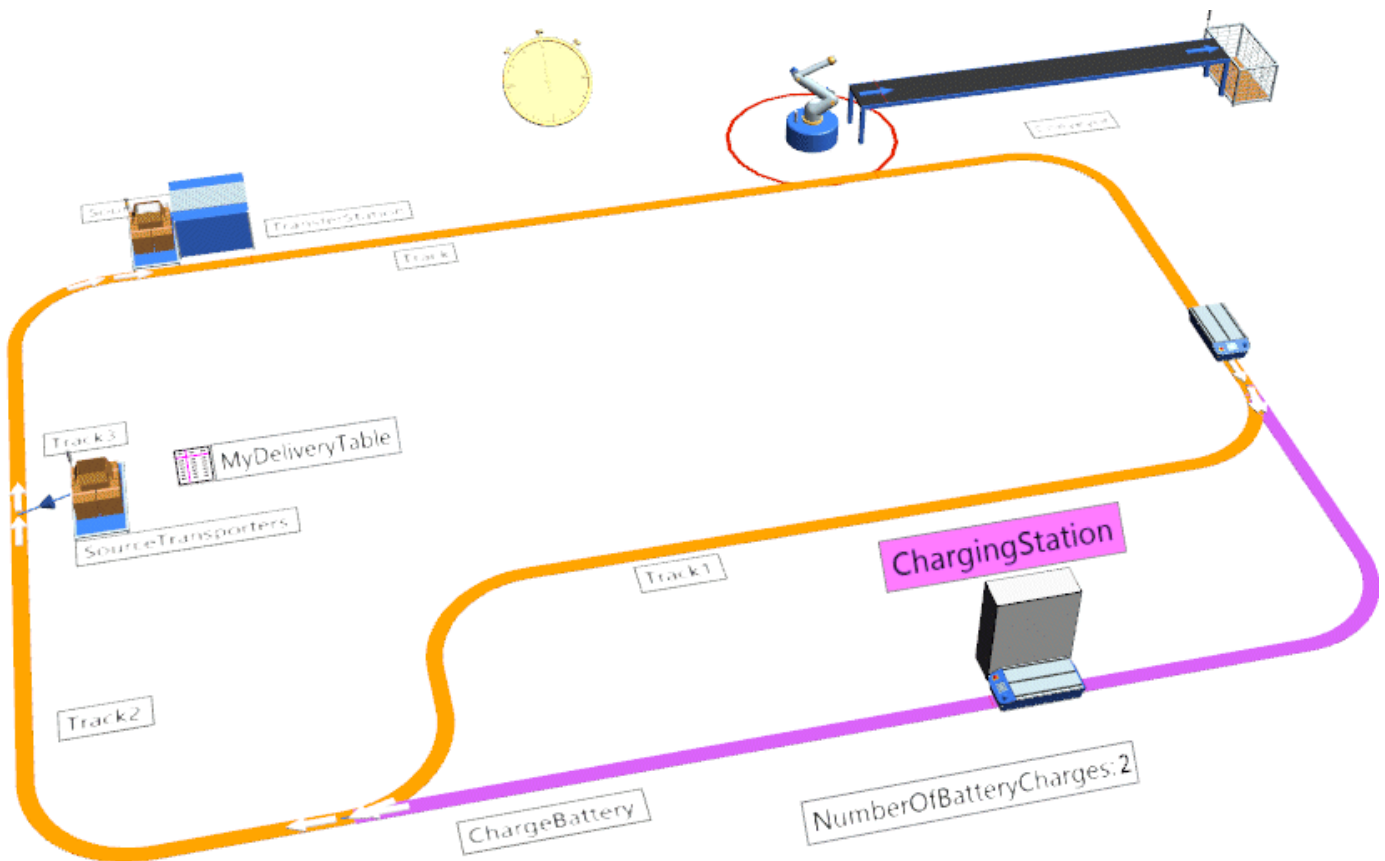
We typed in these instructions into the **Sensor Control**:

```
param SensorID: integer, Front: boolean, BookPos: boolean  
  
@.Stopped := true  
@.BatCharging := true  
  
NumberOfBatteryCharges += 1 // increments the number of battery charges  
                           // in the Variable
```

- We finally inserted a graphic representing the charging station and the *Variable* named `NumberOfBatteryCharges`, which shows how often the batteries were charged.



When we now run the simulation, it looks like this.



Compare Model a Battery-powered Transporter, Extended

Back to [Model Transport Systems](#)

Modeling a Battery-powered Transporter, Extended

Model a Battery-powered Transporter, Extended

With the settings on the tab **Battery** you can model a battery-powered *Transporter*.

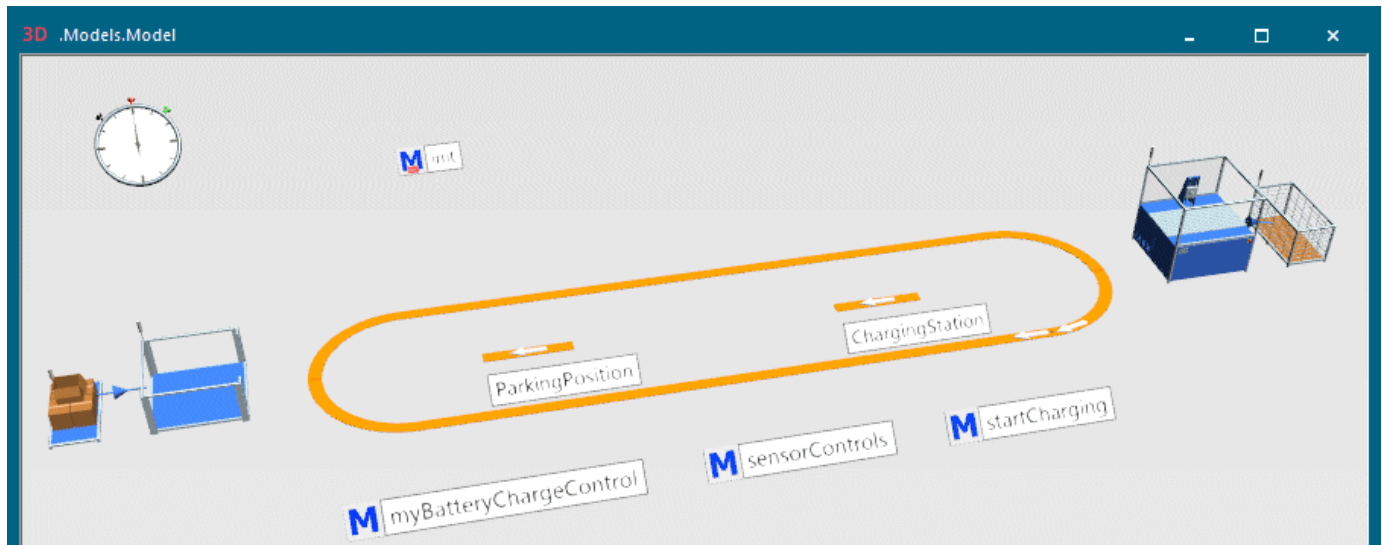
The **Battery** battery has to be charged when the battery charge is lower than the reserve charge of the battery. During battery operation power consumption consists of the **Driving Consumption** and the **Base Consumption**. Within the **Charge Control** of the *Transporter* we have to prevent that the battery gets discharged all the way and that the *Transporter* stops because of that. For charging the battery the *Transporter* requires **Charge Current**. The time recharging takes is the difference between the **Capacity** of the battery and its current charge and the charge current of the *charging station*.

Once the **Reserve Charge** during driving or the battery capacity at the end of charging is reached, the **Charge Control** of the *Transporter* is called. We can differentiate these two events in the **Charge Control** with the value of the battery charge using the attribute *BatCharge*.

We will demonstrate how to:

- [Model a Battery-powered Transporter, Extended](#)
- [Program the Sensor Control for Loading and Unloading the Parts](#)
- [Configure Battery-powered Operation of the Transporter](#)
- [Program the Battery Controls](#)

The finished simulation model looks like this:



Compare [Model a Battery-powered Transporter, Basic](#)

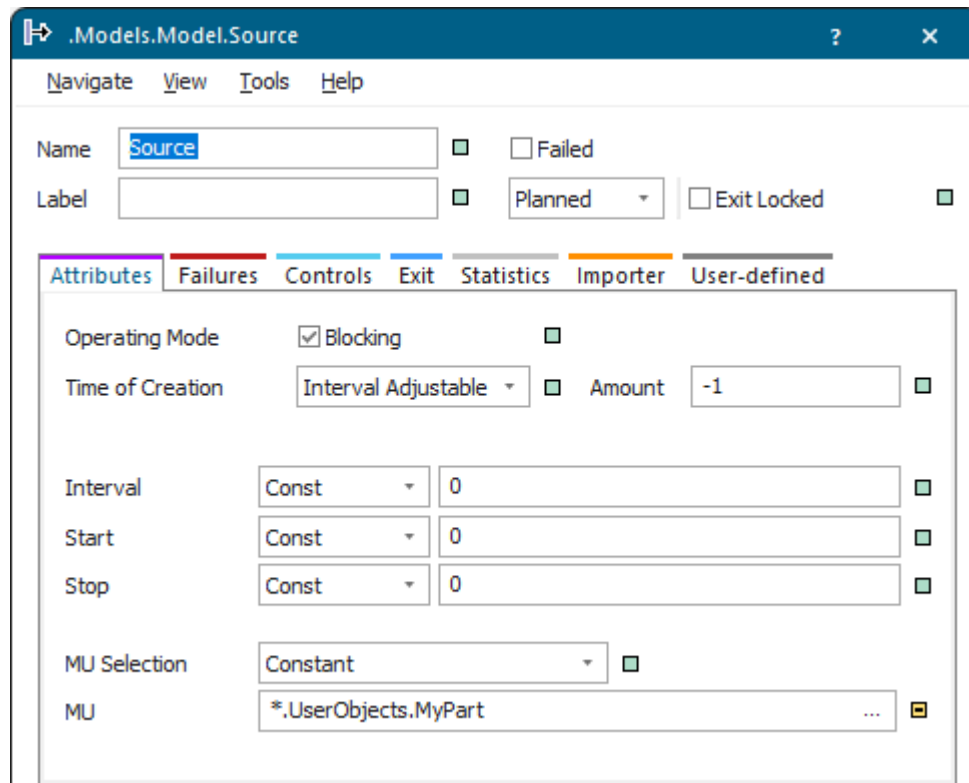
Back to [Model Transport Systems](#)

Model the Material Flow

In our first step we model the material flow for the battery-operated *Transporter*.

We proceeded as follows:

- The *Source* **constantly** creates parts of type *MyPart*.



- These are then moved on to the *Buffer*. We use the *Buffer* with its standard settings.
- For the *Track* we created two sensors with the settings below. The parts are loaded onto the *Transporter* at the *Buffer* and they are unloaded at the *Station*:

ID	Position	Front	Rear	Booking point	Light barrier	Path	Created here
1	17.2m	x				sensorControls	*
2	37.45m	x				sensorControls	*

Make sure that the starting point and the end point of the *Track* are connected so that the *Transporters* can drive around in a circle.

- We use the *Station* and the *Drain* with their respective standard settings.
- In addition we inserted a *Track* as the *ChargingStation* and another *Track* for the *ParkingPosition* of the *Transporter*.
- In our **Init Control** we create a *Transporter* each on the *Track* and on the *ParkingPosition*.

```
// create two Transporters
.UserObjects.MyTransporter.create(Track, 8)
.UserObjects.MyTransporter.create(ParkingPosition)
```

Back to [Model a Battery-powered Transporter, Extended](#)

Program the Sensor Control for Loading and Unloading the Parts

We would like to load the produced parts from the *Buffer* onto the *Transporter* at the first sensor and unload them from the *Transporter* onto the *Station* at the second sensor.

We programmed this in our **Sensor Control** as follows:

```
param SensorID: integer, Front: boolean, BookPos: boolean
// load parts at sensor 1 from the Buffer onto the Transporter
if sensorID = 1
  @.Stopped := true
  repeat
    waituntil Buffer.Occupied prio 1
    Buffer.cont.move(@)
    wait 3 // the loading time is 3 seconds
  until @.Full
  @.Stopped := false

// unload parts at sensor 2 from the Transporter onto the Station
elseif sensorID = 2
  @.Stopped:=true
  repeat
    waituntil Station.Empty prio 1
    @.cont.move(Station)
    wait 3 // the unloading time is 3 seconds
  until @.Empty
  @.Stopped := false
end
```

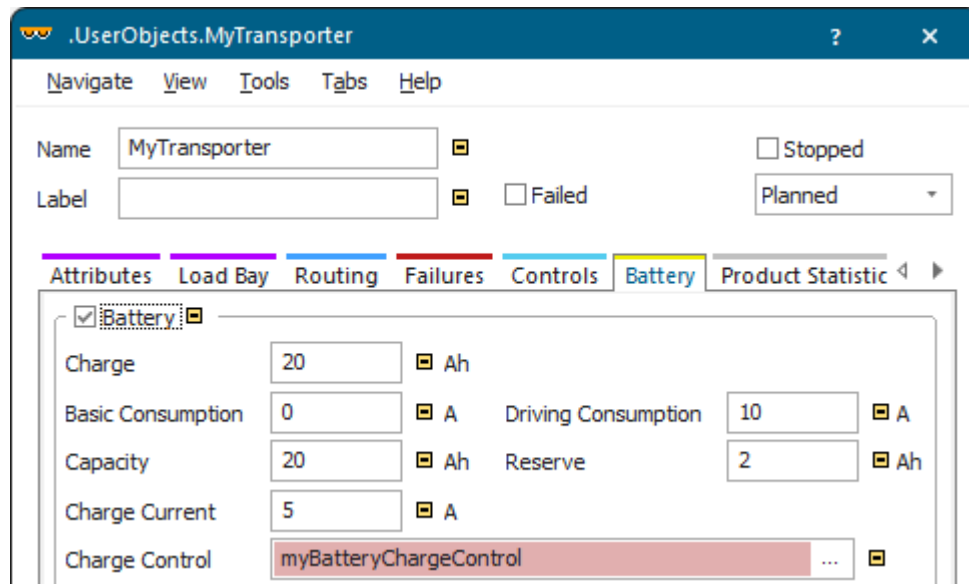
Back to [Model a Battery-powered Transporter, Extended](#)

Configure Battery-powered Operation of the Transporter

In this step we configure battery-powered operation of the *Transporter*.

Battery Settings

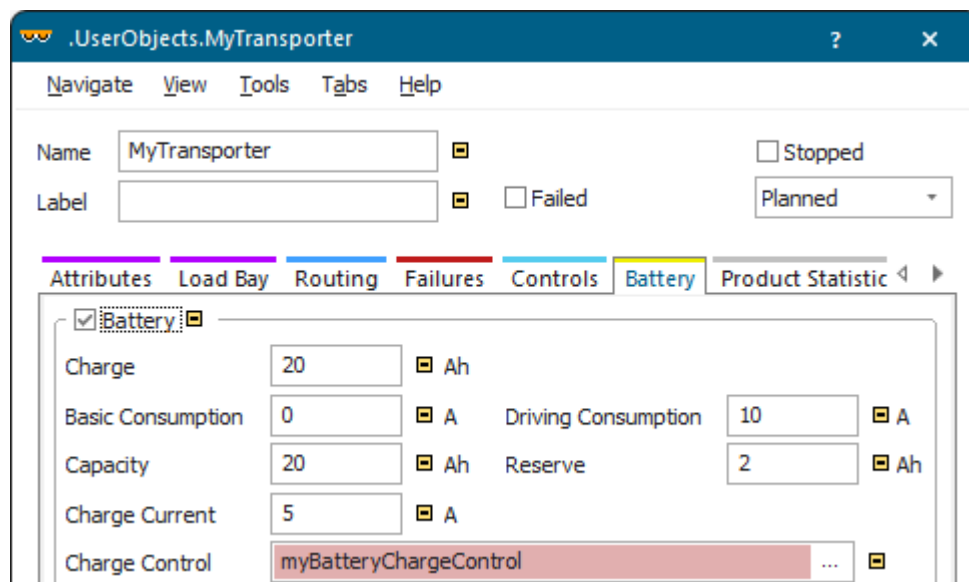
We entered the settings below on the **Tab Battery**:



We did not change other settings.

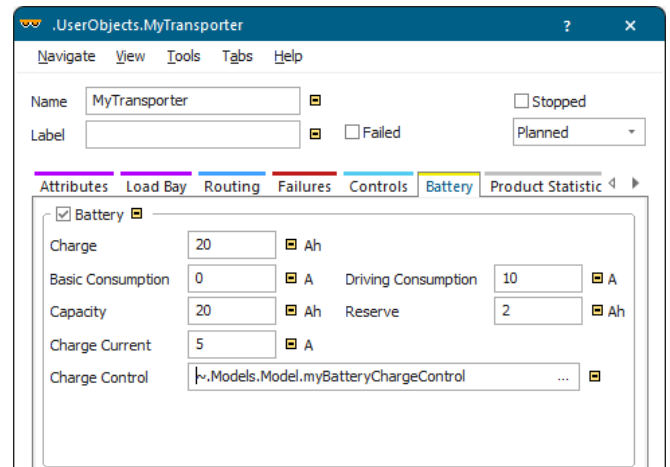
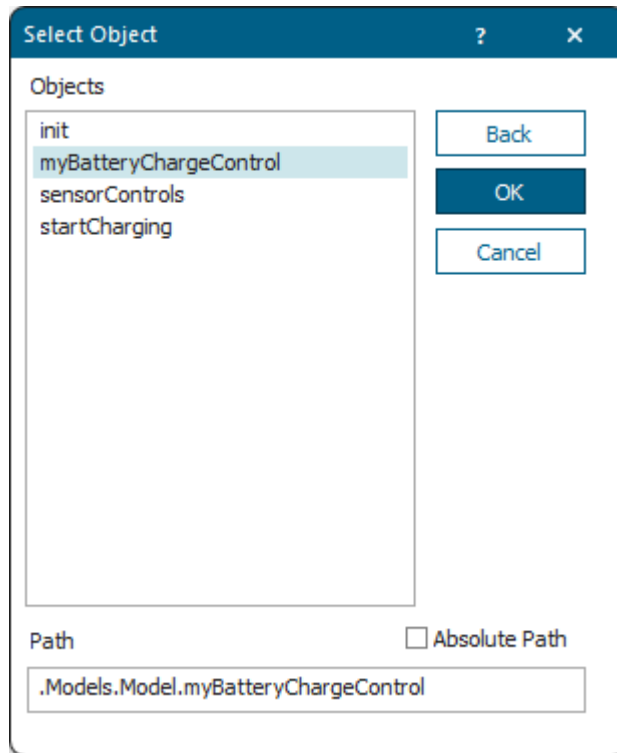
Excursus: The relative and the absolute path

For the **Charge Control** we just entered its name into the text box. Once we click **Apply**, *Plant Simulation* highlights the text box in red as it does not find the charge control within the namespace of the *Class Library* with the specified **relative path**. We accept that though as the *Transporters* are created in our *Frame* named **Model** and are thus located within the same namespace as the **Charge Control**.

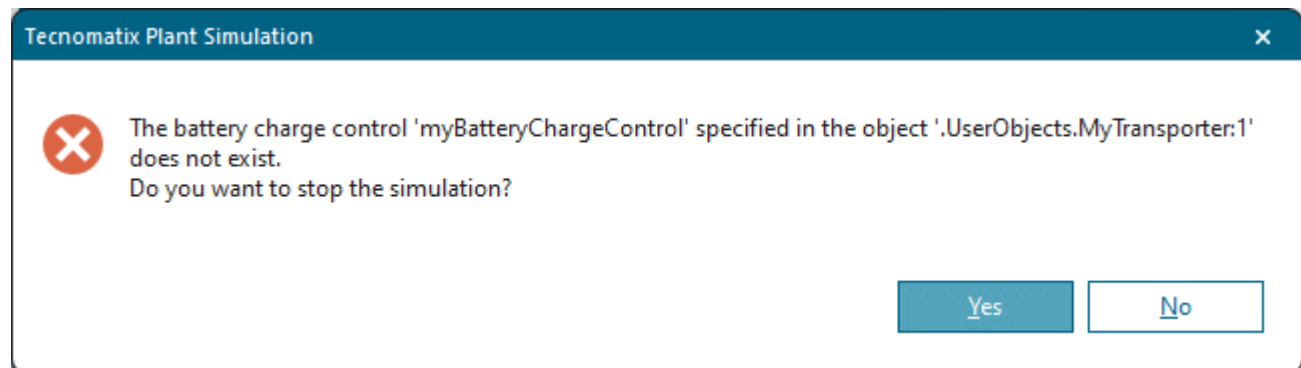


Note

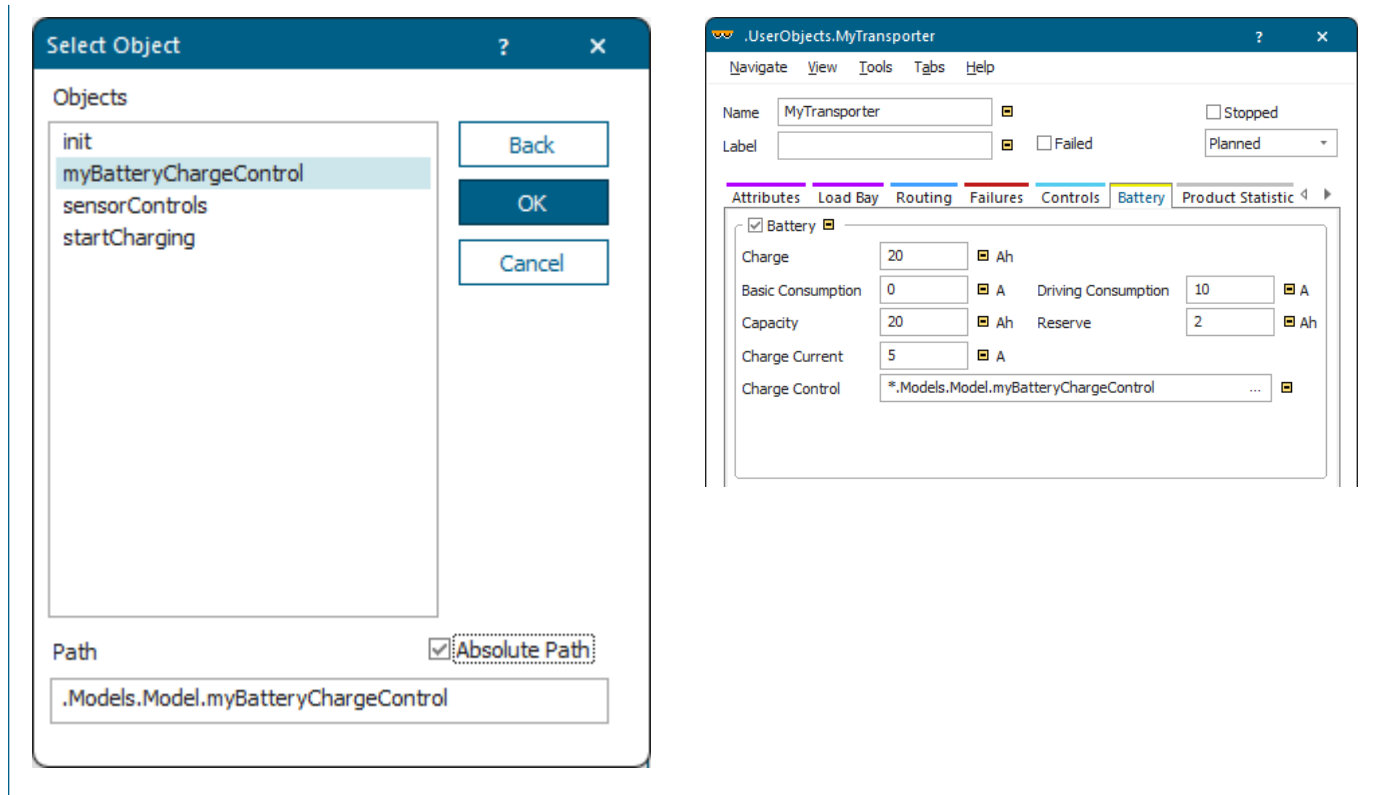
If we click the  button in the dialog of the *Transporter*, navigate to the *Frame* named **Model**, and select the charge control in the dialog **Select Object**, *Plant Simulation* inserts the **relative path** (designated by the tilde ~) to the charge control within the namespace of the *Frame*.



When we then run the simulation, *Plant Simulation* notifies us that the charge control does not exist in the class of the object within the namespace of the *Class Library*.

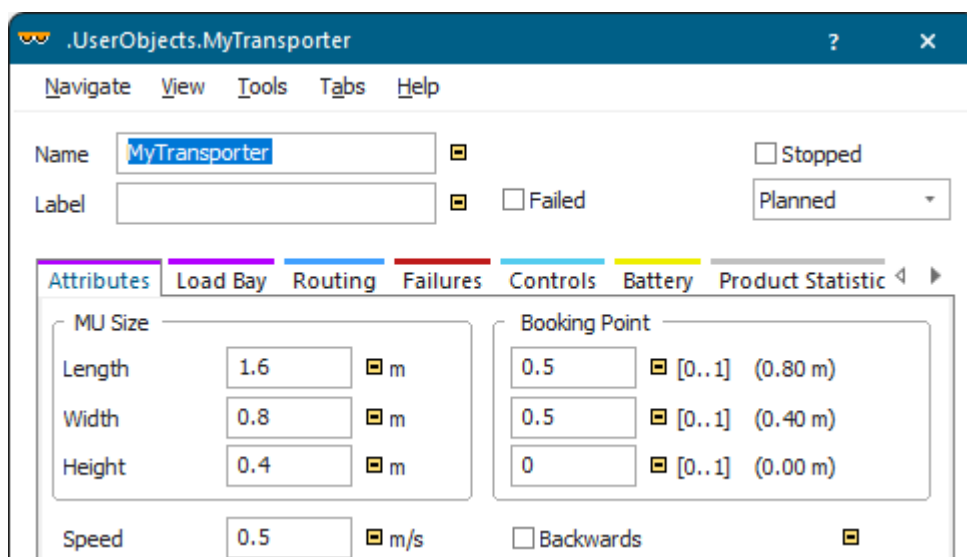


When we select the check box **Absolute Path**, *Plant Simulation* enters the **absolute path** (designated by the asterisk *) to the charge control. In this case *Plant Simulation* runs the simulation without errors. If we then rename the *Frame*, the absolute path will no longer work though. For this reason the relative path is to be preferred in most cases.

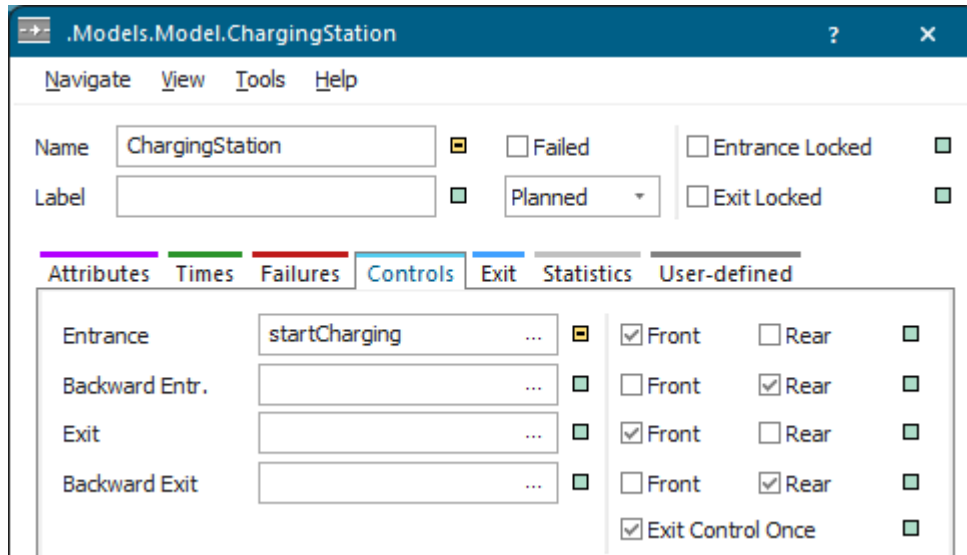


Additional Settings

We entered 0.5 m/s as the speed of the *Transporter*.



We entered our *startCharging* control as the **Entrance Control** into the *ChargingStation* of the *Transporter*.



For the *ParkingPosition* to which the *Transporter* with fully charged battery drives we use the standard settings of the *Track*.

Back to [Model a Battery-powered Transporter, Extended](#)

Program the Battery Controls

In the last step we program the **Charge Control** *myBatteryChargeControl*. It charges the battery and then, once it is fully charged, moves the *Transporter* onto the *ParkingPosition*.

```
// battery charge control of the Transporter, recognizes that the battery
// has to be charge,
// once the charge is smaller than or equal to the battery reserve
if @.BatCharge <= @.BatReserve
    @.BatCharging := true // start charging the battery
    @.transfer(ChargingStation)

elseif @.BatCharge >= @.BatCapacity
    @.move(ParkingPosition) // move Transporter with charged battery to
    ParkingPosition
end
```

Lastly we program the **Entrance Control** of the *ChargingStation*. It moves the fully charged *Transporter* from the *ParkingPosition* onto the *Track*.

```
// entrance control of the ParkingPosition
waituntil ParkingPosition.NumMU > 0 prio 1
ParkingPosition.Cont.move(Track, 9)
```

When we now run the simulation, we can trace how the *Transporters* circulate and first move onto the *ChargingStation* when the battery charge is low and then move on to the *ParkingPosition*.